



SCHOOL OF COMPUTING, ENGINEERING AND BUILT
ENVIRONMENT

HONOURS PROJECT FINAL REPORT

MHW224192-23-C-MAUR
2023/24C

**AdaptEd: Harnessing AI for Personalised and Multimodal
Learning in Higher Education**

SUPERVISOR

DR. STEPHEN NAICKEN SNAICKEN@ALUEDUCATION.COM

SUBMITTED ON 4TH, AUGUST, 2024

Author:
Ganiyat Adekunle

Student Number:
S2110996

**I declare that all work submitted for this coursework is the work of Ganiyat Adekunle, S2110996,
alone unless stated otherwise.**

Table of Contents

Abstract.....	7
Acknowledgement.....	8
1.0 Introduction.....	9
1.1 Project Background.....	9
1.2 Project Aims and Objectives.....	9
1.2.1 Problem Statement.....	9
1.2.2 Secondary Research Objectives.....	10
1.2.3 Primary Research Objectives.....	10
2.0 Literature and Technology Review.....	11
2.1 Literature review.....	11
2.1.1 The diverse learning needs and preferences of students.....	11
2.1.2 Traditional educational approaches and their limitations.....	11
2.1.3 The benefits and impact of personalised and multimodal learning approaches.....	12
2.1.4 The application of AI technologies in creating personalised and adaptive learning experiences.....	12
2.1.4.1 Evolution and the use of AI in Education.....	12
2.1.4.2 AI Technologies.....	12
2.1.4.3 Challenges and Considerations.....	13
2.1.5 Case studies and evaluations of existing adaptive learning platforms and AI-powered educational solutions.....	13
2.1.6 Best practices and guidelines for designing effective multimodal learning content.....	14
2.1.7 Challenges and considerations in integrating AI-powered solutions with Learning Management Systems (LMS).....	15
2.1.8 Evaluation and Assessment of Personalised Learning Systems.....	15
2.1.9 Research Gaps and Future Directions.....	16
2.2 Technology Review.....	17
2.2.1 Multimodal Content Generation Technologies.....	17
a) Text-to-Speech (TTS) Generation.....	17
b) Slide Generation.....	17
c) Video Generation.....	18
2.2.2 Recommender System.....	20
2.2.3 Learning Analytics and Early Prediction of At-risk student.....	21
2.2.4 AI-Powered Virtual Assistant.....	22
3.0 Methods.....	24
3.1 System Design Analysis.....	24

3.1.1 Architecture Overview.....	24
3.2 Tools and Technology.....	25
3.3 Agile Development Methodology.....	27
3.3.1 Agile Phases and Practices.....	28
3.4 Data Collection and Analysis.....	29
3.4.1 Data Collection.....	29
3.4.2 Data Analysis.....	29
3.5 Evaluation and Assessment.....	29
3.6 Ethical Considerations.....	30
4.0 Execution.....	31
4.1 Requirements Gathering and Analysis.....	31
4.1.1 Functional Requirements.....	32
4.1.2 Non-Functional Requirements.....	33
4.1.3 Use Cases.....	34
4.2 Design and Modelling.....	40
4.2.1 System Design.....	40
4.2.2 Virtual Assistant Design and Modelling.....	42
4.2.3 Recommender System Design and Modelling.....	44
4.2.4 Multimodal Content Generator Design and Modelling.....	46
4.3 Implementation.....	48
4.3.1 Docker and Moodle Set up.....	49
Database setup.....	49
NGINX set up.....	51
Moodle Set up.....	53
4.3.2 Virtual Assistant Implementation.....	55
4.3.3 Recommender System Implementation.....	60
4.3.4 Multimodal Content Generator Implementation.....	68
4.4 Testing.....	73
4.4.1 Functional Testing.....	73
4.4.2 Non-Functional testing.....	75
5.0 Evaluation.....	77
5.1 Context-Aware Chatbot Evaluation.....	77
5.1.1 Unit Testing.....	77
5.1.2 Functional Testing.....	77
5.1.3 Non-Functional Testing.....	78

5.2 Recommender System Evaluation.....	78
5.2.1 Performance Metrics.....	78
5.2.2 Hyperparameter Tuning.....	79
5.2.3 Functional and Non-Functional Testing.....	80
5.3 Multimodal Content Generation Evaluation.....	80
5.3.1 Unit Testing.....	80
5.3.2 Functional and Non-Functional Testing.....	80
5.4 Overall Evaluation and Broader Context.....	81
6. Conclusion and Future Work.....	82
6.1 Conclusion.....	82
6.1.1 Achievement of Project Objectives.....	82
6.1.2 Critical Analysis of Project Execution.....	82
6.1.3 Reflection on Project Results.....	83
6.1.4 Broader Implications.....	83
6.2 Future Work.....	83
Appendices.....	85
References.....	110

Table of Figures

Figure 1: VARK model (El-Sabagh, 2021, Fig 1.)	11
Figure 2: Benefit of Adaptive Learning Systems	14
Figure 3: Multimodal content generation service architecture	20
Figure 4: Techniques for early prediction of at-risk students	22
Figure 5: AdaptEd's system architecture	24
Figure6: survey summary 1	32
Figure7: survey summary 2	32
Figure8: Use case Diagram	35
Figure 9: System Architecture design	41
Figure 10: Activity Diagram for Virtual Assistant	43
Figure 11: Sequence Diagram for Virtual Assistant	44
Figure 12: Activity Diagram for Recommender System	45
Figure 13: Sequence Diagram for Recommender System	46
Figure 14: Activity Diagram for Multimodal Content Generator	47
Figure 15: Sequence Diagram for Multimodal Content Generator	48
Figure 16: Docker Set up Visualisation	49
Figure 17: Database Docker Set up	50
Figure 18: Database Tables	51
Figure 19: Database Table Definition with XML	51
Figure 20: Nginx Docker Set up	52
Figure 21: Nginx Configuration	53
Figure 22: Moodle Docker Set up	54

Figure 23: Moodle Dockerfile	54
Figure 24: Moodle PHP Configuration	55
Figure 25: Virtual Assistant Mustache	56
Figure 26: Backend Processing	57
Figure 27: local_adapted_chatbot Class	58
Figure 28: local_adapted_chatbot Class contd.	59
Figure 29: Moodle Authentication	59
Figure 30: Navigation Extension	60
Figure 31: MoodleRecommender Class	61
Figure 31: Data Preprocessing	62
Figure 32: Train CB	63
Figure 33: Content Based Recommendation	63
Figure 34: Train CF	64
Figure 35: Collaborative Filtering	64
Figure 36: Train Hybrid Model	64
Figure 37: Hybrid Recommendation	65
Figure 38: Tune Hyperparameters	65
Figure 39: Tune SVD	66
Figure 40: Tune Random Forest Regressor	67
Figure 41: Train and Save Models	68
Figure 42: Convert Text-to-speech	69
Figure 43: Generate Slides	70
Figure 44: Generate Video	71
Figure 45: Generated Video	71
Figure 46: Multimodal Conversion Server	72
Figure 47: Multimodal Conversion Job Processing	72
Figure 48: Recommender System Evaluation Result	79
Figure 49: Project timeline Gantt Chart	86

Table of Tables

Table 1: Methods for evaluating personalised learning systems	15
Table 2: Metrics for evaluating personalised learning systems	16
Table 3: Comparative analysis of TTS techniques	17
Table 4: Comparative analysis of text-to-slide generation techniques	18
Table 5: Comparative analysis of text-to-video generation techniques	19
Table 6: Comparative analysis of recommender systems	21
Table 7: comparative analysis of virtual assistants	23
Table 8: Justification for microservices architecture	25
Table 9: Justification for tools and technologies	25
Table 10: Justification for agile methodology	27
Table 11: Project timeline	28
Table 12: Functional Requirements	32
Table 13: Non-Functional Requirements	33
Table 14: Use Cases	34
Table 15: Use case: Query Virtual Assistant	36
Table 16: Use case: View Personalised Content Recommendations	37

Table 17: Use case: Generate Multimodal Content	38
Table 18: System Components	40
Table 19: Design Consideration for Virtual Assistant	42
Table 20: Design Consideration for Recommender System	44
Table 21: Design Consideration for Multimodal Content Generator	46
Table 22: Test Summary for functional requirements	73
Table 23: Test Summary for Non-Functional Requirements	75
Table24: Recommender System Evaluation Result	78
Table 25: Project Progress	85
Table 26: Product Backlog	85
Table 27: Functional Testing	100
Table 28: Non-Functional Testing	104

Abstract

Delivering personalised and engaging learning experiences remains a challenge as institutions struggle to meet the diverse needs of students. The AdaptEd project aims to develop an AI-powered Learning Management System (LMS) plugin to address these challenges. By integrating natural language processing, machine learning, and multimedia content generation, AdaptEd provides multimodal content, personalised content recommendations, and proactive support for students. The literature and technology review span diverse learning needs, traditional educational limitations, and AI applications and technologies in personalised learning. A mixed-methods evaluation will combine quantitative performance metrics with qualitative feedback to assess its effectiveness. Expected outcomes include improved student engagement, higher retention rates, and enhanced academic performance.

Keywords: AI in education, learning styles, personalised learning, multimodal learning, learning analytics, recommender systems, machine learning

Acknowledgement

I would like to express my deepest gratitude to my project supervisor, Dr. Stephen Naicken, for his unwavering support, guidance, and encouragement throughout the development of this project.

1.0 Introduction

1.1 Project Background

Higher education faces significant challenges in delivering effective learning experiences tailored to diverse student needs, preferences, and abilities. Traditional one-size-fits-all teaching methods result in disengagement and suboptimal outcomes, with up to 40% of students feeling disconnected from their coursework (Popenici & Kerr, 2017). Existing Learning Management Systems (LMS) lack advanced personalisation capabilities, placing a significant burden on educators who spend over 10 hours weekly on content customisation and student support (Attri et al., 2023). Moreover, these systems often fail to proactively identify and support at-risk students, contributing to higher dropout rates, with about 30% of undergraduates leaving within their first year (NCES, 2020).

The goal of this project is to address these critical issues by developing AdaptEd, an AI-powered LMS plugin designed to provide personalised content recommendations, multimodal content delivery, and proactive student support. Utilising advanced technologies such as natural language processing and machine learning, AdaptEd dynamically adapts educational content to meet individual student needs. It also includes a recommender system and learning analytics service to identify at-risk students and offer actionable insights for educators. By integrating these technologies, AdaptEd aims to enhance student engagement, comprehension, and academic success, filling the gaps in current educational models (Holmes et al., 2019; Luckin et al., 2016).

1.2 Project Aims and Objectives

1.2.1 Problem Statement

The current landscape of higher education faces significant challenges in delivering effective and engaging learning experiences that cater to the diverse needs, preferences, cognitive abilities, accessibility requirements, and learning styles of students. Traditional one-size-fits-all approaches to teaching and learning often fail to address the individual differences among learners, leading to disengagement, suboptimal learning outcomes, and an increased risk of academic failure. While some adaptive learning technologies exist, they are frequently fragmented and not seamlessly integrated with popular LMS platforms. They require significant manual effort from educators and fail to provide a comprehensive, AI-driven personalised learning experience, limiting their effectiveness and scalability. The lack of personalised support and timely intervention for students who may be struggling or at risk of falling behind further exacerbates these challenges.

Existing learning management systems (LMS) and educational technologies, while powerful in their own right, often lack the advanced capabilities necessary to deliver truly personalised and adaptive learning experiences. These systems often rely on manual intervention from educators to tailor content and provide individualised support, which can be time-consuming, resource-intensive, and difficult to scale. The absence of sophisticated learning analytics and predictive modelling tools within these systems hinders the ability to identify and proactively support students who may be at risk of academic failure. To address these challenges, there is a critical need for an intelligent, AI-powered LMS plugin that can seamlessly integrate with existing platforms and deliver highly personalised and engaging learning experiences.

The proposed AdaptEd plugin aims to bridge this gap by utilising AI technologies for personalised content recommendations, multimodal content delivery, and proactive student support. AdaptEd will leverage technologies such as natural language processing, machine learning, and multimedia.

generation to dynamically adapt educational content based on individual student needs and preferences. Additionally, it will feature a recommender system for tailored learning materials and a learning analytics service to identify at-risk students and provide actionable insights for educators.

1.2.2 Secondary Research Objectives

1. Examine the cognitive and affective factors influencing learning styles and preferences, using insights from educational psychology, cognitive science, and cross-cultural studies.
2. Critique traditional educational models to identify limitations and the need for adaptive approaches through a review of historical and contemporary practices.
3. Analyse the impact of personalised and adaptive learning on educational equity and academic performance.
4. Investigate the evolution and integration of AI technologies in personalised learning, focusing on multimodal content, educational recommender systems, and learning analytics through literature and case studies.
5. Conduct a comparative analysis of personalised and adaptive learning platforms to evaluate their strengths, limitations, and effectiveness in improving student engagement and performance.
6. Examine the role of instructional design principles, multimedia learning theories, and user-centred design in developing engaging and personalised learning experiences.
7. Investigate challenges in integrating AI-powered solutions with existing LMS, ensuring interoperability, data privacy, and accessibility.
8. Examine frameworks and metrics for evaluating personalised learning systems.
9. Investigate the ethical and privacy implications of using student data in personalised learning, recommender systems, and learning analytics, examining relevant regulations and best practices in data governance and privacy protection.

1.2.3 Primary Research Objectives

1. Develop an intelligent and context-aware recommender system to suggest educational resources based on learning preferences and similarities with other learners, using advanced filtering techniques and evaluating its effectiveness through experimentation and user studies.
2. Design and develop a multimodal content generator that dynamically converts text-based educational content to audio, slides, and video using machine learning and multimedia generation techniques while adhering to instructional design principles.
3. Develop an AI-driven virtual assistant to provide personalised guidance, support, and feedback to students using natural language processing and conversational AI technologies.
4. Construct a learning analytics service to monitor and analyse student engagement, performance, and progress, using machine learning and statistical techniques to identify patterns, predict at-risk students, and provide actionable insights while ensuring data privacy and ethical considerations.
5. Integrate AdaptEd with popular LMS, ensuring interoperability, performance, and scalability through industry-standard protocols and APIs, with rigorous testing to ensure compatibility and a cohesive user experience.
6. Evaluate AdaptEd through a mixed-methods approach, combining quantitative analysis of student performance and engagement metrics with qualitative insights from user studies, focus groups, and educator feedback to assess its effectiveness in improving personalised learning and supporting at-risk students.

2.0 Literature and Technology Review

2.1 Literature review

2.1.1 The diverse learning needs and preferences of students

Understanding the diverse learning needs and preferences of students is crucial for effective education. Learning styles, such as visual, auditory, and kinesthetics' modalities, significantly impact student performance. Research into learning styles, such as VARK (Visual, Auditory, Reading/Writing, Kinesthetics), and Kolb's Experiential Learning Theory highlights the importance of tailoring educational strategies to different preferences and cognitive abilities (Kolb, 2014). The VARK model categorises learners to help educators develop strategies that engage students in the ways they learn best (Fleming and Mills, 1992). Figure 1 demonstrates the VARK model.

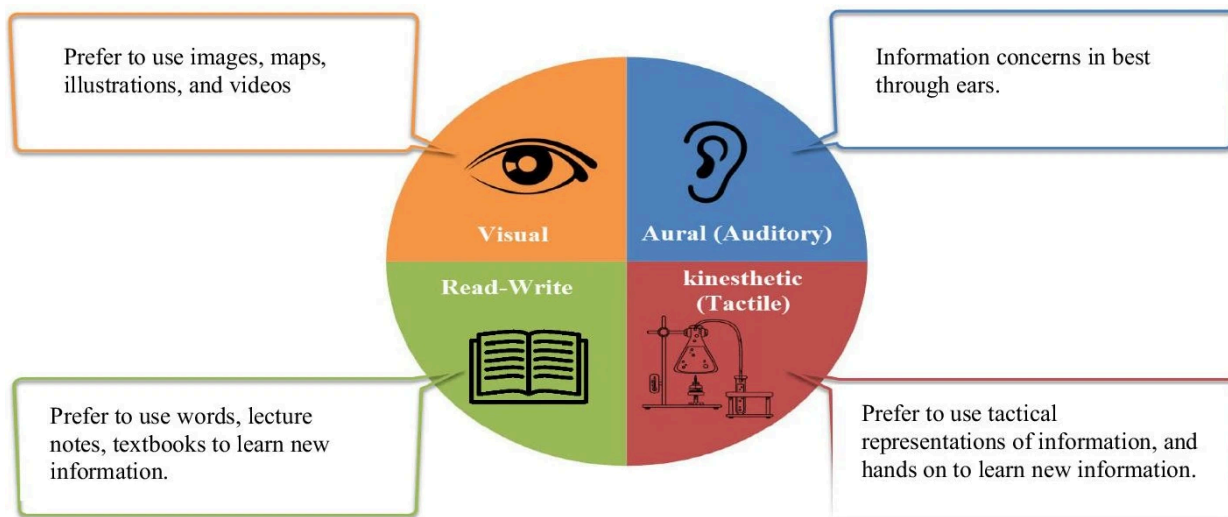


Figure 1: VARK model (El-Sabagh, 2021, Fig. 1.)

Research by Rahman and Budivanto (2019) demonstrates that adaptive e-learning systems, which align educational content with these styles, enhance student engagement and knowledge retention. However, the validity of learning styles as a concept is contested. An and Carr (2017) argue that tailoring education to specific learning styles lacks empirical support and suggest focusing on cognitive and metacognitive strategies instead. Cognitive abilities and processing speeds further differentiate learners. For instance, students with higher cognitive abilities process information more quickly, necessitating advanced content, while those with slower processing speeds benefit from additional support. Accessibility requirements also vary, with some students needing assistive technologies or adapted materials to fully participate in learning activities. The difficulty in early identification of at-risk students compounds these challenges as educators struggle to discern and address individual support needs (Khamparia & Pandey, 2020). This gap emphasises the need for personalised and adaptive learning systems that can dynamically cater to diverse student profiles, thereby enhancing engagement and reducing the risk of academic failure. Critiquing traditional educational models, which often fail to provide personalised support and timely interventions, sets the stage for addressing these varied needs.

2.1.2 Traditional educational approaches and their limitations

Historically, educational practices such as lecture-based instruction, standardised testing, and fixed curricula prioritised uniformity and standardisation, aiming to streamline teaching processes. Although education systems have evolved from rigid, standardised methods to more flexible and student-centred

approaches, many traditional models still lack the adaptability required to cater to individual learning preferences and cognitive abilities (Attri et al., 2023). This rigidity frequently leads to disengagement and suboptimal learning outcomes among students whose needs are not met by conventional methods (Hashim et al., 2022). Lecture-based instruction often fails to engage students, while standardised testing assesses all students using the same metrics, ignoring individual strengths and weaknesses. Additionally, traditional methods struggle to identify and support at-risk students promptly, exacerbating challenges related to academic performance and retention (Albreiki et al., 2023). The lack of sophisticated learning analytics and predictive modelling tools further hampers the ability to provide timely and effective interventions, highlighting the need for more advanced and integrated solutions (Adiguzel, Kaya, & Cansu, 2023). Also, current methods often rely heavily on manual efforts to tailor support, which is resource-intensive and not scalable. This inadequacy points to a pressing need for adaptive learning technologies that can offer personalised and responsive educational experiences, seamlessly integrating with existing LMS platforms.

2.1.3 The benefits and impact of personalised and multimodal learning approaches

Personalised and multimodal learning approaches, leveraging technologies like intelligent tutoring and recommender systems, provide tailored experiences (Zawacki-Richter et al., 2019). They integrate visual, auditory, and kinesthetics materials, catering to diverse learning preferences and enhancing retention (Blikstein, 2013). Studies suggest that adaptive learning systems significantly improve outcomes by accommodating different learning styles and abilities (Graf et al., 2007). Real-time analytics in personalised systems enable prompt intervention for at-risk students, contrasting with reactive traditional models (Al-Shabandar et al., 2019). These approaches enhance engagement and academic success and promote inclusive education, showcasing the transformative potential of AI integration in education.

2.1.4 The application of AI technologies in creating personalised and adaptive learning experiences

2.1.4.1 Evolution and the use of AI in Education

The evolution and integration of AI technologies in education have the potential to revolutionise personalised and adaptive learning experiences. As AI technologies continue to evolve, they can address the limitations of traditional educational models and create more inclusive and effective learning environments. AI in Education (AIEd) leverages advanced algorithms to adapt instructional content, identify at-risk students, and provide real-time feedback, making learning more responsive and tailored to individual needs. AI applications in education have evolved from simple automated grading systems to sophisticated platforms that offer personalised learning experiences. Early implementations focused on automating administrative tasks, such as grading and attendance, to reduce the burden on educators. However, recent advancements have enabled the development of intelligent tutoring systems (ITS), adaptive learning platforms, and learning analytics tools that provide more nuanced and personalised educational experiences (Holmes et al., 2019).

2.1.4.2 AI Technologies

Multimodal Content Generation

AI significantly advances multimodal content generation by integrating text, audio, video, and interactive elements to accommodate diverse learning preferences. Natural Language Processing (NLP) generates text summaries, explanations, and interactive dialogue-based materials, while machine learning algorithms analyse large datasets to personalise content for individual student needs (Zawacki-Richter et al., 2019). This is particularly impactful in STEM fields, where AI-generated

simulations and interactive labs provide hands-on experience and visualisations of complex concepts (Kaplan & Haenlein, 2019).

Educational Recommender Systems

AI enhances education through recommender systems, which utilise machine learning to suggest personalised learning materials, resources, and activities. These systems consider past performance, learning pace, and specific difficulties to tailor suggestions, thereby improving learning efficiency and engagement (Manouselis et al., 2011). They save educators time by automating the identification of suitable materials, allowing more focus on instruction, and enabling students to discover new resources and learning paths (Santos & Boticario, 2015).

Learning Analytics and Identifying At-Risk Students

AI plays a crucial role in learning analytics, particularly in identifying at-risk students. Learning analytics involves collecting, analysing, and reporting data to optimise learning environments. AI-powered predictive modelling and early warning systems detect patterns indicating academic struggles, such as low engagement or poor performance (Siemens & Long, 2011). Early identification facilitates targeted interventions, like tutoring sessions, to prevent academic failure, contrasting with traditional reactive methods where interventions often occur after students have already fallen behind (Arnold & Pistilli, 2012).

AI-Driven Virtual Assistants in Education

AI-driven virtual assistants, such as chatbots, provide real-time support and enhance learning experiences using NLP to respond to student queries, offer explanations, and guide learners through content. Available 24/7, these assistants offer support outside regular hours and reduce educators' workloads by handling routine queries and providing immediate feedback (Vázquez-Cano et al., 2021; Kiptonui, 2013).

2.1.4.3 Challenges and Considerations

The integration of AI in education offers significant potential benefits but also presents several challenges and considerations. Ensuring the ethical use of AI, particularly regarding data privacy and security, is a major challenge, necessitating compliance with legal standards like the GDPR and adherence to ethical guidelines to protect student privacy (Popenici & Kerr, 2017). Algorithmic bias is also a concern, as AI systems can reinforce existing biases if trained on unrepresentative datasets, highlighting the need for diverse and comprehensive data to ensure fair educational outcomes (Holmes et al., 2019). Additionally, the adoption of AI requires substantial investment in infrastructure and professional development, with educators needing training in both technical skills and the ethical and pedagogical implications of AI in education (Luckin et al., 2016).

2.1.5 Case studies and evaluations of existing adaptive learning platforms and AI-powered educational solutions

The transformative potential of AI in education is evidenced by numerous case studies. The ASSISTments platform, for instance, provides personalised feedback for math problems, effectively addressing individual learning gaps and significantly enhancing student performance (Heffernan & Heffernan, 2014). Similarly, OpenStax Tutor leverages AI and open educational resources to create customised learning experiences, resulting in increased student engagement and improved academic outcomes (OpenStax, 2020).

Carnegie Learning's MATHia software exemplifies AI-driven adaptive learning, creating individualised learning paths. A study involving over 1,000 middle school students revealed that those using MATHia

scored 20% higher on standardised math tests compared to their peers using traditional methods (Ritter et al., 2007). Despite these benefits, issues of algorithmic fairness and potential biases in the training data pose significant challenges.

Rahman and Budivanto (2019) demonstrated an adaptive e-learning system tailored to visual, auditory, and kinesthetics learning styles. Their implementation for 150 students resulted in a 30% increase in engagement and a 25% improvement in knowledge retention. However, the system's reliance on accurate initial learning style assessments can lead to suboptimal content delivery if misidentified. Khamparia and Pandey (2020) reviewed various adaptive e-learning systems and reported a 40% reduction in dropout rates and a 35% increase in engagement. Despite these advantages, they highlighted the complexities and costs associated with implementing such systems, which can pose financial and technical barriers to widespread adoption.

Graf et al. (2007) explored the Felder-Silverman learning style dimensions in adaptive learning systems. Their analysis of 200 students showed a 20% improvement in learning outcomes and a 15% increase in satisfaction. However, they raised concerns about privacy due to continuous monitoring and data collection, highlighting the need for robust data security and student consent.

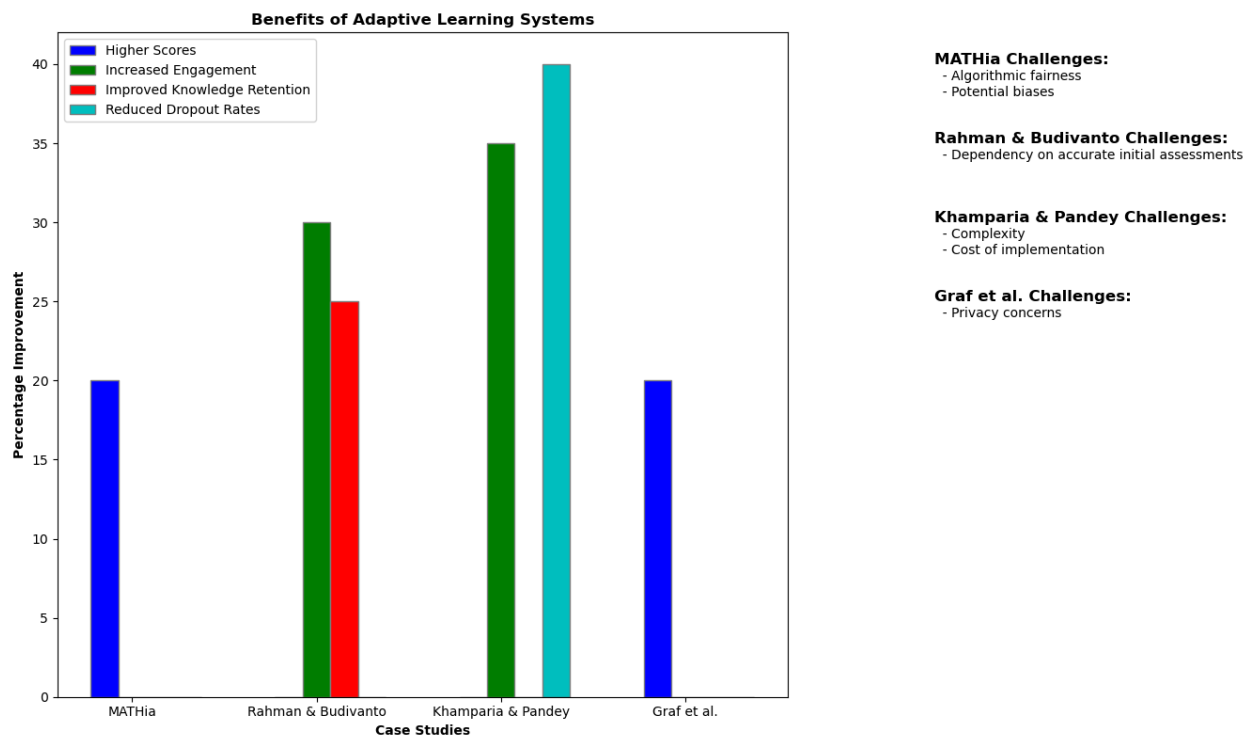


Figure 2: Benefits of Adaptive Learning Systems

These case studies illustrate the significant benefits of adaptive learning platforms for enhancing engagement and academic performance through personalised experiences. Nonetheless, they also highlight critical challenges, including the need for accurate assessments, financial and technical barriers, and privacy concerns. Addressing these issues is essential to fully realising the potential of AI-powered adaptive learning technologies in education (Popenici & Kerr, 2017).

2.1.6 Best practices and guidelines for designing effective multimodal learning content

Designing effective multimodal learning content requires adherence to instructional design principles, cognitive load management, and accessibility considerations. Instructional design principles, such as

those outlined by the Universal Design for Learning (UDL) framework, emphasise the need for flexible and inclusive educational materials that accommodate diverse learning needs (Bontchev & Vassilev, 2012). Cognitive load management involves structuring content to avoid overwhelming students with excessive information and ensuring that learning materials are presented in a clear and manageable format (Hamal et al., 2022).

Accessibility considerations are also crucial in creating inclusive educational environments. This includes designing content that is accessible to students with disabilities, using assistive technologies, and ensuring compliance with accessibility standards (Boesl, Achtenberg, & Bergler, 2023). By incorporating these best practices, educators can create multimodal learning experiences that are engaging, effective, and inclusive.

2.1.7 Challenges and considerations in integrating AI-powered solutions with Learning Management Systems (LMS)

Integrating AI-powered solutions with learning management systems (LMS) poses several technical, ethical, and privacy challenges. Ensuring seamless interoperability between AI tools and LMS platforms requires adherence to technical standards and protocols like Learning Tools Interoperability (LTI) and Experience API (xAPI) (Santos & Boticario, 2015). These standards facilitate data exchange and integration, ensuring that various tools can work together smoothly within an LMS environment. Privacy and data security are crucial, as AI usage raises concerns about consent, data protection, and ethical use, necessitating compliance with regulations such as GDPR (Popenici & Kerr, 2017). Transparency in data usage and AI decision-making is vital for trust. Additionally, effective user adoption requires training in technical skills and ethical implications, and overcoming resistance to AI-enhanced teaching methods requires robust support and ongoing professional development for educators (Holmes et al., 2019).

2.1.8 Evaluation and Assessment of Personalised Learning Systems

Effective evaluation and assessment of personalised learning are crucial to understanding its impact on student engagement, comprehension, and overall academic performance. Various methods and metrics can be employed to evaluate the effectiveness of personalised learning systems.

Methods for Evaluation

Table 1: Methods for evaluating personalised learning systems

Methods	Description
Learning Analytics Frameworks	Frameworks such as the Learning Analytics Cycle, which includes steps like data collection, analysis, action, and reflection, provide a structured approach to evaluating personalised learning (Clow, 2012).
Randomised Controlled Trials (RCTs)	RCTs are considered the gold standard for evaluating the impact of educational interventions. By randomly assigning students to either a personalised learning group or a control group, researchers can assess the effectiveness of personalised learning in a controlled environment (Koedinger et al., 2013).
Quasi-Experimental Designs	When RCTs are not feasible, quasi-experimental designs can be used. These designs involve comparing groups that are not

	randomly assigned but are matched on relevant characteristics to isolate the effects of personalised learning interventions (Cheung & Slavin, 2013).
Comparative Analysis	Comparing personalised learning systems with traditional educational approaches can highlight strengths and areas for improvement. Studies have shown that personalised learning can lead to higher engagement and better academic outcomes compared to traditional methods (Pane et al., 2017).

Metrics for Student Engagement, Comprehension, and Performance

Table 2: Metrics for evaluating personalised learning systems

Metrics	Description
Engagement Metrics	Metrics such as time spent on tasks, participation in discussions, and frequency of interactions with learning materials can indicate levels of student engagement. Learning management systems can track these metrics automatically, providing valuable data for analysis (Baker & Inventado, 2014).
Comprehension Metrics	Assessing comprehension often involves analysing student performance on formative and summative assessments. Metrics such as quiz scores, assignment grades, and performance on standardised tests can provide insights into how well students understand the material (Shute & Wang, 2016).
Performance Metrics	Overall academic performance can be measured using grades, retention rates, and completion rates. Additionally, comparing the performance of students in personalised learning environments with those in traditional settings can highlight the effectiveness of personalised approaches (Holmes et al., 2019).

2.1.9 Research Gaps and Future Directions

Identifying gaps and limitations in the current literature is critical for advancing the field of AI in education. Key areas for future research include the development of more robust and interpretable AI models, the integration of interdisciplinary perspectives, and the exploration of emerging technologies (Bozkurt et al., 2021). Additionally, there is a need for more comprehensive studies that address the ethical and practical challenges of implementing AI in educational settings (Hamal et al., 2022).

Future research should also focus on the potential of AI to enhance educational equity and access, particularly for underserved and marginalised populations (Attri et al., 2023). By addressing these gaps and exploring new directions, researchers and educators can continue to innovate and improve the use of AI in education, ultimately promoting better learning outcomes for all students.

2.2 Technology Review

2.2.1 Multimodal Content Generation Technologies

a) Text-to-Speech (TTS) Generation

Generating multimodal educational content enhances accessibility and caters to diverse learning preferences by offering content in formats such as audio, video, and slides. This section reviews different text-to-speech (TTS) synthesis methods.

Shen et al. (2018) introduced WaveNet, achieving high naturalness and audio quality, but it is computationally intensive and requires substantial training data. Google Text-to-Speech (gTTS) is a cloud-based solution offering high-quality, natural-sounding audio, multiple language support, and scalability with minimal computational resources, making it suitable for educational contexts. Sotelo et al. (2017) presented Char2Wav, an end-to-end neural network model that simplifies pre-processing but falls short in audio quality and control. Wang et al. (2017) used NLP techniques for text-to-audio and video summaries, producing high-quality outputs dependent on text complexity and the NLP model.

gTTS is the most suitable choice for AdaptEd's multimodal content generation due to its ease of implementation, high-quality output, language support, and scalability.

Table 3: Comparative analysis of TTS techniques

Source	Approach	Environment	Outcome	Strengths	Limitations
Shen et al. (2018)	WaveNet	Laboratory setting	High naturalness and audio quality	Captures complex speech characteristics	Computationally expensive
gTTS (Google Text-to-Speech)	Cloud-based TTS	Various educational contexts	High-quality, natural-sounding audio	Easy to implement, supports multiple languages	Requires internet connection
Sotelo et al. (2017)	Char2Wav	Laboratory setting	Simplified pipeline, end-to-end model	Eliminates the need for complex pre-processing	Limited audio quality and control
Wang, Yi, et al. (2020)	NLP techniques	Simulated educational setting	Efficiently converts text to audio and video summaries	High-quality audio and video generation	Quality depends on text complexity and the NLP model

b) Slide Generation

The studies explored various approaches for generating slides from textual content. Li et al. (2021) used unsupervised mutual learning to extract topic-relevant sentences, showing superior performance in maintaining topic relevance but limited to text extraction. Fu et al. (2022) applied a template-based NLP approach, providing straightforward slide generation but with limited flexibility and monotonous

results. Sun et al. (2021) used a hybrid of rule-based methods and machine learning to handle complex content, offering comprehensive summaries but requiring extensive rule tuning. The method adopted for AdaptEd is a combination of NLP models and a Python library using Hugging Face Transformers for text summarisation and python-pptx for slide creation. This approach is selected for its high accuracy in text summarisation and flexibility in slide creation.

Table 4: Comparative analysis of text-to-slide generation techniques

Sources	Methodology	Technology/Framework	Outcome	Strengths	Limitations
Li et al. (2021)	Unsupervised mutual learning	Neural networks, log-linear model	Topic-relevant sentence extraction	Topic-aware, effective without labelled data	Limited to text, manual annotation is needed for evaluation.
Fu et al. (2022)	Template-based NLP	NLP techniques	Summarise key points using templates	User-friendly, straightforward	Monotonous slides lack flexibility.
Sun et al. (2021)	Hybrid rule-based and machine learning	Rule-based methods, machine learning	Summarises main contributions, methodologies, and findings	Handles complex content, comprehensive summaries	Extensive tuning is needed; it depends on rule quality.
Hugging Face Transformers + python-pptx	Advanced NLP for summarisation, Python library for slide generation	Hugging Face Transformers, Python pptx	High-quality text summarisation and automated slide creation	High accuracy and relevance of summaries, flexible slide creation	Requires familiarity with advanced NLP and Python libraries

c) Video Generation

The comparative analysis of text-to-video generation approaches reveals distinct methodologies and technologies across different systems. CogVideo utilises multi-frame-rate hierarchical training, fine-tuning a pre-trained text-to-image model to enhance text-clip alignment and generation accuracy (Hong et al., 2022). C3LLM employs a multimodal LLM structure with hierarchical audio tokenisation for versatile content generation, although it faces complexity in training and real-time integration (Wang et al., 2024). Make-a-Video leverages GANs to produce high-quality, realistic video content but requires extensive computational resources and large datasets (Singer et al., 2022). Mo, Shi, and Tian (2024) focus on synchronising text-to-audio generation with video content, effectively enhancing audio-visual coherence, though real-time synchronisation for complex videos remains challenging. Each system's unique strengths and limitations highlight the ongoing need for improved efficiency, dataset availability, and real-time capabilities in text-to-video generation technology.

Mo, Shi, and Tian's (2024) approach will be adopted for the AdaptEd plugin to synchronise the audio output and the slide output as shown in Figure 3 because of its simplicity and effective audio-visual synchronisation.

Table 5: Comparative analysis of text-to-video generation techniques

Name	Methodology	Technology/ Framework	Outcome	Strengths	Limitations
CogVideo	Multi-frame-rate hierarchical training	Fine-tuning a pre-trained text-to-image model	Improved text-clip alignment and accuracy	Efficient fine-tuning and control over intensity	Text-action relation challenges, dataset issues
C3LLM	Multimodal LLM with hierarchical audio generation	LLM backbone, hierarchical tokenisation	Versatile multimodal generation	Unified model, improved semantic alignment	Complexity in training and real-time challenges
Make-a-Video	Generative adversarial networks (GANs)	GANs with advanced video synthesis techniques	High-quality video content	High-quality visuals and diverse input handling	High computational resources and large datasets
Mo, Shi, and Tian (2024)	Text-to-audio generation synchronised with video	Synchronisation algorithms	Coherent and synchronised audio-visual outputs	Effective audio-video synchronisation	Real-time synchronisation for complex videos

Multimodal Content Generation Service

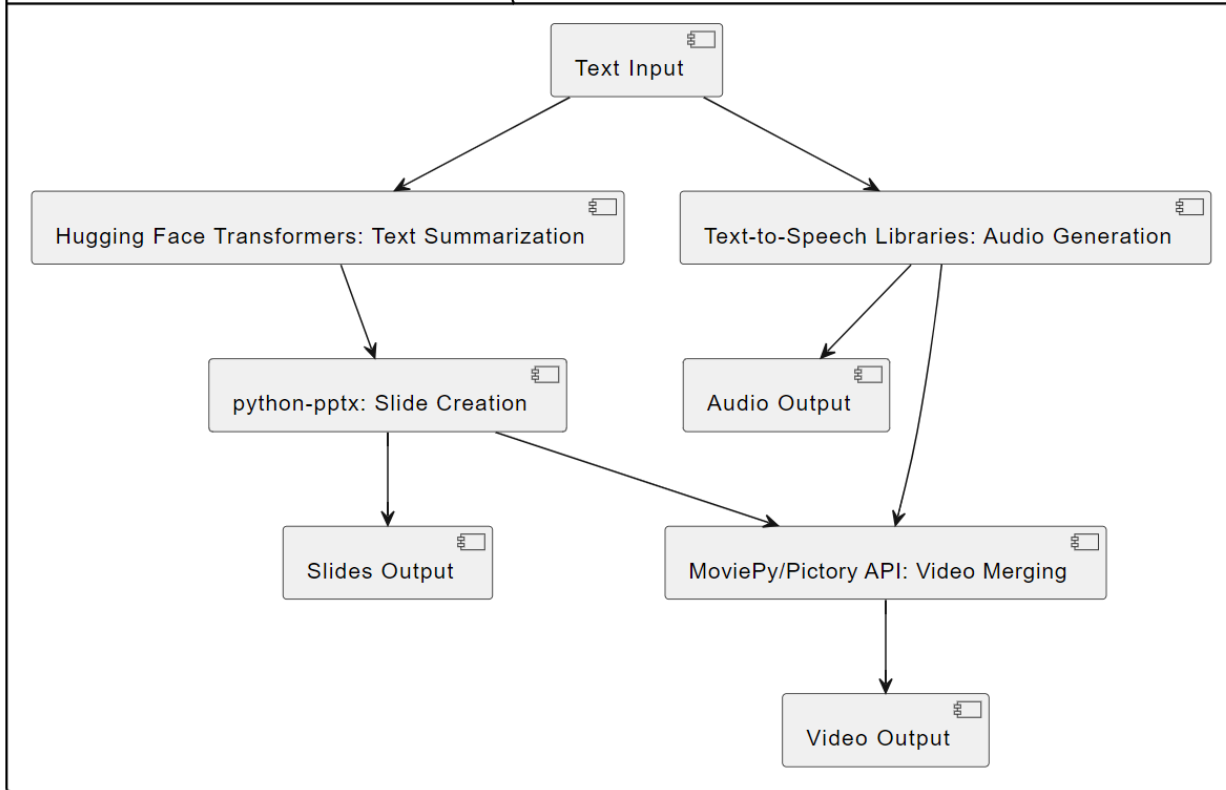


Figure 3: Multimodal content generation service architecture

2.2.2 Recommender System

The reviewed recommender systems employ various techniques for personalised learning. Imran et al.'s PLORS system uses a hybrid approach combining collaborative filtering and content-based methods with ontology-based mapping for semantic understanding. Bhaskaran and Marappan's system utilises advanced machine learning techniques, including clustering and sequential pattern mining, to analyse user data, focusing on scalability. Idrissi et al. (2023) integrate collaborative filtering, content-based methods, and autoencoders for dynamic and personalised recommendations. Aher and Lobo's approach employs simple k-means clustering and the association rule algorithm to identify effective learning sequences and group similar learners. Hybrid approaches, balancing multiple techniques, are particularly effective, offering improved recommendation accuracy and personalisation. Given the strengths of hybrid techniques in enhancing recommendation accuracy and personalisation, adopting a hybrid approach is recommended for the AdaptEd plugin. This will integrate collaborative filtering and content-based methods, combining their strengths to provide more accurate and personalised course recommendations.

Table 6: Comparative analysis of recommender systems

Sources	Primary Techniques	Additional Techniques	Strengths	Weaknesses	Complexity
PLORS (Imran et al., 2016)	Hybrid (Collaborative + Content-Base)	Ontology-Based Mapping	Enhanced semantic matching	Dependence on comprehensive profiles/ontologies	Moderate to High
Bhaskaran and Marappan (2023)	Machine Learning (Clustering, Sequential Mining)	Feature Extraction, Scalability	Advanced analysis, high scalability	Requires extensive data and resources	High
Idrissi et al. (2023)	Hybrid (Collaborative + Content-Base)	Autoencoders	Dynamic adaptation, high relevance	Complexity in integrating autoencoders	High
Aher and Lobo (2011)	Clustering (K-Means), Association Rule Algorithm	Collaborative Filtering	Insights from learning sequences and effective grouping	Dependency on clustering accuracy	Moderate

2.2.3 Learning Analytics and Early Prediction of At-Risk Students

Learning analytics and data mining techniques monitor student performance and predict at-risk students using machine learning, data mining algorithms, and predictive modelling. Three key studies on predicting at-risk students implemented various machine learning techniques. Abdrakhmanov et al. (2024) used Convolutional Neural Networks (CNN) to predict academic performance in STEM and non-STEM education, offering high accuracy, robust feature extraction, and scalability. Alalawi et al. (2021) used RF, SVM, DT, KNN, and NB, achieving the highest binary classification accuracy of 96% with KNN, and introduced a web application for real-time prediction and intervention. Chui et al. (2020) focused on a variant of SVM, Reduced Training Vector Support Vector Machine (RTV-SVM), which reduced training time while maintaining high accuracy and performing well in both binary and multiclass predictions.

Overall, ensemble methods like RF and AdaBoost showed high accuracy but had higher computational demands, making SVM variants, especially RTV-SVM, more suitable for large-scale and real-time applications.

Recent studies suggest using ensemble and hybrid models for educational data to improve accuracy by combining different algorithms (Abdrakhmanov et al., 2024). Therefore, AdaptEd will implement RTV-SVM and RF, leveraging the strengths of both algorithms for enhanced accuracy and robustness.

Key Studies on Predicting At-Risk Students

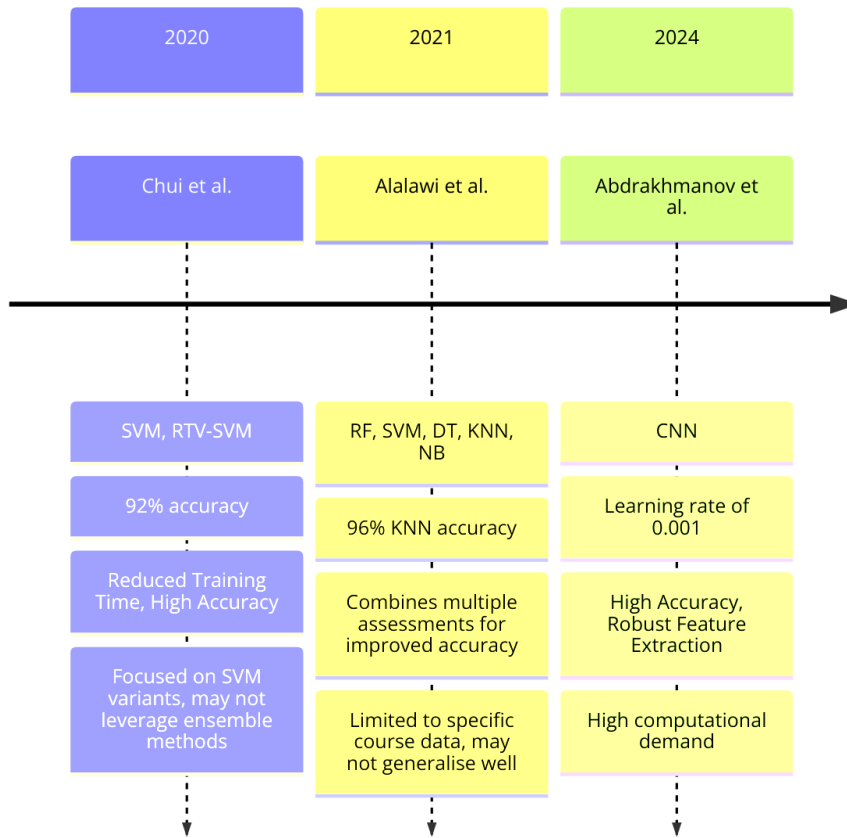


Figure 4: Techniques for early prediction of at-risk students

2.2.4 AI-Powered Virtual Assistant

AI and NLP-powered virtual assistants provide 24/7 support and enhance student engagement. Radziwill et al. (2017) explored AI-powered virtual assistants, demonstrating advanced contextual understanding and personalised support, but with complex development. Cahn (2017) examined NLP-based chatbots, effectively responding to a wide range of queries but struggling with complex queries and requiring continuous updates.

Radziwill et al.'s (2017) AI-powered virtual assistant approach is recommended for AdaptEd due to its advanced contextual understanding and personalised support.

Table 7: comparative analysis of virtual assistants

Sources	Approach	Outcome	Strengths	Limitations
Radziwill, Nicole M., et al. (2017)	AI-Powered Virtual Assistants	Advanced contextual understanding and personalised support	High engagement and contextual understanding	Complex to develop and maintain requires significant resources.
Cahn (2017)	NLP-Based Chatbots	Understands and responds to a wide range of queries	Improves student support and engagement	May struggle with complex queries and require continuous updates.

3.0 Methods

3.1 System Design Analysis

3.1.1 Architecture Overview

Microservices Architecture

AdaptEd is designed using a microservices architecture, an architectural style that structures an application as a collection of loosely coupled services, each responsible for a specific functionality. Microservices are independently deployable and scalable, which aligns well with the needs of modern, complex applications like AdaptEd that require flexibility, scalability, and maintainability (Dragoni et al., 2017). Each feature of the tool—multimodal content generation, recommendation system, learning analytics, and virtual assistant—will be developed as an independent service that communicates with others through APIs. In comparison, a monolithic architecture integrates all functionalities into a single codebase, simplifying development and deployment initially but becoming unwieldy and difficult to manage as the application grows, with updates and scaling becoming challenging (Balalaie et al., 2016). Service-Oriented Architecture (SOA) is similar to microservices in decomposing applications into services, but SOA services are typically larger and share resources more than microservices. Microservices offer greater independence and finer granularity (Lewis & Fowler, 2014). Figure 5 below shows the architecture diagram of AdaptEd.

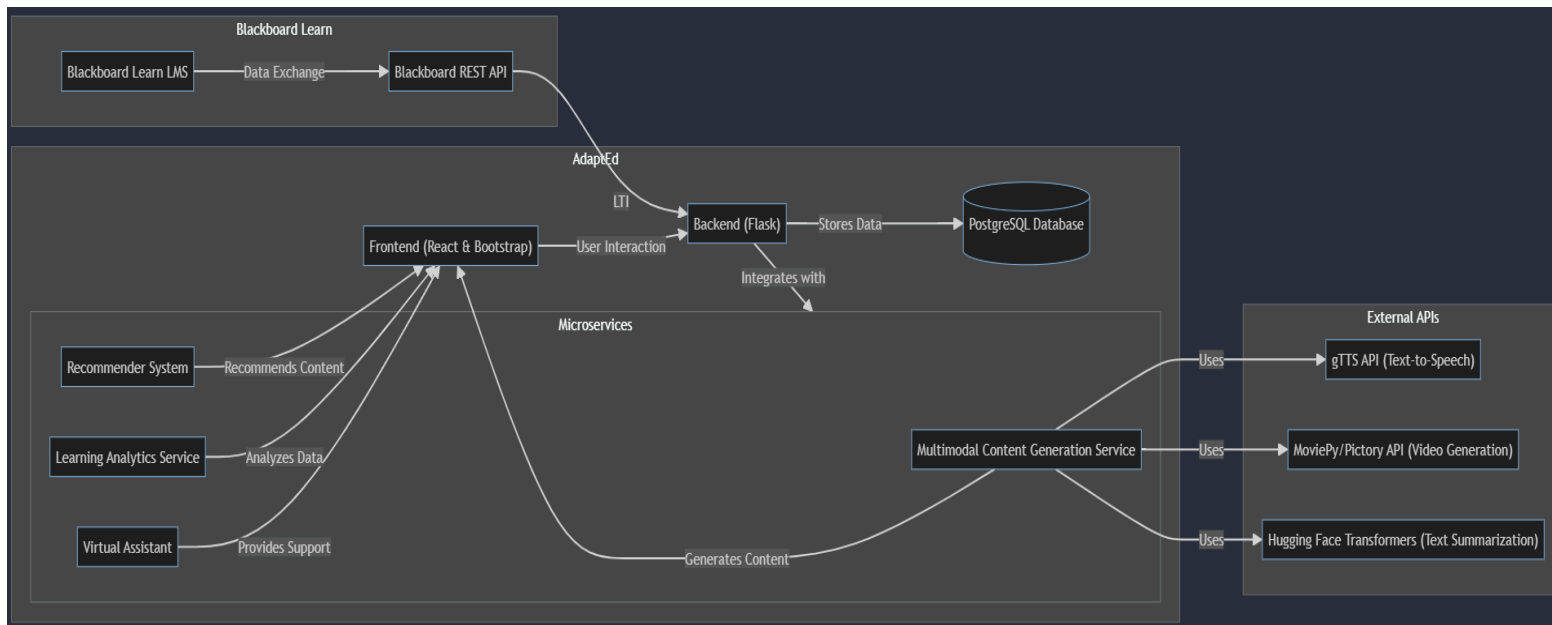


Figure 5: AdaptEd's system architecture

Justification for Microservices Architecture

Table 8: Justification for microservices architecture

Reason	Justification
Scalability	Each service can be scaled independently based on demand, ensuring efficient resource use (Newman, 2015).
Flexibility	Allows for the use of different technologies for different services, enhancing flexibility (Lewis & Fowler, 2014).
Fault Isolation	Issues in one service do not directly impact others, improving overall system resilience (Dragoni et al., 2017).
Continuous Deployment	Facilitates continuous deployment and integration, crucial for Agile development (Newman, 2015).

3.2 Tools and Technology

The development of AdaptEd utilises a comprehensive tech stack comprising various libraries, frameworks, and services. Python serves as the primary programming language, supported by Flask for the web framework. React and Bootstrap are employed for the user interface to ensure a responsive and interactive design. The virtual assistant functionality is implemented using the ChatGPT API, while TensorFlow Recommenders and Scikit-Learn power the recommendation system. Text summarisation for slide generation is managed by Hugging Face Transformers, with Python-pptx employed for the creation of slides. Video generation is achieved through the integration of text-to-speech libraries with MoviePy or the Pictory API. For early prediction of at-risk students, the stack incorporates PyTorch for model building and training, alongside Scikit-Learn for data pre-processing and traditional machine learning algorithms. PostgreSQL is used for data storage and management, with Git facilitating version control. The tool will be deployed on AWS. Table 9 below shows the justification for the tools and technologies.

Table 9: Justification for tools and technologies

Component	Tool/Technology	Alternative	Justification
Web Framework	Flask	Django	Flask is lightweight, flexible, and widely used for building microservices. Its simplicity allows for quick development and easy maintenance (Grinberg, 2018).
Frontend	React & Bootstrap	Angular	React offers component-based architecture, facilitating reusable UI components. Bootstrap ensures responsive design, enhancing user experience across devices (Banks & Porcello, 2020; Otwell, 2018).
Database	PostgreSQL	MySQL	PostgreSQL is a robust, open-source relational database known for its reliability and support for complex queries (Momjian, 2001).

Deployment	AWS	Azure	AWS provides scalable and reliable cloud services, essential for deploying and managing microservice architecture (Amazon Web Services, 2024).
Recommendation System	TensorFlow Recommenders	Apache Mahout	TensorFlow Recommenders leverages powerful machine learning models to deliver accurate recommendations based on user data (Chollet, 2017).
Learning Analytics	PyTorch	TensorFlow	PyTorch is favoured for its dynamic computation graph and ease of use in developing and testing machine learning models (Paszke et al., 2019).
Data Pre-processing	Scikit-Learn	Pandas	Scikit-Learn is a comprehensive library for data mining and analysis, providing efficient tools for data preprocessing (Pedregosa et al., 2011).
Text Summarisation	Hugging Face Transformers	SpaCy	These pre-trained models provide state-of-the-art performance for NLP tasks such as text summarisation (Wolf et al., 2020).
Slide Generation	python-pptx	LibreOffice API	Python-pptx allows for the programmatic creation and manipulation of PowerPoint files, facilitating automated slide generation (Taylor, 2020).
Text-to-Speech	gTTS	Amazon Polly	gTTS is a simple yet effective tool for converting text to speech using Google's Text-to-Speech API (Google Text-to-Speech, 2024).
Video Generation	MoviePy/Pictory API	OpenShot API	MoviePy is a flexible library for video editing, while Pictory API offers automated video creation suitable for educational content (Pictory AI, 2024; Zulko, 2024).

Version Control	GitHub	GitLab	GitHub provides robust version control, collaboration features, and CI/CD integration, essential for managing development projects (GitHub, 2024).
Integration	LTI & Blackboard REST API	Custom API	LTI ensures interoperability with LMS, and Blackboard REST API allows seamless integration with Blackboard Learn (IMS Global, 2020; Blackboard, 2024).

3.3 Agile Development Methodology

AdaptEd will be developed using Agile methodology to facilitate iterative development, continuous feedback, and rapid adaptation to changes. This approach ensures the tool evolves based on user feedback and changing requirements (Beck et al., 2013). In comparison with other software development methodologies, the waterfall model follows a linear and sequential approach, making it rigid and difficult to accommodate changes once a phase is completed (Royce, 1970). This inflexibility is unsuitable for projects like AdaptEd, which require iterative development and continuous integration. The V-Model, an extension of the waterfall model, emphasises verification and validation but shares the same rigidity and lack of support for iterative development (Forsberg & Mooz, 1991). The spiral model combines iterative development with systematic aspects of the waterfall model, making it more flexible than the waterfall approach. However, it can be complex and costly due to its iterative nature and the requirement for risk analysis at each phase (Boehm, 1988). Agile methodologies strike a balance between flexibility and structure, allowing for iterative development while providing a framework for managing the process effectively.

Justification for Agile Methodology

Table 10: Justification for Agile methodology

Reason	Justification
Flexibility and Adaptability	Agile allows for changes and refinements throughout the development process, which is essential for accommodating evolving user requirements (Schwaber & Beedle, 2002).
Customer Collaboration	Regular feedback from stakeholders ensures the product meets user needs and expectations (Beck et al., 2001).
Incremental Delivery	Delivers functional components in iterations, providing value to users early and often (Schwaber & Beedle, 2002).
Risk Management	Frequent testing and review reduce the risk of significant issues going unnoticed until later stages (Sutherland, 2014).

3.3.1 Agile Phases and Practices

Scrum

Scrum is a popular Agile framework that structures development in cycles of work called sprints, typically lasting two weeks. Each sprint aims to produce a potentially shippable product increment (Schwaber & Sutherland, 2017).

Key Practices:

- **Sprint Planning:** Define the scope and objectives of the sprint, breaking down tasks and assigning them to team members (Sutherland, 2014).
- **Daily Stand-ups:** Hold daily reviews to discuss progress, identify roadblocks, and adjust plans as needed.
- **Development and Testing:** Develop features incrementally, continuously integrating and testing new code.
- **Sprint Review:** Demonstrate completed work to stakeholders, gather feedback, and plan for the next sprint.
- **Sprint Retrospective:** Reflect on the sprint to identify what went well, what didn't, and how processes can be improved.

Implementation Timeline

The timeline for the project is six weeks; hence, the project will be divided into three two-week sprints:

Table 11: Project timeline

Week	Phase	Activity Category	Activities
1-2	Sprint 1	Planning and Requirement Analysis Initial design Backend Development Frontend Development	Gather detailed requirements from stakeholders, define user stories, and create the product backlog. Develop system architecture and microservice design. Create UI/UX prototypes using Figma. Implement core backend services using Flask and Python. Set up the PostgreSQL database. Develop initial UI components with React and Bootstrap.
3-4	Sprint 2	AI Model Integration API Development Multimodal Content Generation Frontend Enhancement	Integrate TensorFlow Recommenders for the recommendation system and Hugging Face Transformers for text summarisation. Develop APIs for communication between microservices. Refine UI components and integrate with backend services.
5-6	Sprint 3	Advanced Backend Development Frontend Enhancement Testing and Deployment	Implement learning analytics using PyTorch and data pre-processing with Scikit-Learn. Implement the ChatGPT API for the virtual assistant. Perform integration testing, user acceptance testing (UAT), and security

			testing. Deploy the tool on AWS and conduct final performance monitoring.
--	--	--	---

3.4 Data Collection and Analysis

3.4.1 Data Collection

Surveys: Administer questionnaires to educators and students to gather comprehensive insights into their experiences, preferences, and challenges with existing learning systems. The survey will probe into aspects such as preferred learning modalities, the effectiveness of current LMS tools, and feedback on adaptive learning technologies.

System Logs and Usage Data: Collect detailed data on student interactions with the LMS, including metrics such as time spent on various content types, engagement levels, and performance outcomes. This data will be critical for understanding user behaviour and system efficacy.

3.4.2 Data Analysis

Descriptive Statistics: Employ descriptive statistical methods to summarise the survey results, identifying prevalent trends and common user preferences. This analysis will highlight key areas for improvement and user satisfaction.

Predictive Analytics: Utilise machine learning models, specifically PyTorch, to predict at-risk students based on engagement and performance data. Feature engineering techniques in Scikit-Learn will be applied to preprocess the data, enhancing model accuracy and predictive power.

3.5 Evaluation and Assessment

The evaluation and assessment of the AdaptEd plugin will employ a mixed-methods approach, combining quantitative and qualitative data to provide a comprehensive analysis of its effectiveness. This approach aligns with the principles outlined in the literature on personalised learning evaluation.

Quantitative Methods:

- **Surveys and Questionnaires:** Administered to students and educators to gather data on user satisfaction, perceived effectiveness, and usability of the AdaptEd plugin.
- **System Analytics:** Collection of usage data such as time spent on the platform, engagement levels, and academic performance metrics to measure the impact on learning outcomes.

Qualitative Methods:

- **Interviews and Focus Groups:** Conducted with a sample of students and educators to obtain in-depth feedback on their experiences and suggestions for improvement.
- **Case Studies:** Detailed examination of individual or group experiences with the AdaptEd plugin to understand its impact on specific learning contexts.

Data from these methods will be analysed to identify trends, strengths, and areas for improvement. The mixed-method approach ensures a robust evaluation by capturing both numerical data and personal insights, providing a holistic view of the plugin's effectiveness in enhancing personalised learning.

3.6 Ethical Considerations

Data Privacy: Ensure stringent adherence to data protection regulations, such as GDPR. Obtain informed consent from all participants regarding data collection and usage (European Union, 2016).

Bias and Fairness: Mitigate potential biases in AI models by employing diverse and representative datasets. Conduct regular audits of the models to ensure they provide fair and equitable treatment to all students (Mehrabi et al., 2021).

Transparency: Maintain a high level of transparency in data usage and AI-driven decision-making processes. Provide users with clear explanations about the functionality and limitations of the system to foster trust and understanding (Doshi-Velez & Kim, 2017).

User Consent: Clearly communicate to users about the types of data being collected and their intended uses. Ensure that users have the option to opt-out at any stage of the process (Nissenbaum, 2011).

Security: Implement robust security measures to safeguard user data against unauthorised access and potential breaches, thereby ensuring the integrity and confidentiality of the data (Sullivan & Liu, 2011).

4.0 Execution

This section covers the comprehensive development process of the AdaptEd plugin, focusing on three pivotal areas: requirements gathering and analysis, design and modelling, and implementation. Each subsection provides in-depth insights into the methodologies used, challenges encountered, and solutions devised throughout the development lifecycle. The development was executed in three sprints, as outlined in the Methods section, ensuring an agile and iterative approach to the plugin's creation.

4.1 Requirements Gathering and Analysis

The requirements gathering and analysis phase was crucial for understanding the needs of the end-users and ensuring that the AdaptEd plugin addresses the key challenges faced by students and instructors and aligns with the primary research objectives. This phase involved a methodical approach to collect and analyse both functional and non-functional requirements, ensuring a holistic understanding of the project scope and guiding the subsequent design and implementation phases.

A key component of this phase was the deployment of a comprehensive survey using Google Forms, which garnered responses from 24 anonymous ALU students. This survey served as a valuable tool for collecting empirical data on user experiences, preferences, and challenges with existing LMS. The survey results, detailed in the Appendix, revealed several critical insights that significantly influenced the plugin's development direction. 100% of the respondents use AI for learning purposes, highlighting the necessity for educational institutions to incorporate AI tools like the AdaptEd plugin into their LMS with clear usage policies rather than oppose its use. Over 70% of the respondents find video, visuals, and interactive contents most engaging and helpful, compared to 33.3% who find text more helpful. 50% of the respondents state that the course contents on their current LMS are fairly tailored to their learning style, while only 8.3% believe that the course contents are well-tailored to their learning style. 70.8% of respondents would find an AI-powered virtual assistant helpful for personalised guidance and support, highlighting the need for the AdaptEd chatbot service.

What types of content (text, video, audio, interactive) do you find most engaging and helpful?

 Copy

24 responses

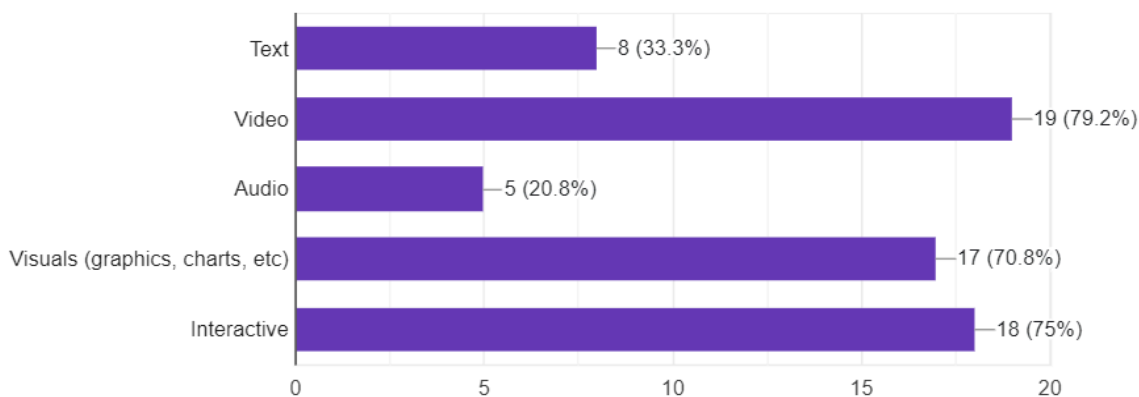


Figure6: Survey Summary 1

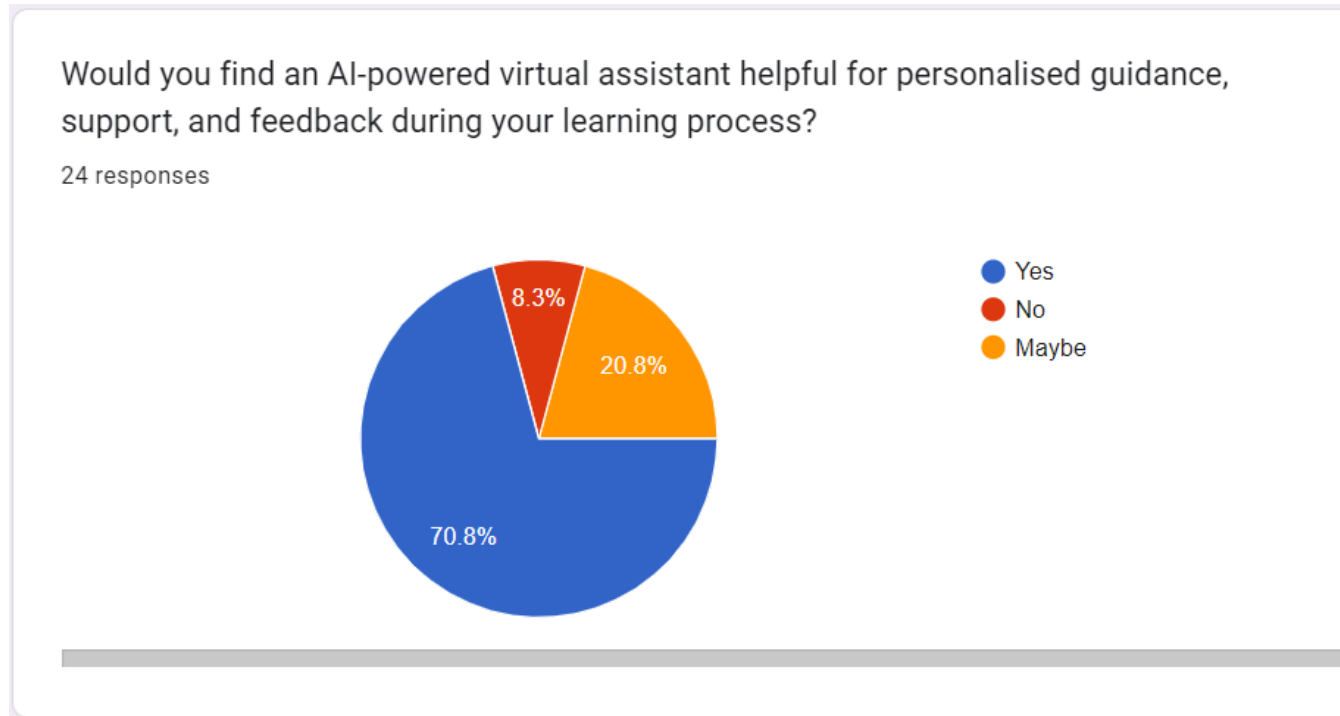


Figure7: survey summary 2

These findings were subsequently refined and prioritised to develop the functional and non-functional requirements. The analysis phase also involved a critical evaluation of project constraints, leading to the strategic decision to exclude the learning analytics feature from the initial release, reserving it for future updates to ensure a focused and timely delivery of core functionalities.

The requirements were further elaborated through the creation of use cases, a use case diagram, and use case forms, providing a visual and narrative representation of system interactions. This approach facilitated a deeper understanding of user interactions with the plugin and helped identify potential edge cases and alternative scenarios.

4.1.1 Functional Requirements

The functional requirements were meticulously defined to encapsulate the core capabilities of the AdaptEd plugin, directly addressing the needs identified in the survey and aligning with the project's objectives. These requirements form the backbone of the plugin's functionality, ensuring a user-centric and feature-rich learning environment.

Table 12: Functional Requirements

ID	Requirements	Description
FRE01	Generate recommendations	The recommender system should be able to generate course content recommendations in student's preferred formats (e.g., text, video, audio) based on the student's interaction history and preferences.

FRE02	View recommendations	Students should be able to view the generated recommendations.
FRE03	Query virtual assistant	Students should be able to query the chatbot about Moodle courses, resources, deadlines, and receive context-aware responses.
FRE04	Generate multimodal content	Instructors should be able to upload text-based content and convert it into audio, slides, and video automatically.
FRE05	Upload generated content to course page	Instructors should be able to upload the generated content to the course page
FRE06	Access generated content	Students should be able to access the generated content

4.1.2 Non-Functional Requirements

The non-functional requirements were carefully crafted to ensure the AdaptEd plugin meets high standards of performance, usability, and reliability. These requirements address critical aspects such as scalability, compatibility, and accessibility, which are essential for the plugin's successful integration and adoption within diverse educational contexts.

Table 13: Non-Functional Requirements

ID	Requirements	Description
NRE01	Performance	The plugin should be optimised for fast response times and minimal latency in delivering content and running models.
NRE02	Scalability	The plugin should be scalable to handle increasing numbers of users and data without performance degradation.
NRE03	Compatibility	The plugin should be compatible with Moodle version 3.9 and above, integrate seamlessly with existing plugins, and function correctly on major web browsers (Chrome, Firefox, Safari, Edge).
NRE04	Usability	The plugin should have a user-friendly interface accessible to users with varying levels of technical expertise.
NRE05	Accessibility	The system should be designed with accessibility standards (e.g., WCAG) to support users with disabilities.
NRE06	Reliability	The system must provide high availability and minimal downtime to ensure continuous accessibility.
NRE07	Maintainability	The codebase should be well-documented and modular to facilitate future updates and maintenance.

4.1.3 Use Cases

The refinement and prioritisation of functional requirements led to the development of comprehensive use cases, as outlined in Table 6.1. These use cases served as the foundation for creating a detailed use case diagram, which visually represented the system's functionality from an end-user perspective. The accompanying use case forms provided in-depth descriptions of both normal and alternative scenarios, facilitating a thorough understanding of potential user interactions and system behaviors.

Table 14: Use Cases

ID	Actor	User Case
USC 1	Instructor	Generate multimodal content
USC 2	Instructor	Manage multimodal content
USC 3	Instructor	Query virtual assistant
	Student	
USC 4	Student	Access multimodal content
USC 5	Student	View personalised recommendations
USC 6	Admin	Manage user access
USC 7	Admin	Manage plugin settings
USC 8	Admin	Update and maintain plugin

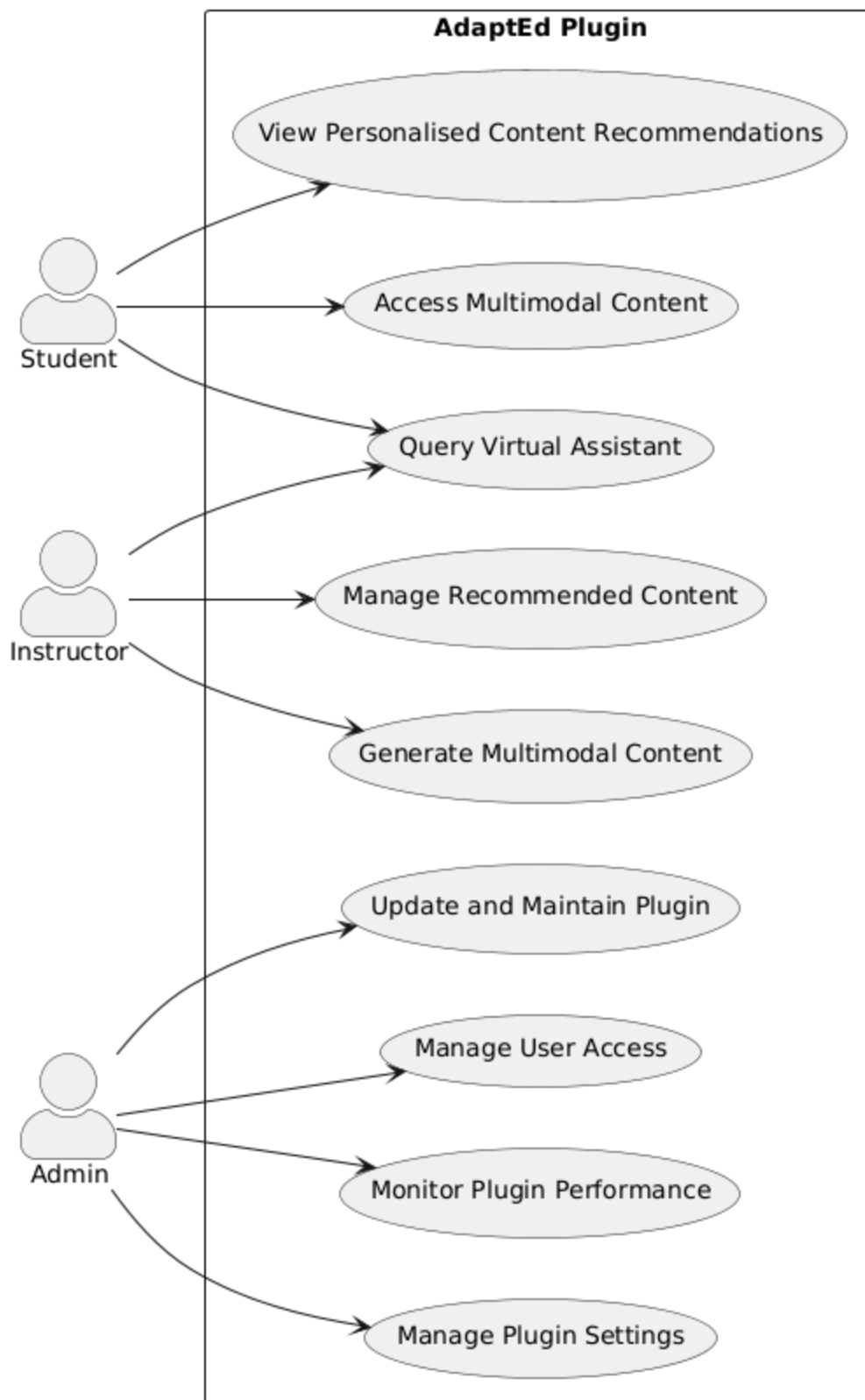


Figure8: Use case Diagram

Use Case Forms

Table 15: Use case: Query Virtual Assistant

Use Case Name	Query Virtual Assistant
Actors	Student, Instructor
Brief Description	Student or Instructor interacts with the chatbot to ask questions about course content, grades, assignments, or other topics.
Pre-Conditions	User is logged into the Moodle system and has access to the AdaptEd plugin.
Possible Scenarios for "Query Virtual Assistant"	
Scenario 1 (Happy Scenario):	
Step	Action
1	User navigates to the AdaptEd chatbot interface
2	User types a question into the chatbot input field
3	System processes the query and retrieves relevant information
4	Chatbot displays a response to the user's query [A1]
5	User reviews the response and can ask follow-up questions if needed
Alternative Scenarios	
Scenario 2 (Sad Scenario):	
Step	Action
A1	If the chatbot cannot understand or find an answer to the query, it prompts the user to rephrase the question or enter valid prompt. Return to step 2 of the happy scenario.

Table 16: Use case: View Personalised Content Recommendations

Use Case Name	View Personalised Content Recommendations	
Actors	Student	
Brief Description	Student receives and views recommendations for course content in their preferred format (audio, video, text) based on their interaction history.	
Pre-Conditions	Student is logged into the Moodle system and has interacted with course content previously.	
Possible Scenarios for "View Personalised Content Recommendations"		
Scenario 1 (Happy Scenario):		
Step	Action	
1	Student navigates to the course page	
2	System analyses the student's interaction history and preferences [A1]	
3	System generates a list of 10 personalised content recommendations and displays the top three recommendations	
4	Student selects a recommended content item to access	
5	System loads the selected content for the student to interact with	
Alternative Scenarios		
Scenario 2 (Sad Scenario):		
Step	Action	
A1	If user is new or there's not enough interaction history to generate personalised recommendations, the system displays recommendations based on the content. Return to step 4 of the happy scenario.	

Table 17: Use case: Generate Multimodal Content

Use Case Name	Generate Multimodal Content
----------------------	------------------------------------

Actors	Instructor
Brief Description	Instructor uploads text-based content and converts it into audio, slides, and video formats for students to access.
Pre-Conditions	Instructor is logged into the Moodle system and has permission to create and edit course content.
Possible Scenarios for "View Personalised Content Recommendations"	
Scenario 1 (Happy Scenario):	
Step	Action
1	Instructor navigates to the content creation section of the AdaptEd plugin
2	Instructor uploads text-based content [A1]
3	Instructor selects desired output formats (audio, slides, video) [A2]
4	System processes the content and generates the selected formats [A3]
5	System displays a preview of the generated content
6	Instructor reviews and approves the generated content [A4]
7	System uploads the approved content to the course page
Alternative Scenarios	
Scenario 2 (Sad Scenario):	
Step	Action
A1	If the user uploaded a non text-based file, the system displays “Invalid file format” and prompt user to upload a valid file. Return to step 2 of the happy scenario.
A2	If the user did not select at least one output format, the system prompts user to select a format. Return to step 3 of the happy scenario.

A3	If the system encounters technical error during content generation, it displays an error message and suggests troubleshooting steps. Return to step 2 or 3 of the happy scenario.
A4	If the instructor is not satisfied with the generated content, they can edit the original text or adjust generation settings. Return to step 4 of the happy scenario.

This methodical approach to requirements gathering and analysis not only ensured a clear roadmap for development but also established a solid foundation for the subsequent design and implementation phases. By grounding the development process in empirical data and user-centric design principles, the AdaptEd plugin is positioned to effectively address the evolving needs of modern educational environments, fostering enhanced learning experiences through innovative technology integration.

4.2 Design and Modelling

The design and modelling phase of the AdaptEd plugin development involved translating the gathered requirements into a concrete, implementable structure. This phase employed a systematic approach, leveraging modern software design principles and methodologies to ensure the creation of a robust, scalable, and maintainable solution. The design process was guided by the project objectives, particularly the need for seamless integration with Moodle, enhanced user experience, and the provision of adaptive learning features.

4.2.1 System Design

The system architecture for the AdaptEd plugin underwent a significant evolution during the design phase, primarily driven by practical considerations and the need for optimal performance and cost-effectiveness. Initially, the integration was planned for Blackboard Learn; however, a critical analysis of the associated AWS resource costs led to a strategic pivot towards Moodle as the target LMS. This decision was the result of a thorough cost-benefit analysis, which revealed that Moodle offered a more sustainable and cost-effective platform for the plugin's deployment. The open-source nature of Moodle aligns well with the project's objectives of accessibility and scalability, allowing for greater flexibility in customization and integration.

The revised system architectural design, as illustrated in the image below, showcases a microservices-based approach. This architecture was chosen for its ability to provide modular, independently deployable services that can be developed, updated, and scaled individually. The key components of the architecture are highlighted in the table below.

Table 18: System Components

Components	Description and Justification
Frontend - Mustache - NGINX	Mustache was used for templating in place of React and Bootstrap that were intended for the initial Blackboard Learn because of its usability in Moodle development and easy integration as it is Moodle's native template system. NGINX was used as a reverse proxy to ensure efficient request handling and load balancing.
Backend Flask	Flask is a lightweight Python web framework that allows for rapid development and easy integration with various services.

Microservices • Recommender System • Multimodal Content Generator • Chatbot Service	The plugin features are broken down to microservices for flexibility and scalability. Moodle is written in PHP and some backend processes like generating recommendations, are largely dependent on Python, separating the features will enable the use of different technologies and programming languages for different services based on their specific requirements.
Database • PostgreSQL	PostgreSQL is used for persistent data storage, chosen for its robustness and support for complex queries.
External Services • gTTS • Python-pptx • Hugging Face Transformers • Moviepy • OpenAI API • OpenCV • DALL-E	These external services along with other libraries that will be discussed in the implementation section are used to process various backend tasks such as text conversion, model training and recommendation generation, natural language processing tasks, and advanced language model capabilities.

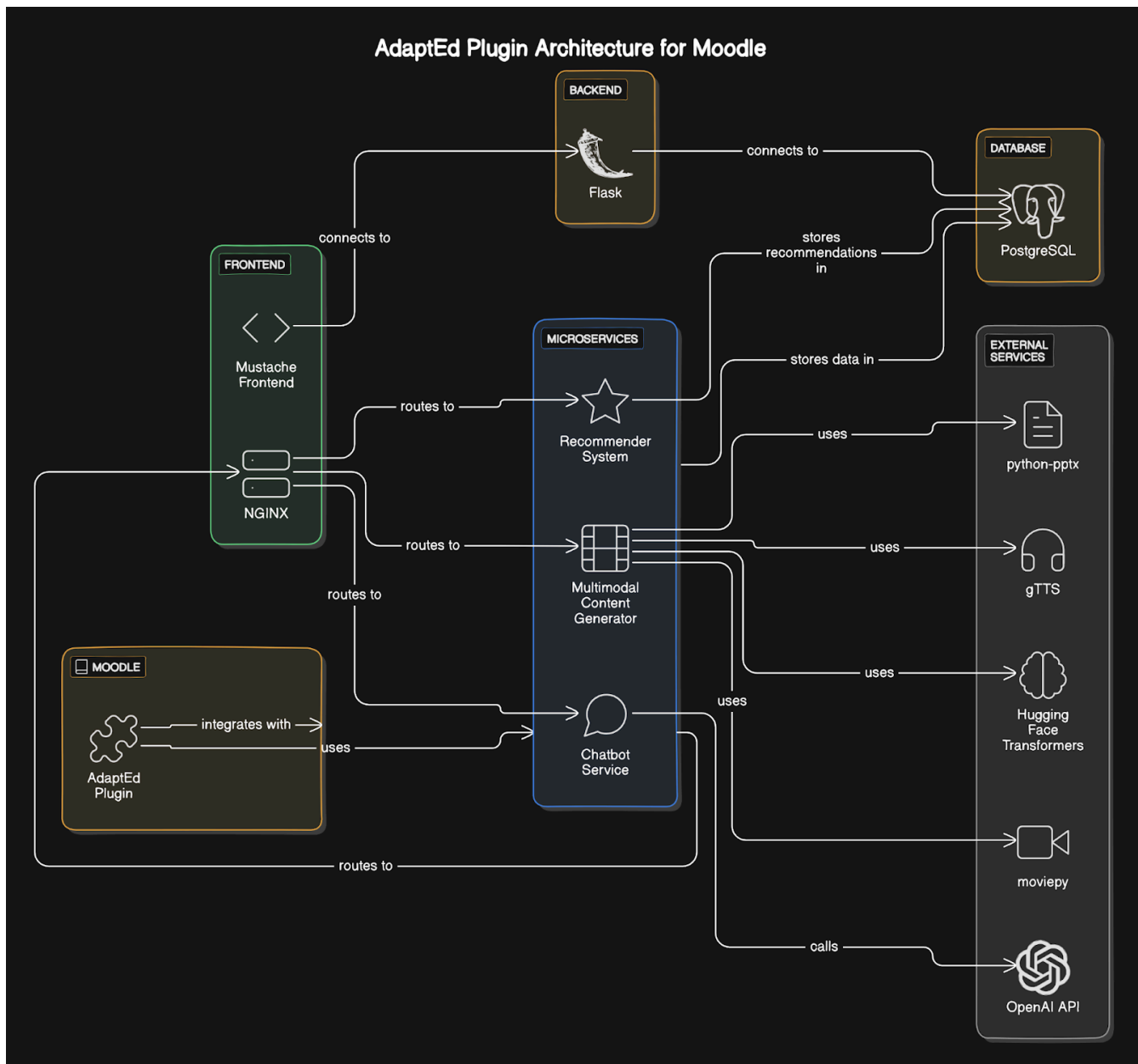


Figure 9: System Architecture design

The design also incorporates principles of loose coupling and high cohesion, ensuring that each component has a well-defined responsibility while minimizing dependencies between services. This approach facilitates easier maintenance, testing, and future enhancements of the plugin.

4.2.2 Virtual Assistant Design and Modelling

The virtual assistant, a key feature of the AdaptEd plugin, was designed to provide context-aware responses to user queries. The activity diagram below provides a higher-level overview of the system's processes and workflows for better understanding of the “Query chatbot” use case outlined in the requirements gathering section. The sequence diagram below illustrates the sophisticated decision-making process of the chatbot and its interactions with user, Moodle, database and external services. This design ensures that the virtual assistant can handle a wide range of queries, from

course-specific information to general knowledge questions, enhancing the overall learning experience. The table below highlights the key design considerations.

Table 19: Design Consideration for Virtual Assistant

Design Consideration	Description
User Input Handling	The system efficiently captures and processes user prompts.
Knowledge Base Integration	MoodleKB serves as the primary source for Moodle-specific information.
Fallback Mechanism	When MoodleKB lacks a suitable response, the system leverages OpenAI GPT-3.5 Turbo for general knowledge queries.
Response Generation	The chatbot synthesises information from various sources to provide comprehensive answers.

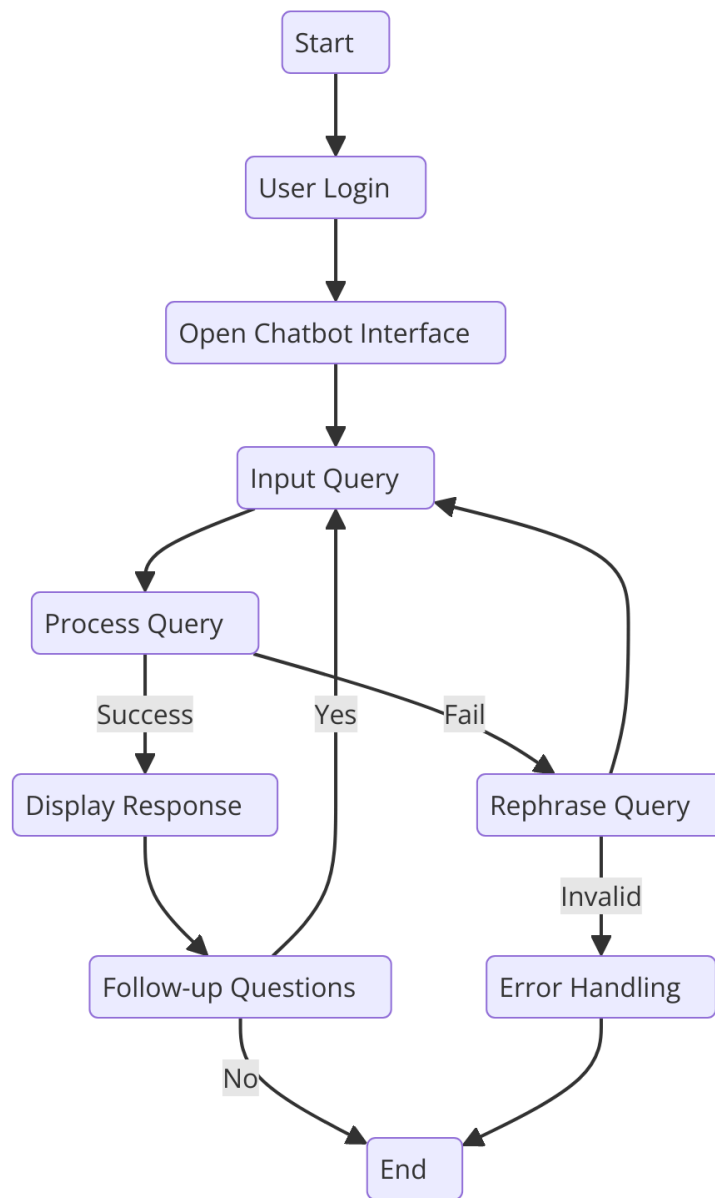


Figure 10: Activity Diagram for Virtual Assistant

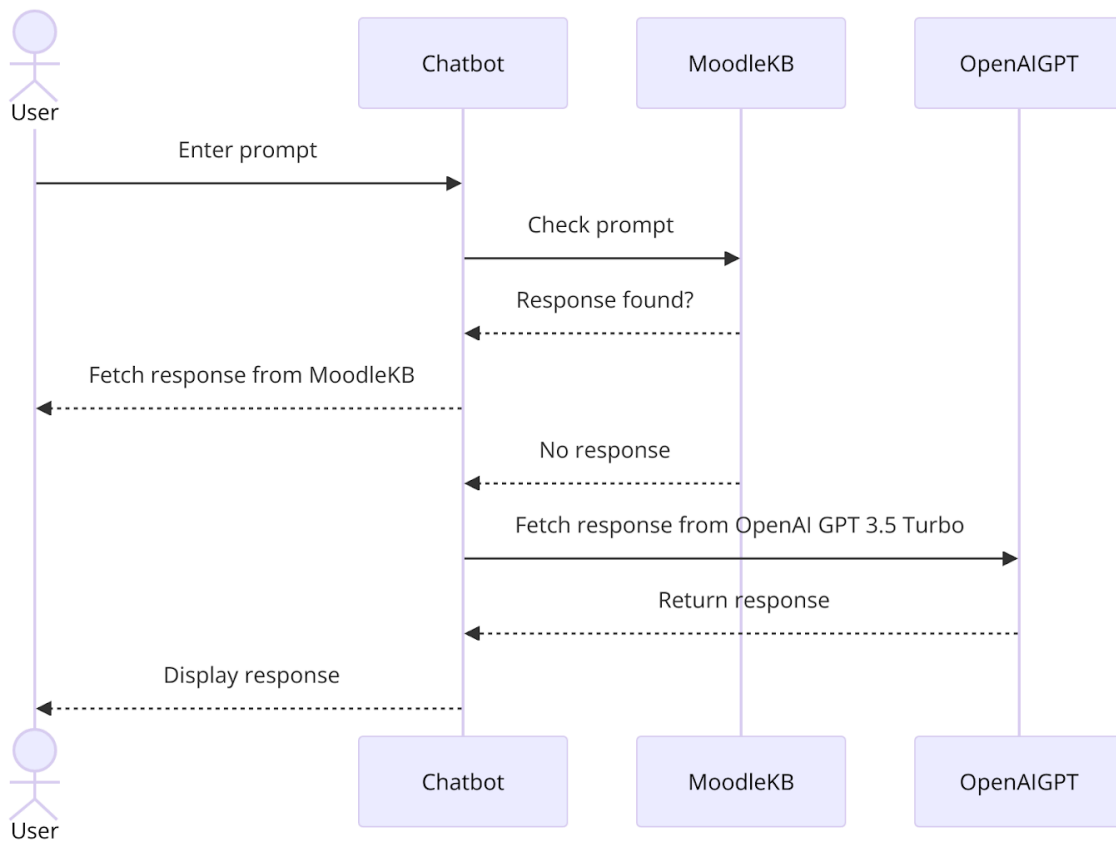


Figure 11: Sequence Diagram for Virtual Assistant

4.2.3 Recommender System Design and Modelling

The recommender system, depicted in Image 4, was designed to provide personalised content suggestions based on individual student interactions. This design approach ensures that the recommender system adapts to changing user preferences over time, continuously improving the relevance of its suggestions. Key design considerations include:

Table 20: Design Consideration for Recommender System

Design Consideration	Description
Data Collection	The system continuously gathers and analyses student interaction history hence the need for the plugin to have access to Moodle's database.
Preference Analysis	Advanced algorithms interpret user behavior to discern content format preferences
Dynamic Recommendations	The system generates and updates recommendations in real-time.
Fallback Mechanism	For new users or those with limited interaction history, the system provides default recommendations to ensure a smooth user experience.

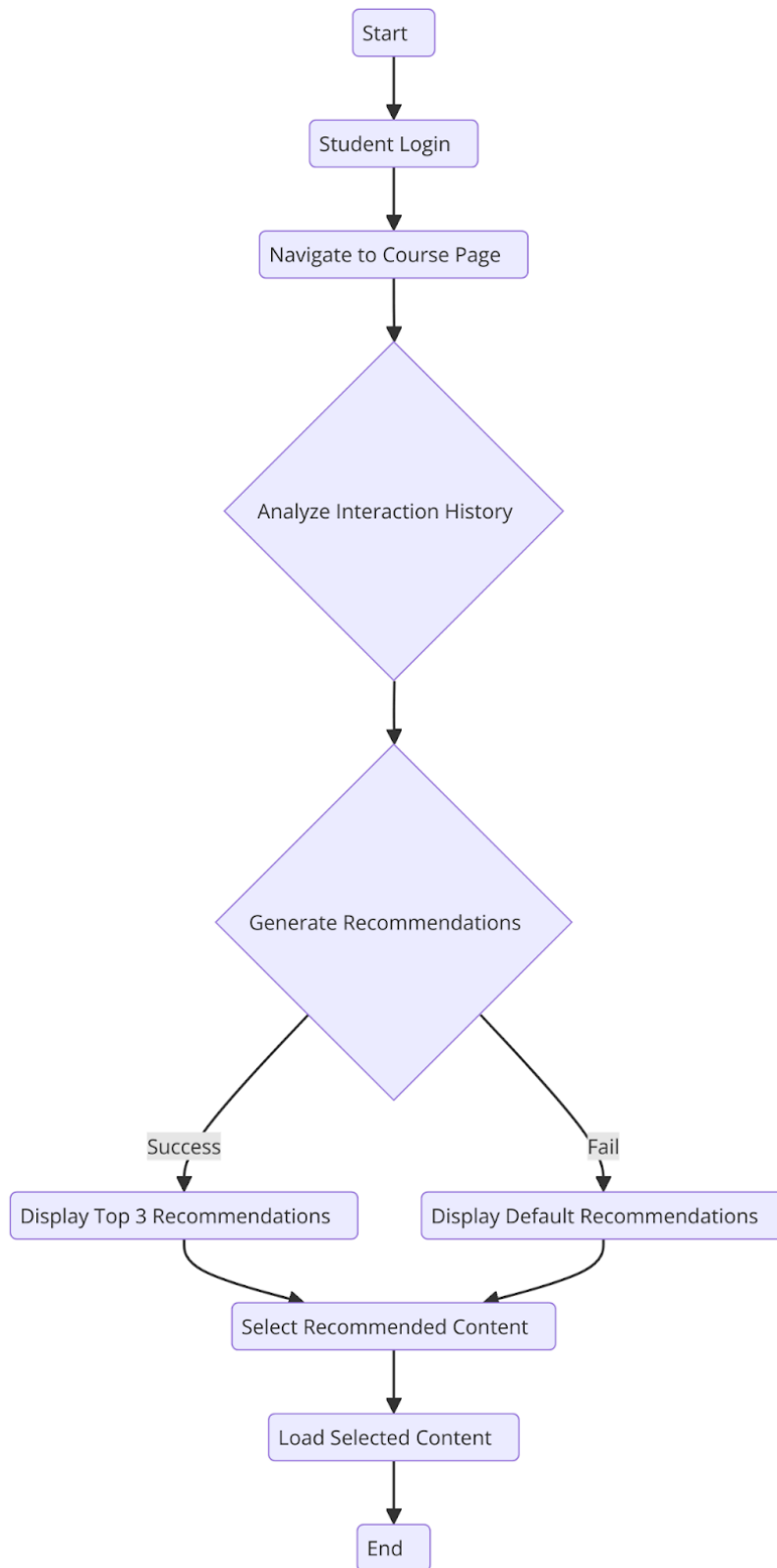


Figure 12: Activity Diagram for Recommender System

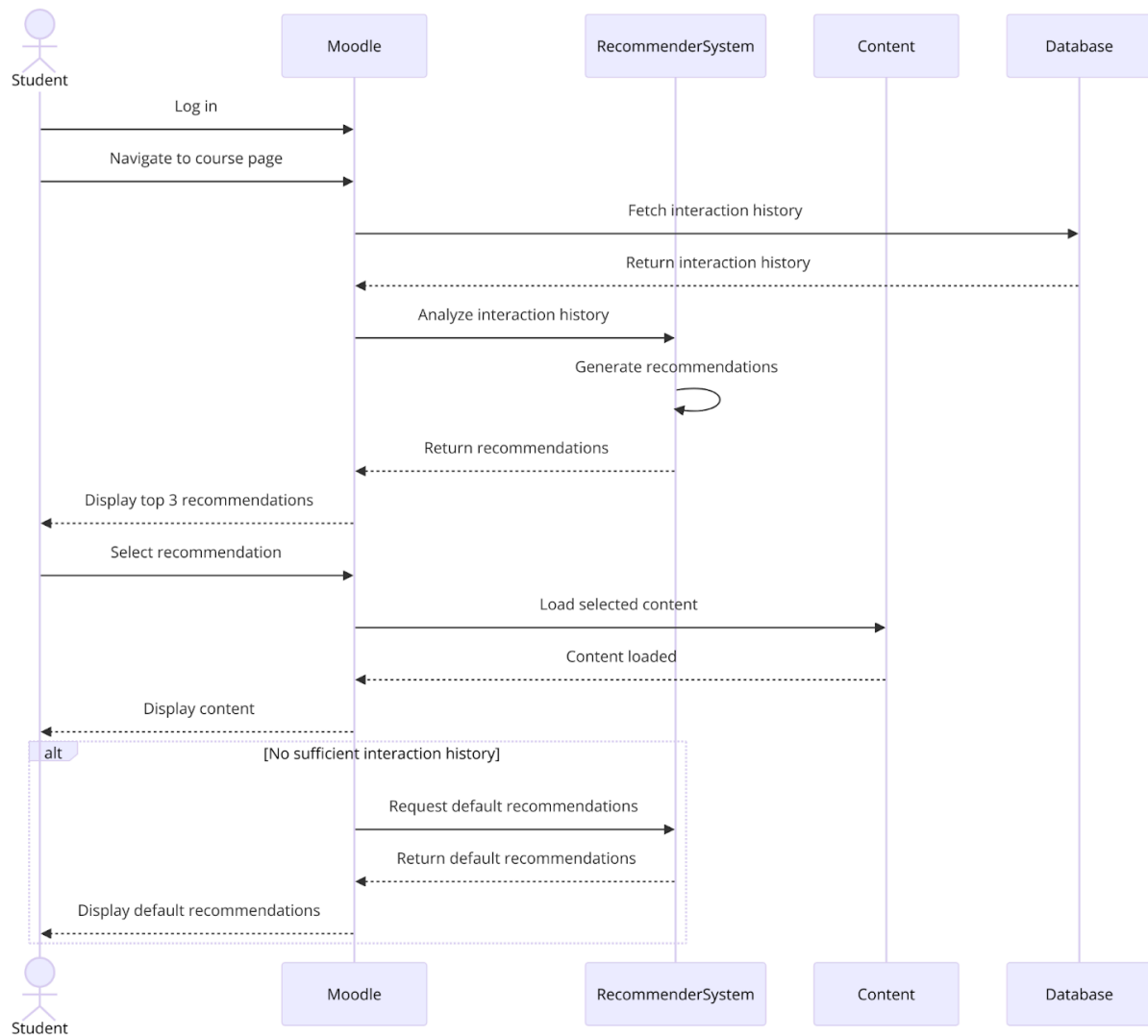


Figure 13: Sequence Diagram for Recommender System

4.2.4 Multimodal Content Generator Design and Modelling

The multimodal content generator, illustrated in Image 3, was designed to transform text-based content into various formats, enhancing accessibility and catering to diverse learning styles. This design not only streamlines the content creation process for instructors but also ensures that students have access to course materials in their preferred format, thereby supporting diverse learning needs. The design key design considerations are shown in the table below.

Table 21: Design Consideration for Multimodal Content Generator

Design Consideration	Description
Flexible Input Handling	The system accepts various text-based formats.
Format Selection	Instructors can choose desired output formats (audio, slides, video).

Automated Processing	Leveraging external services for efficient content transformation.
Quality Assurance	A review and approval process ensures the generated content meets educational standards.
Error Handling	Robust error detection and troubleshooting mechanisms are in place to manage technical issues during content generation.

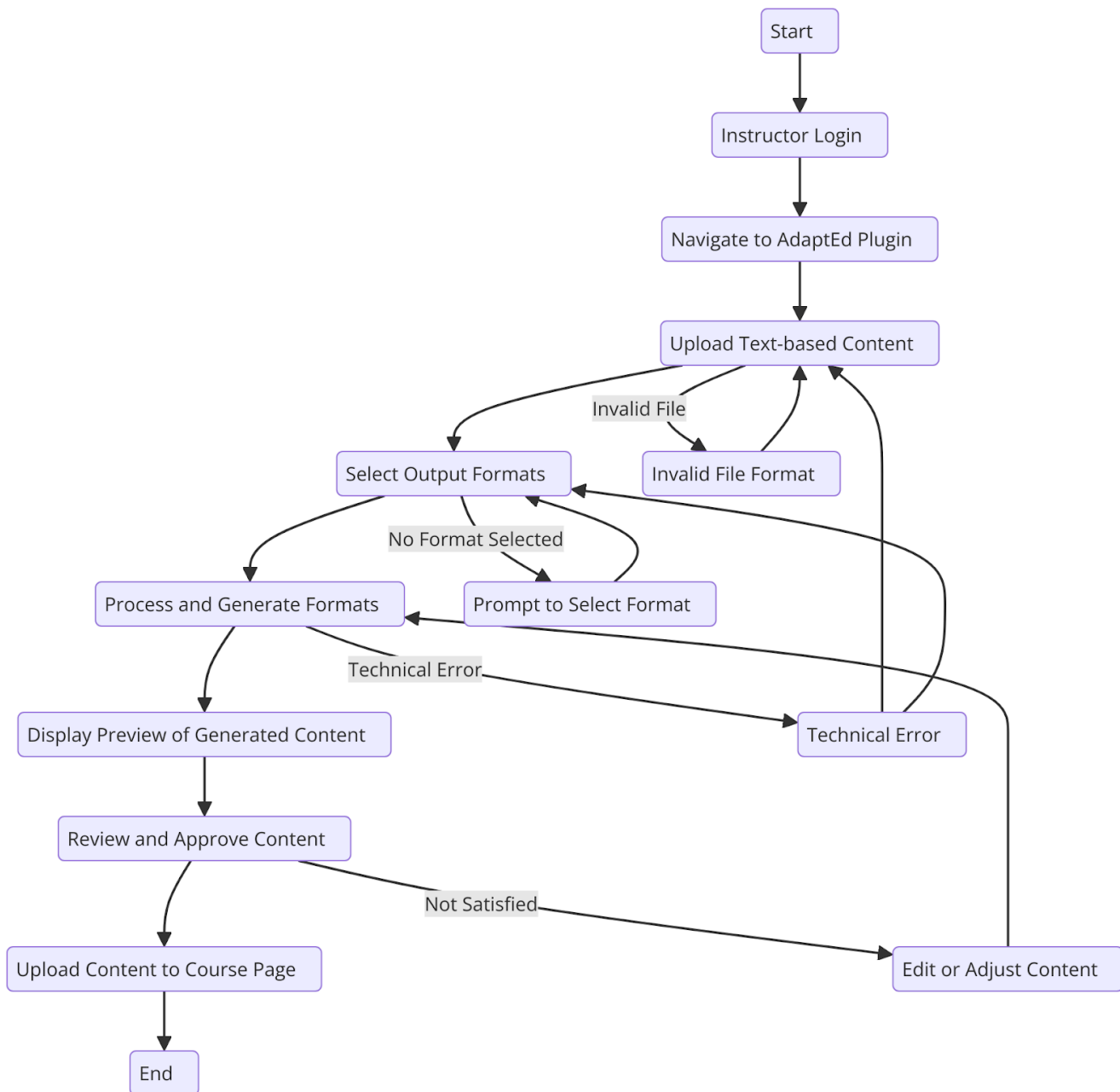


Figure 14: Activity Diagram for Multimodal Content Generator

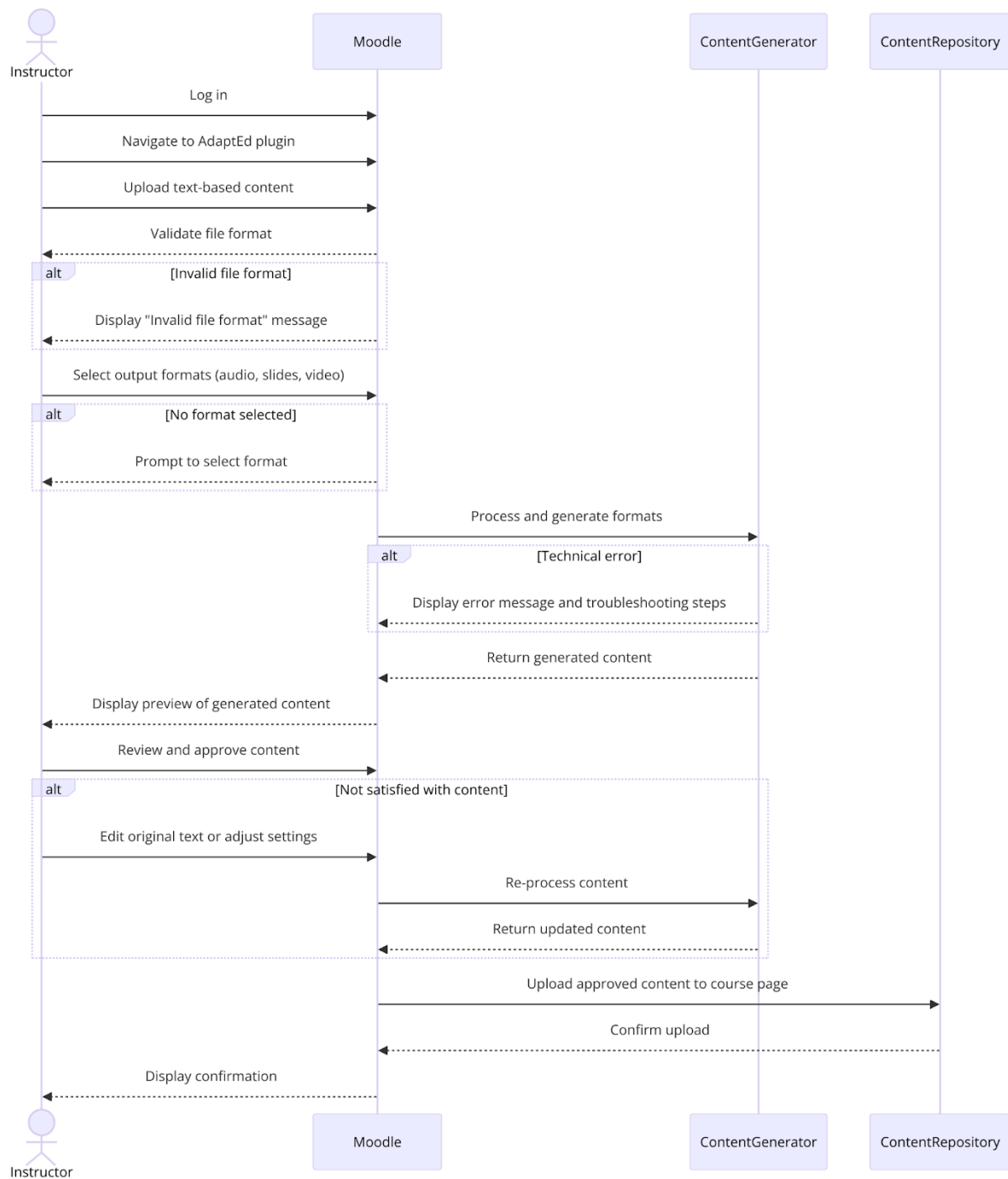


Figure 15: Sequence Diagram for Multimodal Content Generator

4.3 Implementation

The implementation phase of the AdaptEd plugin marked the transition from design to functional software, embodying the project's core objectives of enhancing personalised learning experiences within the Moodle LMS. This phase was characterised by an iterative and incremental development approach, which allowed for continuous refinement and adaptation of each feature in response to

emerging challenges and opportunities. The decision to develop each feature/service individually was strategic, aligning with the microservices architecture outlined in the design phase.

The implementation leveraged a robust tech stack, carefully selected to meet the project's objectives:

- Docker for containerisation, ensuring consistency across development, testing, and production environments.
- PHP 8.1 for core Moodle integration, providing performance improvements and new features over previous versions.
- PostgreSQL 13 as the relational database, chosen for its reliability and advanced features suitable for complex queries and data management.
- Python for backend services, particularly for the recommender system and multimodal content generator, due to its rich ecosystem of machine learning and multimedia processing libraries.
- NGINX as a reverse proxy and load balancer, crucial for managing traffic and ensuring optimal performance.

The project files are well structured with each services having its dedicated directory.

4.3.1 Docker and Moodle Set up

Docker provides a consistent and reproducible environment across different stages of development and deployment. The *docker-compose.yaml* file serves as the blueprint for the entire system architecture, defining five key services: db, moodle, recommender, multimodal, and nginx, volumes, and a defined network for secure communication between the services. By isolating the computationally intensive features as separate services, we ensure that the main Moodle instance remains responsive. This architecture allows for independent scaling of each services as user demand grows without affecting the whole application.

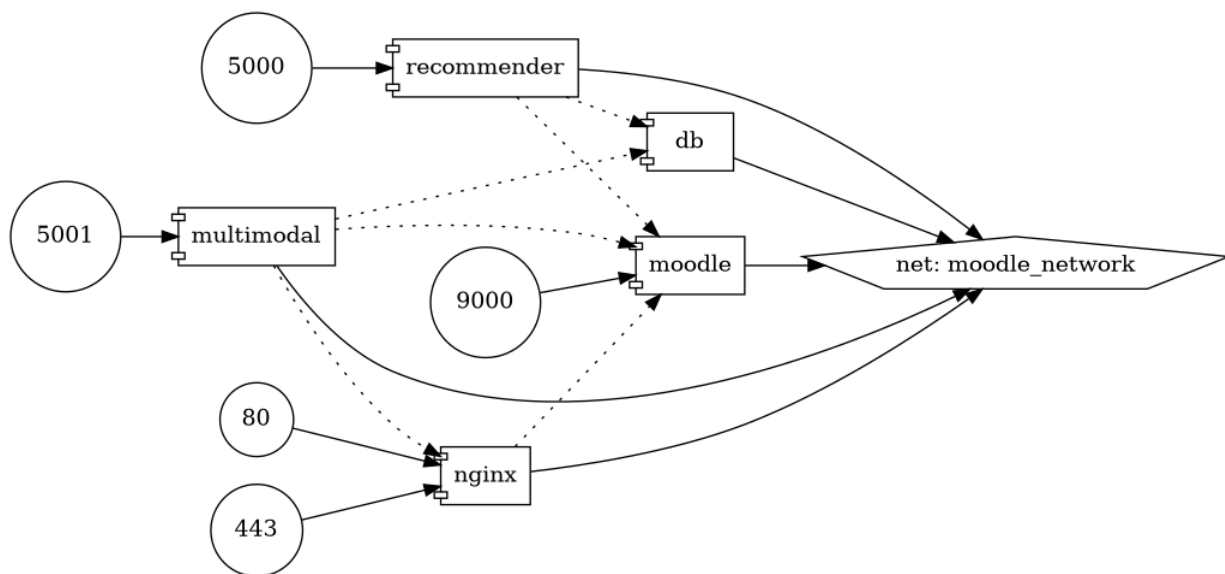
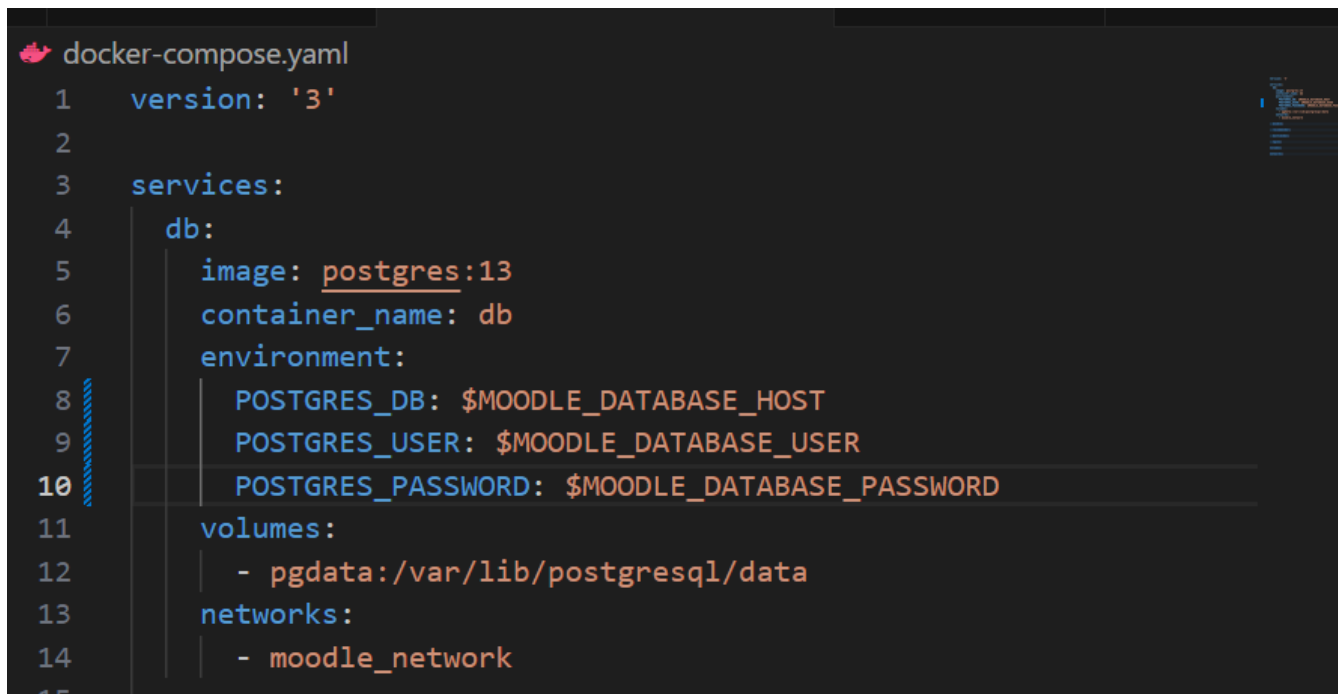


Figure 16: Docker Set up Visualisation

Database setup

The setup uses PostgreSQL 13 as the database backend. This choice is appropriate for a project of this scale, as PostgreSQL offers robust performance, reliability, and advanced features that can support complex queries required for the recommender system. The use of environment variables for database

credentials enhances security and configurability. With potentially many users accessing the system simultaneously (students, educators, AI processes), PostgreSQL's excellent concurrency control through Multi-Version Concurrency Control (MVCC) ensures high performance and data consistency under heavy loads. The recommender and multimodal services depend on the database (db). This is to make sure that the database service is up and running and is able to provide the services with the necessary data before starting the services. If the database is not running and the services are started, the recommender system will use the cached recommendations to display the last set of recommendations but the multimodal service interacts with the database actively, temporarily saving and retrieving generated content during the conversion process, hence it will throw an error as shown in the testing table. The plugin is designed to use the database of the Moodle instance it is running on so it is imperative that additional tables needed by the plugin are created using XML as shown in the image below. Each table definition consists of fields, keys, and indexes.

A screenshot of a code editor with a dark theme. The file is named 'docker-compose.yaml' and is located in a directory indicated by a red folder icon. The code is as follows:

```
1  version: '3'
2
3  services:
4    db:
5      image: postgres:13
6      container_name: db
7      environment:
8        POSTGRES_DB: $MOODLE_DATABASE_HOST
9        POSTGRES_USER: $MOODLE_DATABASE_USER
10       POSTGRES_PASSWORD: $MOODLE_DATABASE_PASSWORD
11      volumes:
12        - pgdata:/var/lib/postgresql/data
13      networks:
14        - moodle_network
15
```

Figure 17: Database Docker Set up

```

1 <?xml version="1.0" encoding="UTF-8" ?>
2 <XMLDB PATH="local/adapted/db" VERSION="20240706" COMMENT="XMLDB file for Moodle local/adapted plugin"
3   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4   xsi:noNamespaceSchemaLocation="../../../../lib/xmldb/xmldb.xsd">
5   <TABLES>
6     <TABLE NAME="local_adapted_course_meta" COMMENT="Custom metadata for courses">...
22   </TABLE>
23
24   <TABLE NAME="local_adapted_course_recomm" COMMENT="Course recommendations">...
40   </TABLE>
41
42   <TABLE NAME="local_adapted_chat_log" COMMENT="Stores chat log entries">...
56   </TABLE>
57
58   <TABLE NAME="local_adapted_stats" COMMENT="Stores plugin statistics">...
72   </TABLE>
73
74   <TABLE NAME="local_adapted_jobs" COMMENT="Stores job information for multimodal content generation">...
86   </TABLE>
87 </TABLES>
88 </XMLDB>

```

Figure 18: Database Tables

```

2 <XMLDB PATH="local/adapted/db" VERSION="20240706" COMMENT="XMLDB file for Moodle local/adapted plugin"
5   <TABLES>
6     <TABLE NAME="local_adapted_course_meta" COMMENT="Custom metadata for courses">
7       <FIELDS>
8         <FIELD NAME="id" TYPE="int" LENGTH="10" NOTNULL="true" SEQUENCE="true"/>
9         <FIELD NAME="courseid" TYPE="int" LENGTH="10" NOTNULL="true" SEQUENCE="false" COMMENT="ID of the course"/>
10        <FIELD NAME="difficulty_level" TYPE="char" LENGTH="20" NOTNULL="false" SEQUENCE="false" COMMENT="Difficulty level of the course"/>
11        <FIELD NAME="estimated_duration" TYPE="int" LENGTH="10" NOTNULL="false" SEQUENCE="false" COMMENT="Estimated duration in hours"/>
12        <FIELD NAME="tags" TYPE="text" NOTNULL="false" SEQUENCE="false" COMMENT="Tags associated with the course"/>
13        <FIELD NAME="last_updated" TYPE="int" LENGTH="10" NOTNULL="true" SEQUENCE="false" COMMENT="Timestamp of last update"/>
14      </FIELDS>
15      <KEYS>
16        <KEY NAME="primary" TYPE="primary" FIELDS="id"/>
17        <KEY NAME="courseid" TYPE="foreign" FIELDS="courseid" REFTABLE="course" REFFIELDS="id"/>
18      </KEYS>
19      <INDEXES>
20        <INDEX NAME="courseid" UNIQUE="true" FIELDS="courseid"/>
21      </INDEXES>
22    </TABLE>
23
24    <TABLE NAME="local_adapted_course_recomm" COMMENT="Course recommendations">...
40  </TABLE>
41
42  <TABLE NAME="local_adapted_chat_log" COMMENT="Stores chat log entries">...
56  </TABLE>
57

```

Figure 19: Database Table Definition with XML

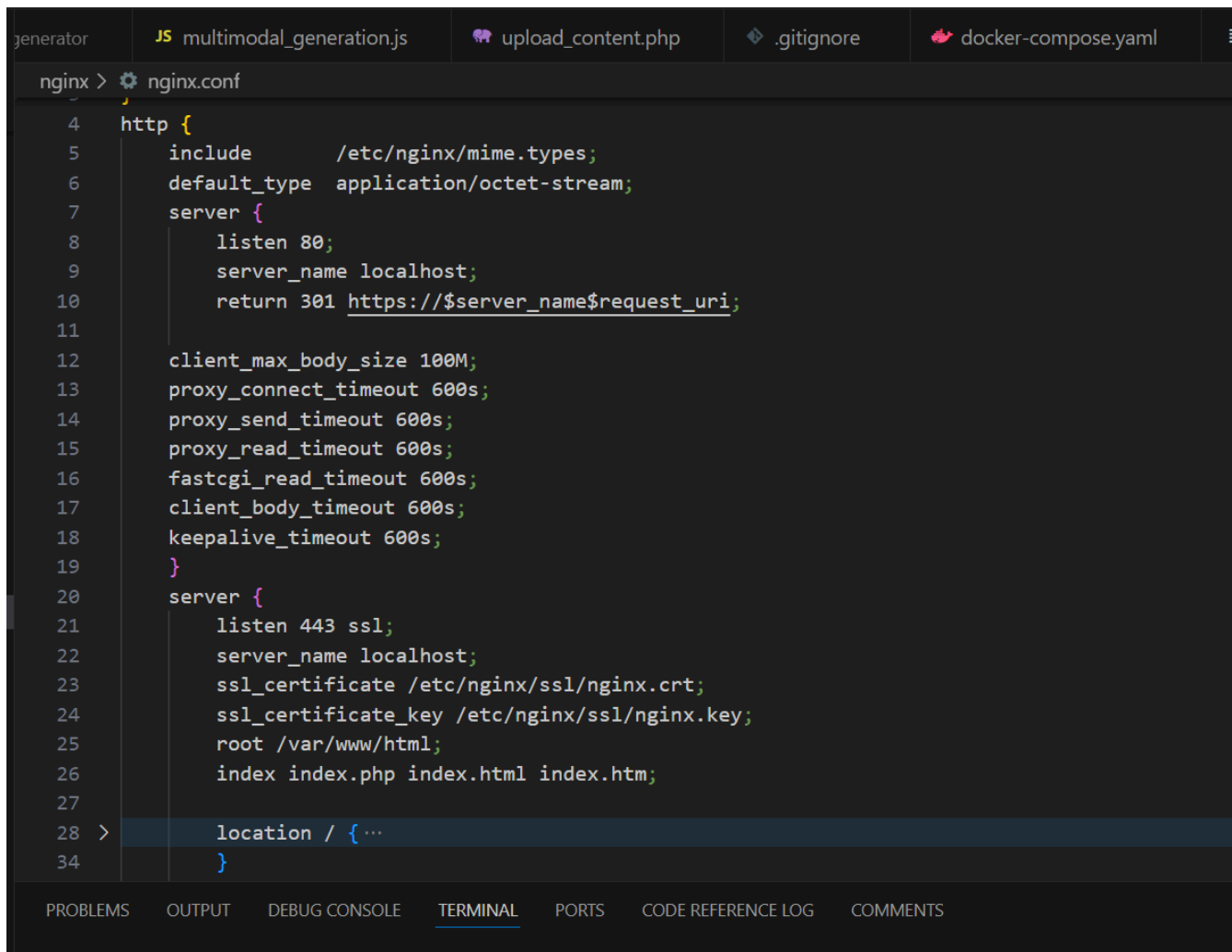
NGINX set up

Nginx was implemented as a reverse proxy and load balancer, a decision driven by several factors such as SSL termination, load distribution and static content serving. It forces all HTTP traffic to redirect to HTTPS to ensure that all client-server communications are encrypted, protecting user data and adhering to modern web security standards. SSL certificates are mounted as volumes in the Docker setup, allowing for easy management and updates of SSL certificates without rebuilding the container. This

approach facilitates agile security management, crucial for maintaining the integrity of the learning environment. The ***nginx.conf*** file defines the port for HTTP (80) and HTTPS (443) requests, and delegates PHP processing to the Moodle container leveraging Docker's internal networking. Only the NGINX container exposes ports to the host system minimizing the attack surface by limiting direct access to application containers. The use of ***fastcgi_pass moodle:9000*** demonstrates the seamless integration between NGINX and the PHP-FPM process running in the Moodle container. The configuration file also provides dedicated routing for the AdaptEd plugin, ensuring that plugin-specific requests are correctly handled and allows for easy integration of the plugin within the larger Moodle ecosystem. Due to the resource intensive nature of the multimodal content generation process, the Nginx was optimised for handling large file uploads and long running processes. Nginx also prevents access to hidden files and directories, enhancing security.

```
nginx:
  build:
    context: ./nginx
    dockerfile: Dockerfile
  container_name: nginx
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./nginx/nginx.conf:/etc/nginx/nginx.conf
    - ./nginx/ssl:/etc/nginx/ssl
    - ./log/nginx:/var/log/nginx
    - moodledata:/var/www/html:ro
    - ./local/adapted:/var/www/html/local/adapted
    - ./blocks/multimodal_generator:/var/www/html/blocks/multimodal_generator
  depends_on:
    - moodle
  networks:
    - moodle_network
```

Figure 20: Nginx Docker Set up



```
generator JS multimodal_generation.js upload_content.php .gitignore docker-compose.yaml
nginx > nginx.conf
4 http {
5     include      /etc/nginx/mime.types;
6     default_type  application/octet-stream;
7     server {
8         listen 80;
9         server_name localhost;
10        return 301 https://$server_name$request_uri;
11
12        client_max_body_size 100M;
13        proxy_connect_timeout 600s;
14        proxy_send_timeout 600s;
15        proxy_read_timeout 600s;
16        fastcgi_read_timeout 600s;
17        client_body_timeout 600s;
18        keepalive_timeout 600s;
19    }
20    server {
21        listen 443 ssl;
22        server_name localhost;
23        ssl_certificate /etc/nginx/ssl/nginx.crt;
24        ssl_certificate_key /etc/nginx/ssl/nginx.key;
25        root /var/www/html;
26        index index.php index.html index.htm;
27
28        > location / { ...
34    }
```

Figure 21: Nginx Configuration

Moodle Set up

The Moodle service was built using a custom Dockerfile, which installs necessary PHP extensions and clones the Moodle 4.0.4 stable branch. This approach ensures compatibility with the latest Moodle features while maintaining stability. PHP 8.1 with FPM (FastCGI Process Manager) was used as the base image because of its advanced process management with graceful stop and start and to handle the concurrency issues of PHP's built-in server by spawning multiple workers, which can handle multiple requests simultaneously. The *config.php* file was mounted directly into the container, allowing for easy configuration changes without rebuilding the image. The AdaptEd plugin directory was also mounted into the container for direct and seamless integration with the Moodle instance. The Moodle service was optimised by configuring OPcache as shown in the image below for handling the increased computational load introduced by the AdaptEd plugin. The service exposes port 9000 for the PHP-FPM which it uses to communicate with the Nginx service.

```

moodle:
  build: .
  container_name: moodle
  ports:
    - "9000:9000"
  environment:
    - MOODLE_DATABASE_TYPE=pgsql
    - MOODLE_DATABASE_HOST=$MOODLE_DATABASE_HOST
    - MOODLE_DATABASE_PORT=$MOODLE_DATABASE_PORT
    - MOODLE_DATABASE_NAME=$MOODLE_DATABASE_NAME
    - MOODLE_DATABASE_USER=$MOODLE_DATABASE_USER
    - MOODLE_DATABASE_PASSWORD=$MOODLE_DATABASE_PASSWORD
  volumes:
    - moodledata:/var/www/html
    - ./config.php:/var/www/html/config.php
    - ./moodle_themes/adaptable:/var/www/html/theme/adaptable
    - ./local/adapted:/var/www/html/local/adapted
    - ./blocks/multimodal_generator:/var/www/html/blocks/multimodal_generator
  networks:
    - moodle_network

```

Figure 22: Moodle Docker Set up

```

1  FROM php:8.1-fpm
2
3  # Create www-data user if it doesn't exist
4  > RUN set -x \...
5
6
7
8  # Install dependencies
9  > RUN apt-get update && apt-get install -y \...
10
11
12
13
14
15
16
17
18
19  # Install PHP extensions
20  RUN docker-php-ext-install gd soap zip intl pgsql pdo_pgsql exif
21
22  # Set the working directory
23  WORKDIR /var/www/html
24
25  # Clone Moodle repository
26  RUN git clone -b MOODLE_404_STABLE git://git.moodle.org/moodle.git .
27
28  # Set permissions
29  RUN chown -R www-data:www-data /var/www/html
30
31  # Create moodledata directory and set permissions
32  > RUN mkdir -p /var/www/moodledata && \...
33
34
35
36  COPY www.conf /usr/local/etc/php-fpm.d/www.conf
37

```

Figure 23: Moodle Dockerfile

```

# Configure PHP settings
RUN echo "max_input_vars = 5000" >> /usr/local/etc/php/conf.d/docker-php-ext-max-input-vars.ini
RUN echo "memory_limit = 512M" >> /usr/local/etc/php/conf.d/docker-php-memlimit.ini

# Enable and configure opcache
RUN docker-php-ext-enable opcache && \
    echo "opcache.enable=1" >> /usr/local/etc/php/conf.d/docker-php-ext-opcache.ini && \
    echo "opcache.enable_cli=1" >> /usr/local/etc/php/conf.d/docker-php-ext-opcache.ini && \
    echo "opcache.memory_consumption=128" >> /usr/local/etc/php/conf.d/docker-php-ext-opcache.ini && \
    echo "opcache.interned_strings_buffer=8" >> /usr/local/etc/php/conf.d/docker-php-ext-opcache.ini && \
    echo "opcache.max_accelerated_files=4000" >> /usr/local/etc/php/conf.d/docker-php-ext-opcache.ini && \
    echo "opcache.revalidate_freq=60" >> /usr/local/etc/php/conf.d/docker-php-ext-opcache.ini && \
    echo "opcache.fast_shutdown=1" >> /usr/local/etc/php/conf.d/docker-php-ext-opcache.ini

# Expose port 9000 for PHP-FPM
EXPOSE 9000

```

Figure 24: Moodle PHP Configuration

4.3.2 Virtual Assistant Implementation

The implementation of the Virtual Assistant represents a sophisticated integration of natural language processing capabilities with Moodle's existing infrastructure. This implementation directly addresses the project objective of developing an AI-driven virtual assistant to provide personalised guidance, support, and feedback to students using natural language processing and conversational AI technologies. The core logic of the Virtual Assistant is implemented in PHP which made it integrate directly and seamlessly with Moodle. Therefore, the virtual assistant was not separated as a service on its own to reduce the complexity of the docker setup.

Architecture and Integration

The Virtual Assistant is implemented through a multi-tiered architecture that seamlessly integrates with Moodle:

1. **Frontend Interface:** The chatbot interface is implemented using a combination of HTML, CSS, and JavaScript. This approach ensures a responsive and user-friendly interface that can be easily embedded within the Moodle environment. The use of Mustache templating allows for dynamic content injection, enhancing the maintainability and scalability of the interface.

```

local > adapted > templates > chatbot.mustache
1 <div id="chatbot-container">
2   <div id="chatbot-icon">
3     
4   </div>
5   <div id="chatbot-interface">
6     <div id="chat-header">
7       <span>Chat Assistant</span>
8       <button id="close-chat">✕</button>
9     </div>
10    <div id="chat-content"></div>
11    <div id="chat-input-container">
12      <input type="text" id="chat-input" placeholder="Type a message...">
13      <button id="send-button">Send</button>
14    </div>
15  </div>
16 </div>

```

Figure 25: Virtual Assistant Mustache

2. Backend Processing: As shown in the sequence diagram in the design phase, the virtual assistant takes the user's input, check the context, process the request and send a response to the user. This is implemented by calling certain functions in the backend. When the bot receives a prompt, it checks the context, user's capabilities/role, request method, validates the user input, set the username, context, and message, and call the *get_response* function in the *local_adapted_chatbot* class.

```

10 $context = context_system::instance();
11 require_capability('moodle/site:config', $context);
12
13 header('Content-Type: application/json');
14
15 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
16     $input = json_decode(file_get_contents('php://input'), true);
17
18     $message = $input['message'] ?? '';
19     $username = $input['username'] ?? '';
20     $context = $input['context'] ?? [];
21
22     // Validate input
23     if (empty($message)) {
24         echo json_encode(['response' => 'Message is required']);
25         exit;
26     }
27
28     try {
29         $response = local_adapted_chatbot::get_response($message, $username, $context);
30         // Debugging: Check the generated response
31         error_log('Generated response: ' . $response);
32         echo json_encode(['response' => $response]);
33     } catch (Exception $e) {
34         // Debugging: Log any exceptions
35         error_log('Error: ' . $e->getMessage());
36     }
37 }

```

Figure 26: Backend Processing

The *local_adapted_chatbot* class has two functions: *get_response* and *call_moodle_function* functions. The *get_response* function checks if the API key is set and set the OpenAI API url. The function defines two roles: system and user, which are used to query the OpenAI API. The *system* role provides instruction and guides the behaviour of the GPT model as shown in the image below. This context-setting ensures that the AI's responses are tailored to the educational environment. The function uses the OpenAI API GPT 3.5 Turbo model for natural language processing and also includes functions to get context-aware information from the Moodle knowledge base. When the function is called, it analyses the user input and determine its context. If the query is related to Moodle, it calls the *call_moodle_function* function, otherwise it proceeds to query the OpenAI API knowledge base. When the *call_moodle_function* function is called, it takes the function name and arguments sent by the *get_response* function, uses the *switch* statement to match the received function with the case and process the request. The response is then sent to the *get_response* function which calls the OpenAI API again with the received response. The response is then sort and the topmost response is returned and displayed to the user. The integration of the virtual assistant with Moodle's database enables it to provide real-time personalised information about courses, assignments,grades and deadlines while leveraging the general AI capabilities.

```

local > adapted > lib > chatbot.php > local_adapted_chatbot > get_response
4 class local_adapted_chatbot {
5     public static function get_response($message) {
6         global $CFG, $DB;
7
8         $api_key = get_config('local_adapted', 'api_key');
9         if (empty($api_key)) { ...
10     }
11
12     $url = 'https://api.openai.com/v1/chat/completions';
13     $data = [
14         'model' => 'gpt-3.5-turbo',
15         'messages' => [
16             [
17                 'role' => 'system',
18                 'content' => "You are a helpful assistant for a Moodle learning management system.
19                             You can provide information about courses, assignments, grades, and deadlines.
20                             When asked about specific courses, modules, content, assignments, grades,
21                             or deadlines, first extract the relevant information from the user's query,
22                             then use the provided functions to fetch the required data from Moodle before
23                             answering. Always consider the user's context and provide personalized responses.
24                             For queries that are not related to the Moodle context, use OpenAI gpt"
25             ],
26             [
27                 'role' => 'user',
28                 'content' => $message
29             ]
30         ],
31         'functions' => [
32             [
33                 'name' => 'get_course_info'
34             ]
35         ]
36     ];
37
38     $response = curl_exec($url, $data);
39     if ($response === false) {
40         return 'Error: ' . curl_error($ch);
41     }
42
43     $json = json_decode($response, true);
44     if ($json['error']) {
45         return 'Error: ' . $json['error']['message'];
46     }
47
48     $completion = $json['choices'][0]['text'];
49     return $completion;
50 }

```

 AWS: profile:default
  Amazon Q
 Ln 25, Col 33 Spaces: 4 UTF-8 CRLF  

Figure 27: *local_adapted_chatbot* Class

```

upload_content.php  .gitignore  docker-compose.yaml M  JS chatbot.js  lib.php  chatbot.php
local > adapted > lib > chatbot.php > local_adapted_chatbot > get_response
4  class local_adapted_chatbot {
5      public static function get_response($message) {
144      $message = $response['choices'][0]['message'];
145
146      if (isset($message['function_call'])) {
147          $function_name = $message['function_call']['name'];
148          $arguments = json_decode($message['function_call']['arguments'], true);
149
150          $function_response = self::call_moodle_function($function_name, $arguments);
151
152          // Call API again with the function response
153          $data['messages'][] = [
154              'role' => 'function',
155              'name' => $function_name,
156              'content' => json_encode($function_response)
157          ];
158
159          $options['http']['content'] = json_encode($data);
160          $context = stream_context_create($options);
161          $result = file_get_contents($url, false, $context);
162          $response = json_decode($result, true);
163      }
164
165      return $response['choices'][0]['message']['content'] ?? 'No response';
166  }
167
168  private static function call_moodle_function($function_name, $arguments) { ...
320  }
321  }

```

⊗ AWS: profile:default Amazon Q Ln 25, Col 33 Spaces: 4 UTF-8 CRLF

Figure 28: *local_adapted_chatbot Class contd.*

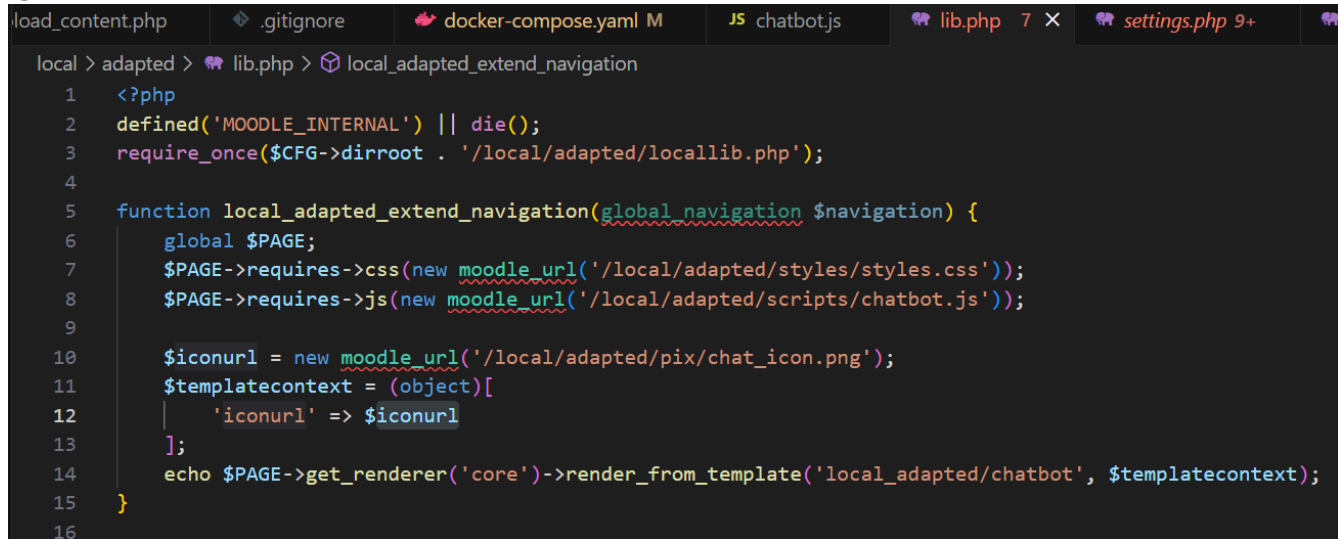
3. Virtual Assistant Integration: The virtual assistant leverages Moodle’s authentication and authorization as shown in the image below to ensure that only authenticated users can access the chatbot. The virtual assistant is integrated with Moodle as part of its global navigation, that is, it is visible on all the pages except the log in page.

```

it.php  .gitignore  docker-compose.yaml M  JS chatbot.js  lib.php 7
local > adapted > chatbot.php > ...
1  <?php
2  require_once(' ../../config.php');
3  require_once($CFG->dirroot . '/local/adapted/lib/chatbot.php');
4
5  require_login();
6  if (!isloggedin() || isguestuser()) {
7      throw new moodle_exception('noguest');
8  }
9

```

Figure 29: Moodle Authentication



```
load_content.php  .gitignore  docker-compose.yaml M  JS chatbot.js  lib.php 7 X  settings.php 9+

local > adapted > lib.php > local_adapted_extend_navigation
1  <?php
2  defined('MOODLE_INTERNAL') || die();
3  require_once($CFG->dirroot . '/local/adapted/lib.php');
4
5  function local_adapted_extend_navigation(global_navigation $navigation) {
6      global $PAGE;
7      $PAGE->requires->css(new moodle_url('/local/adapted/styles/styles.css'));
8      $PAGE->requires->js(new moodle_url('/local/adapted/scripts/chatbot.js'));
9
10     $iconurl = new moodle_url('/local/adapted/pix/chat_icon.png');
11     $templatecontext = (object)[
12         'iconurl' => $iconurl
13     ];
14     echo $PAGE->get_renderer('core')->render_from_template('local_adapted/chatbot', $templatecontext);
15 }
16
```

Figure 30: Navigation Extension

4.3.3 Recommender System Implementation

The recommender system embodies the intersection of data-driven decision making and adaptive learning technologies, directly addressing the need for tailored educational content delivery in modern e-learning environments. The implementation of the recommender system for the AdaptEd plugin represents a critical component in achieving the project's objective of providing personalised learning experiences. Python is used for the implementation of the recommender system backend while PHP is used for its integration with Moodle and Mustache is used for templating. The use of Python for the recommender system allows for the incorporation of advanced machine learning libraries like Scikit-learn and Surprise, which are not natively available in Moodle's primary language (PHP). This technological choice enables the implementation of sophisticated hybrid recommendation algorithms crucial for the project's goal of intelligent and context-aware content suggestions.

Architecture and Integration

The recommender system was developed using an incremental approach. The backend was first developed and evaluated before it was integrated with Moodle. The hybrid approach was used combining the strengths of content based filtering and collaborative filtering with other sophisticated techniques such as score boosting. The core of the recommender system is encapsulated in the **MoodleRecommender** class, which manages data loading, preprocessing, model training, and recommendation generation. The MoodleRecommender Class initializes three core components which will be discussed in detail in the subsequent subsections:

- SVD for collaborative filtering
- TfidfVectorizer for content-based filtering
- RandomForestRegressor for the hybrid model

Codiumate: Options | Test this class

class MoodleRecommender:

Codiumate: Options | Test this method

def __init__(self, engine):

self.engine = engine

self.cf_model = SVD()

self.content_vectorizer = TfidfVectorizer(stop_words='english')

self.content_vectors = None

self.user_preferences = {}

self.hybrid_model = RandomForestRegressor(n_estimators=100, random_state=42)

self.content_vector_size = None

self.course_content = None

Figure 31: MoodleRecommender Class

Data Collection and Preprocessing

Data is the foundation of all recommender systems but suitable comprehensive educational data for this kind of recommendation system is not publicly available mostly due to its ethical and privacy implications. This unavailability of data posed a challenge at the beginning of the development. To address this, synthetic data that closely mimics Moodle data structure was generated using Python Faker library and saved to the PostgreSQL database. The **load_data** function in the recommender.py loads data from the database, including user information, course details, course content, and user interactions. The SQL queries are carefully crafted to join relevant tables and aggregate interaction data. The **preprocess_data** function preprocess the data and sets up the necessary data structures and transformations required for collaborative filtering, content-based filtering, and the hybrid model. The function transforms the raw interaction data into a format suitable for collaborative filtering by creating a Reader object with a rating scale from 0 to 100. The choice of using interaction_count as a proxy for ratings is justified in an educational context where explicit ratings are often unavailable. This approach aligns with the project objective of leveraging implicit feedback to generate recommendations. The data for the content based filtering was prepared using TF-IDF (Term Frequency-Inverse Document Frequency), which is a numerical method used to determine the importance of a word in a document in a collection. This is important to determine the content similar to those a user has liked in the past, based on item features.

```

79 def preprocess_data(self):
80     # Prepare data for collaborative filtering
81     reader = Reader(rating_scale=(0, 100))
82     self.cf_data = Dataset.load_from_df(self.user_content_interactions[['userid', 'cmid', 'interaction_count']
83
84     # Prepare data for content-based filtering
85     self.course_content['content'] = self.course_content['name'] + ' ' + self.course_content['intro'] + ' ' +
86     self.content_vectors = self.content_vectorizer.fit_transform(self.course_content['content'])
87     self.content_vector_size = self.content_vectors.shape[1]
88
89     # Normalize content vectors
90     self.content_vectors = normalize(self.content_vectors.toarray(), norm='l2', axis=1)
91
92     # Create a dictionary mapping content ID to its vector
93     self.content_vector_dict = {id: vec for id, vec in zip(self.course_content['id'], self.content_vectors)}
94
95     # Prepare data for hybrid model
96     self.hybrid_data = self.user_content_interactions.copy()
97     self.hybrid_data['cf_score'] = 0.0
98     self.hybrid_data['cb_score'] = 0.0

```

Figure 31: Data Preprocessing

The *preprocess_data* function concatenates the 'name', 'intro', and 'type' columns of the *course_content* DataFrame into a new 'content' column and uses a *TfidfVectorizer* object (*content_vectorizer*) from *scikit-learn* to transform the textual content of course items (*self.course_content['content']*) into a matrix of TF-IDF features (*self.content_vectors*). This TF-IDF matrix represents the importance of each word in the content across all course items and is the multiplication of the TF and the IDF as shown in formula 1 below. The matrix is then normalized using the *normalize* function from *Surprise* library to ensure consistent scales for further processing and creates a dictionary (*content_vector_dict*) mapping content IDs to their respective vectors for efficient retrieval during the recommendation process.

Although TF-IDF is effective for text-based content it may not capture semantic similarities effectively. More advanced techniques like word embeddings could potentially improve this aspect. *Word2Vec* was also considered for the text vectorization but was later not used due to a compatibility issue with the *Scipy* version already installed and used by *Numpy*. The choice of using *interaction_count* as a proxy for ratings is justified in an educational context where explicit ratings are often unavailable. This approach aligns with the project objective of leveraging implicit feedback to generate recommendations. For the hybrid model, the *preprocess_data* function creates a copy of the *user_content_interactions* DataFrame as *hybrid_data* and initializes 'cf_score' and 'cb_score' columns in *hybrid_data* with default values of 0.0. These columns will store the collaborative filtering and content-based filtering scores, respectively.

Content Based Filtering

Content-based filtering is a recommendation technique that suggests items to users based on the similarity of items themselves. The *get_content_based_recommendations* function generates content-based recommendations for a given user by analyzing their past interactions with course content and computing similarity scores between the user's content profile and all available content. It first retrieves the interactions of a specific user from *user_content_interactions* based on the *user_id*. It then selects the content that the user has interacted with from the *course_content* based on the *cmid* (content id) present in the user interactions. It calculates a user profile by taking the mean of the TF-IDF content vectors of the selected user content generated in the *preprocess_data* function. This profile represents the user's preferences or characteristics based on the content they have interacted

with. It calculates the cosine similarity between the user profile and all content vectors in `content_vectors`. Cosine similarity measures the similarity between two vectors by calculating the cosine of the angle between them, effectively capturing the similarity in direction rather than magnitude. This is particularly useful in text analysis where the frequency of terms (magnitude) may vary greatly between documents of different lengths. It identifies the top content items with the highest similarity scores to the user profile and returns these top content recommendations as a DataFrame sorted by their similarity scores in descending order.

```

114     def train_cb_model(self):
115         for _, row in self.hybrid_data.iterrows():
116             user_profile = self.get_user_profile(row['userid'])
117             content_vector = self.content_vector_dict.get(row['cmid'], np.zeros((1, self.content_vector_size)))
118             self.hybrid_data.at[_, 'cb_score'] = cosine_similarity(user_profile, content_vector.reshape(1, -1))[0]
119

```

Figure 32: Train CB

```

131     def get_content_based_recommendations(self, user_id, n=10):
132         # Get content-based recommendations
133         user_interactions = self.user_content_interactions[self.user_content_interactions['userid'] == user_id]
134         user_content = self.course_content[self.course_content['id'].isin(user_interactions['cmid'])]
135         if user_content.empty:
136             return pd.DataFrame() # Return empty DataFrame if user has no interactions
137         user_profile = self.content_vectors[user_content.index].mean(axis=0)
138         user_profile = np.asarray(user_profile).flatten()
139         content_scores = cosine_similarity(user_profile.reshape(1, -1), self.content_vectors)[0]
140         top_indices = content_scores.argsort()[-n:][::-1]
141         return self.course_content.iloc[top_indices]
142

```

Figure 33: Content Based Recommendation

Collaborative Filtering

The collaborative filtering approach in this recommender system utilizes the Singular Value Decomposition (SVD) algorithm, a matrix factorization technique that forms the cornerstone of many modern recommender systems. The implementation leverages the Surprise library, which is specifically designed for building and analyzing recommender systems. Unlike e-commerce systems where explicit ratings are common, educational content is rarely rated explicitly by students. By interpreting frequency of interaction as a measure of interest or relevance, the system can generate meaningful recommendations without requiring additional user input. The SVD algorithm decomposes the user-item interaction matrix into the product of three matrices. These matrices capture latent factors that explain the observed user-content interactions. The training of the SVD model is performed in the ***train_cf_model*** function. This function builds a full training set from the collaborative filtering data (`cf_data`) and fits the SVD model to this data. After training, it calculates collaborative filtering scores for each user-content pair in the `hybrid_data` DataFrame, which will be used later in the hybrid model. The ***get_collaborative_recommendations*** function considers all the course contents and use the trained `cf_model` to predict their scores. It then sort and selects the top contents with the highest predicted scores.

```

120     def train_cf_model(self):
121         # Train the collaborative filtering model
122         trainset = self.cf_data.build_full_trainset()
123         self.cf_model.fit(trainset)
124
125         # Calculate CF scores for hybrid model
126         for _, row in self.hybrid_data.iterrows():
127             cf_score = self.cf_model.predict(row['userid'], row['cmid']).est
128             self.hybrid_data.at[_, 'cf_score'] = float(cf_score)
129

```

Figure 34: Train CF

```

218     def get_collaborative_recommendations(self, user_id, n=10):
219         # Get collaborative filtering recommendations
220         all_content = self.course_content['id'].unique()
221         cf_scores = [self.cf_model.predict(user_id, content_id).est for content_id in all_content]
222         top_indices = np.array(cf_scores).argsort()[-n:][::-1]
223         return self.course_content.iloc[top_indices]
224

```

Figure 35: Collaborative Filtering

Hybrid Approach

The hybrid model combines the strengths of content-based and collaborative filtering approaches with other factors. It uses a Random Forest Regressor to combine various features and scores. The Random Forest Regressor is an ensemble learning method that operates by constructing multiple decision trees during training and outputting the mean prediction of the individual trees. It is chosen for its ability to capture non-linear relationships between features and handle high-dimensional data effectively. The hybrid model is trained in the *train_hybrid_model* function. This function uses the *cf_score* and *cb_score* as features and the interaction count as the target variable. The Random Forest Regressor is trained on this data. The *get_hybrid_recommendations* function generates personalised content recommendations for a user within a course. It calculates the *cf_score* and *cb_score* for each content item and uses the trained hybrid model to combine these scores. One of the objectives of the AdaptEd plugin is to recommend course content in the user's preferred format to cater to individual students' learning style. Therefore, the *get_hybrid_recommendations* applies additional weighting based on user preferences for content types and considers the user's interaction patterns within the course. The function also applies a time decay factor to balance recency and historical interactions and boosts scores for content within the current course. The scores used for boosting were adjusted several times to find the best score that gives optimal recommendations. The final recommendations are sorted and the top three recommendations are displayed to the user. The hybrid recommendation has a fallback mechanism such that when a user has no interaction, it uses the content-based filtering recommendation only and when there is no course on the LMS at all, it displays the default recommendations (which are just placeholders).

```

130     def train_hybrid_model(self):
131         X = self.hybrid_data[['cf_score', 'cb_score']]
132         y = self.hybrid_data['interaction_count']
133         self.hybrid_model.fit(X, y)
134

```

Figure 36: Train Hybrid Model

```

249     def get_hybrid_recommendations(self, user_id, course_id, n=10):
250         logging.debug(f"Getting recommendations for user {user_id} in context of course {course_id}")
251     >     if self.course_content is None or self.course_content.empty: ...
254
255         all_content = self.course_content['id'].unique()
256         user_profile = self.get_user_profile(user_id)
257         user_preferences = self.get_user_preference(user_id)
258
259         # Get user's interactions in the current course
260         course_interactions = self.user_content_interactions[
261             (self.user_content_interactions['userid'] == user_id) &
262             (self.user_content_interactions['courseid'] == course_id)
263         ]
264
265         recommendations = []
266     >     for content_id in all_content: ...
314
315         recommendations.sort(key=lambda x: x['score'], reverse=True)
316         top_recommendations = recommendations[:n]
317
318         logging.debug(f"Generated {len(recommendations)} recommendations considering course {course_id} context")
319
320         return pd.DataFrame(top_recommendations)

```

Figure 37: Hybrid Recommendation

Hyperparameter Tuning

As part of this project's objective to develop effective recommendation system through experimentation, the system implements hyperparameter tuning for each of the models. The *tune_hyperparameters* function orchestrates this process.

```

135     def tune_hyperparameters(self):
136         logging.info("Starting hyperparameter tuning")
137         self.tune_svd()
138         self.tune_random_forest()
139         self.tune_tfidf()
140

```

Figure 38: Tune Hyperparameters

The *tune_svd* function uses GridSearchCV from Surprise library to perform a grid search over the specified parameter space. It optimizes for both Root Mean Square Error (RMSE) and Mean Absolute Error (MAE), using a 3-fold cross-validation. The best model and its parameters are then stored for use in the recommender system.

```

141     def tune_svd(self):
142         param_grid = {
143             'n_factors': [50, 100, 150],
144             'n_epochs': [20, 30, 40],
145             'lr_all': [0.002, 0.005, 0.01],
146             'reg_all': [0.02, 0.05, 0.1]
147         }
148
149         gs = SurpriseGridSearchCV(SVD, param_grid, measures=['rmse', 'mae'], cv=3)
150         gs.fit(self.cf_data)
151
152         self.cf_model = gs.best_estimator['rmse']
153         self.cf_best_params = gs.best_params['rmse']
154         logging.info(f"Best SVD parameters: {self.cf_best_params}")

```

Figure 39: Tune SVD

The *tune_tfidf* function uses GridSearchCV from the Scikit-learn library to find the best parameters for the TF-IDF vectorizer. It performs a grid search over the specified parameter grid and uses a custom scorer to evaluate the vocabulary size. This helps optimise the TF-IDF vectorization process for better content-based recommendations. The *tune_random_forest* function uses RandomizedSearchCV to perform a randomized search over the specified parameter distributions. It uses a custom score function to evaluate model performance with 3-fold cross-validation and stores the best model and its parameters for use in the recommendation system. The use of GridSearchCV for SVD and TF-IDF and RandomizedSearchCV for Random Forest Regressor, allows for the efficient exploration of the hyperparameter space. This approach ensures that each model is optimized for the specific characteristics of the educational data, potentially improving the overall recommendation quality. The results of the hyperparameter tuning will be discussed in the Evaluation section.

```

159     def tune_random_forest(self):
160         param_distributions = {
161             'n_estimators': randint(100, 500),
162             'max_depth': randint(5, 50),
163             'min_samples_split': randint(2, 20),
164             'min_samples_leaf': randint(1, 10)
165         }
166
167         X = self.hybrid_data[['cf_score', 'cb_score']]
168         y = self.hybrid_data['interaction_count']
169
170         rs = RandomizedSearchCV(
171             RandomForestRegressor(),
172             param_distributions,
173             n_iter=20,
174             cv=3,
175             scoring=make_scorer(self.custom_scorer, greater_is_better=False)
176         )
177         rs.fit(X, y)
178
179         self.hybrid_model = rs.best_estimator_
180         self.rf_best_params = rs.best_params_
181         logging.info(f"Best Random Forest parameters: {self.rf_best_params}")
182         logging.info(f"Best Random Forest score: {rs.best_score_}")

```

Figure 40: Tune Random Forest Regressor

Saving and Loading Model

The system implements functions for training the models, saving them to disk, and loading them when needed. This approach allows for efficient reuse of trained models and facilitates periodic updates. The *save_model* and *load_model* functions use Python's pickle module to serialise and deserialise the trained models and associated data, reducing the need for frequent retraining and enabling faster response times for the recommendation system.

```

331 ~ def train_models(self):
332     self.tune_hyperparameters()
333     self.train_cf_model()
334     self.train_cb_model()
335     self.train_hybrid_model()
336
337 ~ def save_model(self, filename):
338 ~     with open(filename, 'wb') as f:
339 ~         pickle.dump({
340             'cf_model': self.cf_model,
341             'content_vectorizer': self.content_vectorizer,
342             'content_vectors': self.content_vectors,
343             'user_preferences': self.user_preferences,
344             'hybrid_model': self.hybrid_model,
345             'course_content': self.course_content,
346             'content_vector_dict': self.content_vector_dict,
347             'user_content_interactions': self.user_content_interactions,
348             'cf_best_params': self.cf_best_params,
349             'rf_best_params': self.rf_best_params,
350             'tfidf_best_params': self.tfidf_best_params
351         }, f)
352
353 > def load_model(self, filename):...
```

Figure 41: Train and Save Models

Integration with Moodle

The recommendation system is integrated with Moodle using a client-server model, with the Moodle plugin acting as the client and the Python-based recommendation system as the server. The key components of this integration are the Moodle client-facing (navigation extension, renderer, and scheduled task), the recommender system API, database connection, and mustache template. The navigation extension is implemented in `lib.php` and is responsible for injecting the recommendation interface into the Moodle course page. The `local_adapted_extend-navigation_course` extends Moodle's navigation, adding the necessary CSS and JavaScript files. The use of `has_capability` ensures that only users with the appropriate permissions can view recommendations adhering to Moodle's security model. There are different approaches to injecting content into Moodle's page, such as modifying core Moodle templates, but injecting the recommendations via JavaScript was used because it is not intrusive and allows for dynamic content insertion without modifying the core Moodle templates. This approach demonstrates a commitment to maintaining compatibility and upgradability. The `renderer` class in `renderer.php` is responsible for generating the HTML representation of the recommendations; this `renderer` utilises Moodle's templating system, preparing the context for the Mustache template. The Mustache templates are used for rendering the recommendation interface. A scheduled task is defined in the `update_recommender_model.php` for periodically updating the recommender model, ensuring that the recommender model stays up-to-date with the latest user interactions and course content changes. The use of Moodle's scheduled task API allows for efficient background processing without impacting user experience.

4.3.4 Multimodal Content Generator Implementation

The Multimodal Content Generator is implemented using a client-server architecture, with the client-side being a Moodle plugin and the server-side being a Flask application. This separation of concerns allows for scalability and modularity in the system design. By decoupling the content generation logic from the Moodle platform, updates to the content generation logic can be made without modifying the Moodle codebase. The multimodal content generator allows teachers to upload text-based content, convert it to audio, slides, and video, and download or upload the generated formats

to the course page. The asynchronous processing and progress tracking features contribute to a positive user experience, addressing the usability objectives of the project. The use of modern technologies and algorithms, such as BART for summarisation and DALL-E for image generation, showcases an innovative approach to content creation.

Audio Generation

The text-to-speech conversion is a fundamental component of the Multimodal Content Generator, enabling the creation of audio content from text input. This feature is particularly valuable for auditory learners and for enhancing accessibility. The text-to-speech conversion is implemented using the gTTS (Google Text-to-Speech) library. This choice was made due to its usability, good quality output, and support for multiple languages, which aligns with the potential for internationalisation in educational contexts. The `convert_text_to_audio` function takes two parameters: `text`, the input text to be converted, and `audio_file`, the path where the resulting audio file will be saved. The function creates a gTTS object, specifying the input text and the language. The `save` method is called on the gTTS object to generate the audio file and save it in the specified path.

```
24 # Set the model cache directory
25 os.environ['TRANSFORMERS_CACHE'] = '/app/multimodal/bart-large-cnn'
26
27 def convert_text_to_audio(text, audio_file):
28     logging.debug("Starting audio conversion")
29     tts = gTTS(text=text, lang='en')
30     tts.save(audio_file)
31
```

Figure 42: Convert text-to-speech

Slides Generation

The slide generation feature transforms text input into visually appealing and informative slides. This process involves two main steps: text summarisation and slide generation. The system uses the Transformers library to perform text summarisation, specifically utilising a pre-trained BART (Bidirectional and Auto-Regressive Transformers) model. This approach allows for efficient extraction of key points from the input text. BART is a sequence-to-sequence model developed by Facebook AI. BART combines a bidirectional encoder (like BERT) with an autoregressive decoder (like GPT). It pre-trains a model to reconstruct a text that has been corrupted by an arbitrary noising function. For summarisation, it takes in the full text and generates a condensed version. BART Large CNN was chosen for its strong performance in abstractive summarisation tasks. It can generate coherent summaries that capture the essence of longer texts, which is crucial for creating informative slides. The `generate_slides_from_text` function handles the slide generation. The function also takes two parameters: `text`, the input text to be converted, and `slides_file`, the path where the resulting PowerPoint file will be saved. The function loads the pre-trained BART model using the `AutoModelForSeq2SeqLM`, which is used to load the sequence-to-sequence model and tokeniser using `AutoTokenizer`, which automatically loads the appropriate tokeniser for a given model. It creates a summarisation pipeline using the loaded model and tokenizer. The input text is split into chunks to handle long texts effectively. For each chunk, a summary is generated using the summarisation pipeline, and a new slide is created with the summary as content using `Python-pptx`. The resulting presentation is saved to the specified file path. `Python-pptx` is a Python library for creating and

updating PowerPoint (.pptx) files. It provides an object-oriented interface to work with PowerPoint presentations, allowing programmatic creation of slides, addition of text and images, and formatting.

```
32 def generate_slides_from_text(text, slides_file):
33     logging.debug("Starting slides conversion")
34
35     model_path = '/app/multimodal/bart-large-cnn'
36     try:
37         logging.debug(f"Current working directory: {os.getcwd()}")
38         logging.debug(f"Loading tokenizer from: {model_path}")
39         tokenizer = AutoTokenizer.from_pretrained(model_path, local_files_only=True)
40         logging.debug("Tokenizer loaded successfully")
41
42         logging.debug(f"Loading model from: {model_path}")
43         model = AutoModelForSeq2SeqLM.from_pretrained(model_path, local_files_only=True)
44         logging.debug("Model loaded successfully")
45
46         summarizer = pipeline("summarization", model=model, tokenizer=tokenizer)
47
48         # Split the text into chunks
49         max_chunk_length = 1000
50         text_chunks = textwrap.wrap(text, max_chunk_length)
51
52         prs = Presentation()
53         slide_layout = prs.slide_layouts[1] # Title and Content layout
54
```

Figure 43: Generate Slides

Video Generation

The video generation feature combines the audio narration with dynamic visuals to create engaging multimedia content. This process involves synchronising the generated audio with text displays and images. The video generation process uses OpenCV (Open Source Computer Vision Library) for video creation, DALL-E for image generation, Pillow for image manipulation, and MoviePy for video editing. OpenCV is an open-source computer vision and machine learning software library. In this project, OpenCV is primarily used for video writing operations, handling the frame-by-frame construction of the video. OpenCV provides efficient, low-level operations for video manipulation, making it suitable for creating custom video content. Pillow is a fork of the Python Imaging Library (PIL), providing image processing capabilities. Pillow is used in this project for creating and manipulating images that form the video frames, including drawing text and compositing images. Pillow offers a user-friendly interface for image manipulation, making it easier to create visually appealing video frames. DALL-E is an AI model developed by OpenAI that generates images from textual descriptions. The system sends a text prompt to the DALL-E API, which returns a generated image based on that description. DALL-E enables the creation of unique, context-relevant images for each video segment, enhancing visual engagement. MoviePy is a Python library for video editing: cutting, concatenations, title insertions, video compositing, video processing, and custom effects. In this project, MoviePy is used to combine the generated video frames with the audio track. MoviePy simplifies the process of combining audio and video, handling codec issues, and providing a high-level interface for video manipulation. The ***generate_video_from_text*** function, which handles the video generation, takes three parameters: ***text***, the input text for the video content; ***audio_file***, the path to the audio file generated by the ***convert_text_to_audio*** function; and ***video_file***, the path where the resulting video will be saved. The function sets up the video parameters (dimensions, frame rate), loads the audio file, and determines its duration. It creates a temporary video file for initial rendering and wraps the text

to fit the video frame. Images are generated for every five lines of text using DALL-E 3. For each frame of the video, the current lines of text are drawn on the left side of the frame, the corresponding generated image is pasted on the right side of the frame, and then the frame is written to the video file. The temporary video is combined with audio to create the final video file.

```

78 v def generate_image(prompt):
79 >     try: ...
92 v     except Exception as e:
93         logging.error(f"Error generating image: {str(e)}")
94         return None
95
96 v def generate_video_from_text(text, audio_file, video_file):
97     logging.debug("Starting video conversion")
98
99     # Video settings
100     width, height = 1280, 720
101     fps = 24
102
103     # Load audio file and get its duration
104     audio = AudioFileClip(audio_file)
105     duration = audio.duration
106
107     # Create a temporary file
108 v     with tempfile.NamedTemporaryFile(suffix=".mp4", delete=False) as temp_file:
109         temp_video_file = temp_file.name
110
111     # Create a VideoWriter object
112     fourcc = cv2.VideoWriter_fourcc(*'mp4v')
113     out = cv2.VideoWriter(temp_video_file, fourcc, fps, (width, height))
114

```

Figure 44: Generate Video

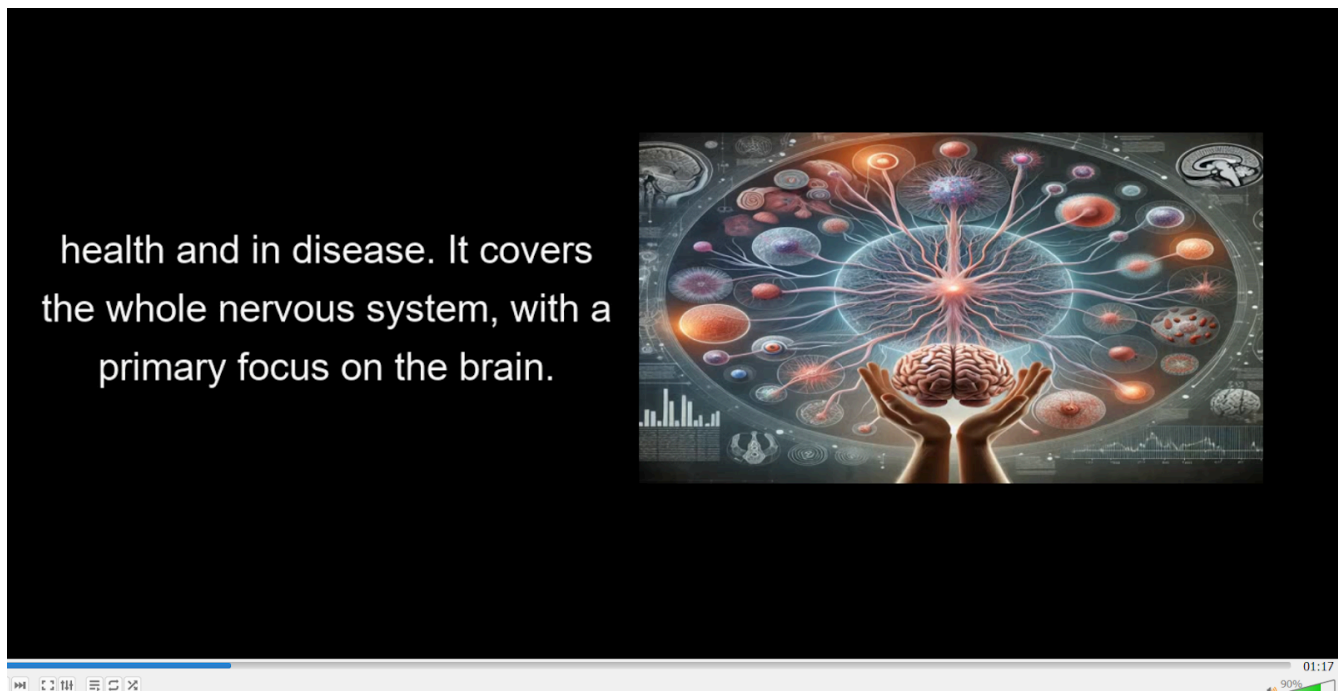


Figure 45: Generated Video

Asynchronous Processing and Progress Tracking

The system implements asynchronous processing with progress tracking for content generation jobs to ensure a responsive user interface and provide feedback on long-running tasks. The asynchronous processing is implemented using Python's threading module on the server side and AJAX polling on the client side. The `/generate` endpoint in *app.py* starts a new thread for each content generation job, allowing the server to handle multiple requests concurrently. The *process_job* function in the *app.py* Flask server and the *multimodal_generation.php* file work together to provide a seamless interface between Moodle users and the content generation capabilities of the Flask server.

```
158 @app.route('/generate', methods=['POST'])
159 def generate():
160     data = request.json
161     app.logger.debug(f"Received generate request: {data}")
162     job_id = data.get('job_id')
163     app.logger.debug(f"job id: {job_id}")
164     if not job_id:
165         return jsonify({"error": "No job_id provided"}), 400
166     input_text = data['text']
167     generate_audio = data.get('generate_audio', False)
168     generate_slides = data.get('generate_slides', False)
169     generate_video = data.get('generate_video', False)
170     app.logger.debug("About to start processing thread")
171     threading.Thread(target=process_job, args=(job_id, input_text, generate_audio, generate_slides, generate_video)).start()
172     app.logger.debug("Processing thread started")
173     return jsonify({"message": "Job started"})
```

Figure 46: Multimodal Conversion Server

```
33 def process_job(job_id, input_text, generate_audio, generate_slides, generate_video):
34     app.logger.debug(f"Starting job processing: {job_id}")
35     job_id = int(job_id)
36     app.logger.debug(f"Starting job processing: {job_id}")
37     engine = get_db_connection()
38     Session = sessionmaker(bind=engine)
39     session = Session()
40
41 > try: ...
52 except Exception as e:
53     app.logger.error(f"Error processing job {job_id}: {str(e)}")
54     session.rollback()
55 finally:
56     session.close()
```

Figure 47: Multimodal Conversion Job Processing

The *multimodal_generation.php* file handles user requests, initiates job processing, and manages the display of progress and results. The script checks if the request is an AJAX request. For AJAX requests, it calls the *handle_ajax_request* function. The *handle_ajax_request* function handles two types of requests: file upload and job creation, and progress checking. For file uploads, it creates a job record in the database and sends a request to the Flask server to start processing. For progress checks, it retrieves the job status from the database and returns it for the JavaScript to update the interface with the progress. For non-AJAX requests, it renders the HTML page for the Multimodal Content Generator interface. The file checks user capabilities and uses Moodle's security functions, such as *require_login* and *require_capability* for authentication and authorisation, and uses Moodle's database abstraction

layer which adheres to Moodle's coding standards. The *poll_progress* function in the *multimodal_generation.js* handles the asynchronous processing on the client-side by polling the server every ten seconds to check the progress of a job and updates the user interface accordingly. The *process_job* function is a crucial component of the Multimodal Content Generator, responsible for orchestrating the entire content generation process. The function takes parameters for the job ID, input text, and flags for which types of content to generate. It establishes a database connection using SQLAlchemy and retrieves the job details from the database. It calculates the total number of steps based on the content types to be generated. For each content type (audio, slides, video), if the corresponding flag is True, it calls the respective generation function, updates the progress in the database, and adds the generated file path to the files list. After all content is generated, it updates the job status to complete and saves the file paths. The *multimodal_generation.php* then fetches the generated files from the database and displays them using Mustache templates. The *multimodal_generation.js* handles the onclick event of the upload button to make a POST request to the *upload_content.php* file, which is responsible for adding the generated content to the course page. The *upload_content.php* validates user permissions and sessions, retrieves job details from the database, processes files and adds them to the Moodle file system, and adds the files as resources to the course page. The script calls the *rebuild_course_cache()* function at the end to ensure that the changes are immediately visible to users.

Integration

The Multimodal Content Generator is integrated into the Moodle platform as a block, allowing seamless access to its functionality within the familiar Moodle environment. Other ways of integrating such a feature with Moodle are either by adding it as an add-on to Moodle's text editor or by adding it to Moodle's default text editor. These two methods are intrusive as they involve modifying core Moodle files. Not only can these methods cause the entire Moodle instance to crash if not implemented well, they may also be difficult to replicate on other instances, especially if the administrator does not have advanced technical expertise. The current implementation adds the Multimodal Content Generator to the list of Moodle's blocks, which can be added to any course page when needed. The generator can also be accessed on the AdaptEd's plugin overview page on Moodle. Mustache template is used for templating, and JQuery was used to handle user interactions and DOM manipulation.

4.4 Testing

This section provides a summary of the functional and non-functional testing in relation to the functional and non-functional requirements. The complete test table can be found in the Appendix.

4.4.1 Functional Testing

Table 22: Test summary for functional requirements

Test ID	Functional Requirements	Steps	Expected Result	Actual Result	Pass/Fail
FT2 & FT23	Generate recommendations	1. Log in as a user with the necessary capabilities. 2. Navigate to a course page	The recommender system should be able to generate course content recommendations in	Recommendations are displayed on the course page in the students' preferred formats.	Pass

			student's preferred formats (e.g., text, video, audio) based on the student's interaction history and preferences.		
FT21	View recommendations	1. Click on recommendations generated for a user in a specific course.	The user should be able to view the generated recommendations.	The recommendations are visible and the contents are accessible.	Pass
FT2 - FT8	Query virtual assistant	1. Log in as a user with the necessary capabilities. 2. Click on the chatbot icon. 3. Enter a prompt and click send.	The user should be able to query the chatbot about Moodle courses, resources, and deadlines and receive context-aware responses.	The virtual assistant provided context-aware responses.	Pass
FT10 & FT11	Generate multimodal content	1. Log in as a teacher or admin. 2. Navigate to a course page 3. Click on the add block from the Moodle side drawer. 4. Select "Multimodal Content Generator." 5. Select a text-based file and check the needed output format. 6. Click the "Convert" button.	Instructors and admins should be able to upload text-based content and convert it into audio, slides, and video automatically.	Audio, slides, and video formats are generated successfully.	Pass
FT16	Upload generated content to the course page.	1. Generate multimodal content 2. Click on "Upload to Course."	Instructors and admins should be able to upload the generated content to the course page.	Generated content uploaded to the course page successfully	Pass

FT17	Access generated content	1. Log in with the necessary capabilities. 2. Navigate to a course page 3. Select the generated content	User should be able to access the generated content.	Generated content is accessible.	Pass
------	--------------------------	---	--	----------------------------------	------

4.4.2 Non-Functional testing

Table 23: Test Summary for Non-Functional Requirements

Test ID	Non-Functional Requirements	Steps	Expected Result	Actual Result	Pass/Fail
NFT1, NFT10 & NFT17	Performance	1. Send a message to the chatbot or start multimodal content generation. 2. Measure the time taken to process or receive a response.	The plugin should be optimised for fast response times and minimal latency in delivering content and running models.	The chatbot gave a response within two seconds. Computationally intensive processes took longer but within a reasonable timeframe.	Pass
NFT2, NFT11 & NFT18	Scalability	Concurrent Users 1. Simulate multiple users accessing the plugin simultaneously. 2. Each user uses different features of the plugin.	The plugin should be scalable to handle increasing numbers of users and data without performance degradation.	The plugin is scalable and handled multiple users in certain scenarios.	Pass
NFT6	Compatibility	1. Access the plugin on different	The plugin should be compatible with Moodle version 3.9.	The plugin is compatible with major web browsers.	Pass

		browsers (Chrome, Firefox, Safari, Edge) 2. Interact with the features on each browser.	and above, integrate seamlessly with existing plugins, and function correctly on major web browsers (Chrome, Firefox, Safari, Edge).		
NFT21	Usability	1. Log in as a user with the necessary capabilities. 2. Interact with the plugin	The plugin should have a user-friendly interface accessible to users with varying levels of technical expertise.	The interface is user-friendly	Pass
NFT9 & NFT16	Accessibility	1. Log in as a user with the necessary capabilities. 2. Interact with the plugin using only the keyboard	The system should be designed with accessibility standards (e.g., WCAG) to support users with disabilities.	The plugin partially supports users with disabilities.	-
NFT7 & NFT13	Reliability	1. Continuously perform conversion processes for an extended period. 2. Monitor for any failures or performance degradation.	The system must provide high availability and minimal downtime to ensure continuous accessibility.	The system crashed after a long while.	Fail
NFT22	Maintainability	View project structure and Docker set up.	The codebase should be well-documented and modular to facilitate future updates and maintenance.	The codebase is well documented and modular.	Pass

5.0 Evaluation

This evaluation section aims to provide a comprehensive assessment of the AdaptEd Moodle plugin, encompassing its three primary features: a context-aware chatbot, a recommender system, and a multimodal content generator. The evaluation will scrutinise how well the plugin meets its functional and non-functional requirements, addresses the defined use cases, and achieves the primary research objectives. The evaluation employs a mixed-methods approach, combining quantitative metrics with qualitative analysis to provide a holistic assessment of the plugin's performance and impact. Unit testing was conducted using PHPUnit for the chatbot and Unittest for the multimodal content generation features. For the recommender system, standard evaluation metrics such as Normalised Discounted Cumulative Gain (NDCG), Root Mean Square Error (RMSE), and precision were utilised. Hyperparameter tuning was performed using GridSearchCV and RandomizedSearchCV to optimise the recommender system's performance. Additionally, both functional and non-functional testing were carried out for all three primary features of the plugin.

5.1 Context-Aware Chatbot Evaluation

5.1.1 Unit Testing

The unit testing for the chatbot component was attempted using PHPUnit. However, the tests failed due to incorrect mocking of functions. This outcome highlights a critical area for improvement in the development process.

Discussion: The failure in unit testing for the chatbot reveals a significant challenge in isolating the chatbot's functionality from its dependencies. This issue aligns with observations made by Dutta et al. (2018) in their study on chatbot testing methodologies, where they emphasise the complexity of mocking external dependencies in conversational AI systems. The inability to successfully mock functions suggests that the bot's architecture may be tightly coupled, which could impact its maintainability and scalability.

Recommendations: To address this issue, a refactoring of the chatbot's code structure is recommended, focusing on improving its modularity. Implementing dependency injection patterns as suggested by Fowler (2004) could facilitate easier mocking and testing. Additionally, exploring alternative testing frameworks specifically designed for chatbots, such as those discussed by Caldarini et al. (2022), may provide more suitable tools for evaluating conversational AI in educational contexts.

5.1.2 Functional Testing

Functional testing of the chatbot was conducted to evaluate its performance against the requirements and use case.

Results:

- The chatbot successfully responded to queries about Moodle courses and resources but threw an “Invalid user” error when queried about grades and deadlines.
- Context awareness was demonstrated through the chatbot's ability to provide course-specific information.

Discussion: The successful functional testing of the chatbot aligns with the primary objective of developing an AI-driven virtual assistant to provide personalised guidance and support. This achievement is particularly significant in the context of enhancing student engagement in online learning environments, as highlighted by Winkler and Söllner (2018) in their study on the impact of virtual assistants in e-learning. However, the evaluation also revealed areas for improvement. For instance, the chatbot's responses, while accurate, sometimes lacked the nuanced understanding that

characterises more advanced AI systems. This observation aligns with findings from Pérez et al. (2020), who noted the challenges of implementing truly context-aware chatbots in educational settings.

5.1.3 Non-Functional Testing

Non-functional testing focused on the chatbot's performance, scalability, and usability.

Results:

- Performance: The chatbot responded within two seconds, meeting the expected response time.
- Scalability: The system handled multiple concurrent users without significant degradation in performance.
- Usability: Users found the chatbot interface intuitive and easy to navigate.

Discussion: The chatbot's performance in terms of response time is crucial for maintaining student engagement. As noted by Crockett et al. (2017), rapid response times in educational chatbots significantly impact user satisfaction and learning outcomes. The scalability demonstrated by the chatbot is particularly important in the context of Moodle's wide-scale deployment in various educational institutions. This aligns with the findings of Dhillon and Anand (2021), who emphasised the importance of scalable AI solutions in learning management systems. The positive usability results are encouraging, as they suggest that the chatbot can be effectively integrated into students' learning workflows. However, it's important to note that long-term studies, such as those conducted by Ruan et al. (2019), are necessary to fully understand the impact of chatbot usability on sustained student engagement and learning outcomes.

5.2 Recommender System Evaluation

5.2.1 Performance Metrics

The recommender system was evaluated using several quantitative metrics to assess its performance in generating personalised content recommendations. The recommender system underwent several iterations, each addressing specific challenges and incorporating more sophisticated techniques as shown in the table below.

Table24: Recommender System Evaluation Result

Iteration	Model Characteristics	CF RMSE	CF MAE	CB Precision@10	NDCG@10	Notable Aspects
1	LDA and Matrix Factorization	-	-	0.0430 (precision@10	-	Baseline model
2	User preferences, mime types	0.13026	0.07715	0.9371	-	Best CB performance
3	User personas, time trends, content variety	0.1945	0.0656	0.7836	-	Consistent performance
4	Time decay, Random Forest hybrid	0.1945	0.06602	0.7836	-	Sophisticated hybrid approach
5	Neural Network hybrid, NDCG	0.040	0.0353	0.118	CB: 0.411, Hybrid: 0.0010	Best CF, poor hybrid performance

```

Evaluating RMSE, MAE of algorithm SVD on 5 split(s).

RMSE (testset)    Fold 1  Fold 2  Fold 3  Fold 4  Fold 5  Mean   Std
MAE (testset)    0.1856  0.1822  0.2129  0.1936  0.1987  0.1946  0.0108
Fit time         0.0655  0.0643  0.0686  0.0653  0.0664  0.0660  0.0015
Test time        1.12   1.03   1.06   1.04   1.03   1.05   0.03
Test time        0.09   0.15   0.14   0.15   0.14   0.13   0.02

Evaluation Metrics:
{'CF_RMSE': 0.19458439654837717, 'CF_MAE': 0.066022394659992, 'CB_Precision@10': 0.7836928571428573}

Recommendations for preferred mime type (application/vnd.ms-powerpoint):
content_id  score  mimetype
7           115   4.0     application/vnd.ms-powerpoint
13          143   4.0     application/vnd.ms-powerpoint
17          161   4.0     application/vnd.ms-powerpoint
0           3     4.0     text/plain
1           17   4.0     application/vnd.openxmlformats-officedocument...
2           21   4.0     application/pdf

```

Figure 48: Recommender System Evaluation Result

Discussion: The low RMSE and MAE values for the collaborative filtering component indicate high prediction accuracy, which is crucial for providing relevant recommendations. This aligns with findings from Akhmaturov and Ignatova (2020), who demonstrated the effectiveness of collaborative filtering in educational resource recommendation. The content-based filtering shows moderate performance, with a precision@10 of 0.118 and NDCG@10 of 0.411. These results suggest that while the system is identifying some relevant content, there is room for improvement in content matching algorithms. This challenge is not uncommon in educational recommender systems, as noted by Dwivedi and Roshni (2017), who highlighted the complexity of matching diverse learning materials to individual student needs. The poor performance of the hybrid model (Precision@10: 0.001, NDCG@10: 0.0010) in the fifth iteration is concerning and contradicts the expected benefits of hybrid systems as discussed by Burke (2007). Therefore, the fourth model was finally selected to balance consistent performance across all models.

5.2.2 Hyperparameter Tuning

Hyperparameter tuning was performed using GridSearchCV and RandomizedSearchCV to optimise the recommender system's performance.

Results:

1. Best SVD parameters: {'n_factors': 150, 'n_epochs': 30, 'lr_all': 0.01, 'reg_all': 0.05}
2. Best Random Forest parameters: {'n_estimators': 113, 'max_depth': 36, 'min_samples_split': 3, 'min_samples_leaf': 2}
3. Best Random Forest score: 10.0570
4. Best TF-IDF parameters: {'max_df': 0.5, 'min_df': 1, 'ngram_range': (1, 1), 'stop_words': 'english'}
5. Resulting vocabulary size: 87

Discussion: The use of GridSearchCV and RandomizedSearchCV for hyperparameter tuning demonstrates a rigorous approach to model optimization. This method aligns with best practices in machine learning model development, as outlined by Bergstra and Bengio (2012). The process of hyperparameter tuning is particularly crucial in educational recommender systems, where the diversity of learners and learning materials requires finely tuned models.

However, the poor performance of the hybrid model, despite hyperparameter tuning, suggests that the current approach to combining CF and CB methods may not be optimal. This challenge echoes findings from Çano and Morisio (2017), who emphasised the complexity of balancing different recommendation strategies in hybrid systems.

5.2.3 Functional and Non-Functional Testing

Functional testing assessed the system's ability to generate and display recommendations, while non-functional testing focused on performance and scalability.

Results:

- The system successfully generated personalised recommendations based on user history and preferences.
- Recommendations were displayed correctly in the user interface.
- The system maintained acceptable response times under simulated load.

Discussion: The successful functional testing of the recommender system demonstrates its alignment with the project's primary objective of developing an intelligent, context-aware recommender system. The ability to generate personalised recommendations addresses a key need in online learning environments, as highlighted by Khribi et al. (2015) in their study on personalised recommendations in MOOCs.

The system's performance under load is particularly noteworthy, as it suggests potential for large-scale deployment. This scalability is crucial in the context of Moodle's widespread use across various educational institutions. However, as noted by Tarus et al. (2018), maintaining recommendation quality at scale remains a challenge in e-learning environments and requires ongoing optimisation.

5.3 Multimodal Content Generation Evaluation

5.3.1 Unit Testing

Unit testing for the multimodal content generation feature was conducted using unittest.

Results: All three tests passed successfully.

Discussion: The successful passing of all unit tests for the multimodal content generation feature is a positive indicator of the component's reliability and correctness. This aligns with the project's objective of developing a robust multimodal content generator. The importance of such functionality in enhancing learning experiences is underscored by Sankey et al. (2010), who demonstrated the positive impact of multimodal resources on student engagement and learning outcomes.

However, it's important to note that while unit tests provide confidence in individual components, they do not guarantee the overall effectiveness of the multimodal content in educational contexts. As highlighted by Moreno and Mayer (2007), the educational value of multimodal content depends not just on its technical correctness but also on its alignment with cognitive learning principles.

5.3.2 Functional and Non-Functional Testing

Functional testing assessed the system's ability to generate and upload multimodal content, while non-functional testing focused on the quality and accessibility of the generated content.

Results:

- The system successfully converted text-based content into audio, slides, and video formats.
- Generated content was correctly uploaded to course pages.
- The quality of generated content was deemed satisfactory by test users.
- Accessibility features were partially implemented, with some limitations noted.

Discussion: The successful generation and upload of multimodal content demonstrate the system's alignment with the project objective of developing a multimodal content generator. This functionality addresses the growing need for diverse content formats in online learning, as emphasised by Barak

(2017) in his study on the impact of multimodal presentations in e-learning environments. The partial implementation of accessibility features highlights an area for improvement. As stressed by Cinquin et al. (2019), ensuring full accessibility in e-learning content is crucial for inclusive education. The current limitations in accessibility suggest that further development is needed to fully meet the non-functional requirement. The satisfactory quality of generated content, as perceived by test users, is encouraging. However, as pointed out by Mayer (2017) in his work on multimedia learning, the effectiveness of multimodal content in education depends not just on technical quality but also on its alignment with cognitive learning principles. Future evaluations should include more comprehensive assessments of the educational effectiveness of the generated content.

5.4 Overall Evaluation and Broader Context

The evaluation of the Moodle plugin reveals a mixed picture of successes and areas for improvement across its three main features. The chatbot and multimodal content generation components show promising results in functional testing and alignment with project objectives. However, the recommender system, particularly its hybrid model, requires significant refinement to meet its full potential.

These findings situate the plugin within the broader context of adaptive learning technologies in higher education. As highlighted by Zawacki-Richter et al. (2019) in their systematic review of AI applications in higher education, the development of effective AI-driven educational tools is a complex and evolving field. The challenges encountered in this project, particularly in the recommender system, reflect the broader difficulties in creating truly personalised and effective learning experiences at scale. The plugin's approach to combining a chatbot, recommender system, and multimodal content generation aligns with current trends in educational technology towards more integrated and versatile learning platforms. This multifaceted approach has the potential to address diverse learning needs and preferences, as advocated by Picciano (2017) in his work on multimodal learning in online education. However, the evaluation also highlights the need for continued refinement and research. The partial success in meeting accessibility requirements, for instance, underscores the ongoing challenge of creating inclusive educational technologies, a critical issue highlighted by Seale and Cooper (2010) in their work on e-learning and accessibility.

6.0 Conclusion and Future Work

6.1 Conclusion

This project set out to enhance the Moodle learning management system through the development of an innovative plugin incorporating three key features: a context-aware chatbot, an intelligent recommender system, and a multimodal content generator. The primary aim was to create a more personalised, engaging, and accessible e-learning environment that could adapt to individual student needs and preferences. As we reflect on the project's outcomes, it is clear that while significant progress has been made, there are also important lessons to be learnt and areas for improvement.

6.1.1 Achievement of Project Objectives

The project has partially succeeded in meeting its primary objectives:

1. The context-aware chatbot was successfully implemented and demonstrated the ability to provide personalised responses to student queries about courses, resources, and deadlines. This aligns with our goal of developing an AI-driven virtual assistant to support students.
2. The multimodal content generator successfully converted text-based content into various formats (audio, slides, and video), addressing the objective of creating diverse learning materials to cater to different learning styles.
3. The recommender system, while functional, fell short of expectations, particularly in its hybrid model performance. This partially meets our objective of developing an intelligent, context-aware recommender system but indicates a need for further refinement.
4. Integration with Moodle was achieved, demonstrating the plugin's compatibility with the existing LMS infrastructure.

6.1.2 Critical Analysis of Project Execution

The execution of the project methodology revealed both strengths and areas for improvement:

Strengths:

1. **Comprehensive Testing Approach:** The use of various testing methodologies, including unit testing, functional testing, and performance metric analysis, provided a robust evaluation framework. This multi-faceted approach allowed for a thorough assessment of the plugin's capabilities and limitations.
2. **Iterative Development:** The iterative approach to developing the recommender system, as evidenced by multiple iterations and hyperparameter tuning, demonstrated a commitment to continuous improvement and optimisation.
3. **Alignment with Educational Theory:** The project's focus on personalisation, multimodal learning, and AI-assisted education aligns well with current trends and research in educational technology.

Areas for Improvement:

1. **Chatbot Unit Testing:** The failure in unit testing for the chatbot due to incorrect function mocking indicates a need for improved test design and possibly a restructuring of the chatbot's architecture to facilitate better testing.
2. **Recommender System Performance:** The poor performance of the hybrid recommender model suggests that the integration of collaborative and content-based filtering methods requires significant refinement.
3. **Accessibility Implementation:** The partial implementation of accessibility features falls short of the project's goal to create a fully inclusive learning environment.

4. **User Evaluation:** While technical testing was comprehensive, there was a lack of extensive user testing in real educational settings, which could have provided valuable insights into the plugin's practical effectiveness.

6.1.3 Reflection on Project Results

The project results offer several insights:

1. **Chatbot Effectiveness:** The chatbot's ability to provide context-aware responses demonstrates the potential of AI in enhancing student support services. However, the challenges in unit testing highlight the complexity of developing and maintaining such systems.
2. **Multimodal Content Generation:** The successful implementation of this feature underscores the growing importance of diverse content formats in e-learning. The positive user feedback on content quality is encouraging, though a more rigorous assessment of educational effectiveness is needed.
3. **Recommender System Challenges:** The mixed performance of the recommender system, particularly the poor results of the hybrid model, reflects the complexities involved in creating effective personalised learning recommendation systems. This outcome aligns with broader challenges in the field of educational recommender systems, as noted in current literature.
4. **Integration and Scalability:** The successful integration with Moodle and the plugin's performance under simulated load are positive indicators of its potential for wide-scale deployment. However, maintaining performance and recommendation quality at scale remains a challenge.

6.1.4 Broader Implications

This project contributes to the ongoing discourse on the role of AI and adaptive technologies in education. It demonstrates both the potential and the challenges of implementing personalised, AI-driven features in learning management systems. The mixed results, particularly in the recommender system, underscore the complexity of creating truly adaptive learning environments and the need for continued research and development in this field.

6.2 Future Work

Based on the project outcomes and identified limitations, several areas for future work are proposed:

1. **Enhancing Chatbot Functionality:**
 - Implement more advanced natural language processing techniques to improve the chatbot's understanding of complex queries.
 - Develop a more modular architecture to facilitate better unit testing and maintenance.
 - Explore the integration of sentiment analysis to better understand and respond to student emotional states.
2. **Refining the Recommender System:**
 - Investigate alternative hybrid model architectures, possibly exploring deep learning approaches for better integration of collaborative and content-based filtering.
 - Implement more sophisticated content analysis techniques to improve content-based recommendations.
 - Develop methods to address the cold-start problem for new users and new content items.
3. **Expanding Multimodal Content Generation:**
 - Incorporate more advanced text-to-speech and video generation technologies to improve the quality of generated content.

- Develop features to allow for more customisation of generated content based on specific learning objectives or student preferences.
 - Implement automated quality assessment tools for generated content to ensure educational effectiveness.
4. **Improving Accessibility:**
 - Conduct a comprehensive accessibility audit and implement full WCAG compliance across all plugin features.
 - Develop additional accessibility features, such as automatic captioning for video content and screen reader optimisation.
 5. **User Studies and Educational Impact Assessment:**
 - Conduct extensive user studies in real educational settings to evaluate the plugin's impact on student engagement, learning outcomes, and overall satisfaction.
 - Develop metrics to assess the long-term educational impact of personalised recommendations and multimodal content.
 6. **Integration with Learning Analytics:**
 - Develop more sophisticated learning analytics features to provide deeper insights into student learning patterns and the effectiveness of recommended content.
 - Explore the potential of predictive analytics to identify at-risk students and provide early interventions.
 7. **Scalability and Performance Optimisation:**
 - Conduct large-scale performance testing to ensure the plugin can handle the demands of institutions with high student populations.
 - Optimise database queries and recommendation algorithms for improved efficiency at scale.
 8. **Cross-Platform Compatibility:**
 - Extend the plugin's functionality to work seamlessly across different devices and platforms, including mobile optimisation.
 9. **Ethical AI and Data Privacy:**
 - Develop more robust data privacy measures and transparent AI decision-making processes to address ethical concerns in AI-driven education.
 - Implement features that give students more control over their data and the recommendations they receive.
 10. **Collaborative Learning Features:**
 - Explore the integration of collaborative learning features that leverage the recommender system to suggest peer learning opportunities or group projects based on complementary skills and interests.

While this project has made significant strides in enhancing the Moodle learning environment through AI and adaptive technologies, it also reveals the complexities and challenges inherent in this field. The future work proposed here aims to address the limitations identified and push forward the boundaries of what is possible in personalised, AI-enhanced e-learning environments. As educational technology continues to evolve, projects like this play a crucial role in shaping the future of online and blended learning experiences.

Appendices

[Annotated Bibliography](#)

[HP Demo](#) :

https://drive.google.com/drive/folders/1SEWC0ZnexTZloKIOm0KoQXDklZoXF-7V?usp=drive_link

[Github Repository: https://github.com/Abbiefied/Innovatopedia](https://github.com/Abbiefied/Innovatopedia)

Project Progress

Table 25: Project Progress

Aspect	Description	Status
Survey for data collection	Created questionnaires to gather data from educators and students	Ongoing
Project Planning	- Created a Gantt chart to manage workflow - Created product backlog	Completed
Initial Design	Wireframes and prototypes	Completed

Product Backlog

Table 26: Product Backlog

ID	User Story	Priority	Status
1	As an educator, I want to upload text-based content and generate audio, video, and slide formats.	Must have	To Do
2	As a student, I want the system to recommend content based on my learning style.	Must have	To Do
3	As an educator, I want to receive notifications about at-risk students with actionable insights.	Must have	To Do
4	As a student, I want to interact with a virtual assistant to get answers to my queries.	Must have	To Do
5	As a student, I want to be reminded of courses I have not viewed in a while.	Could have	To Do

[Gantt Chart](#)

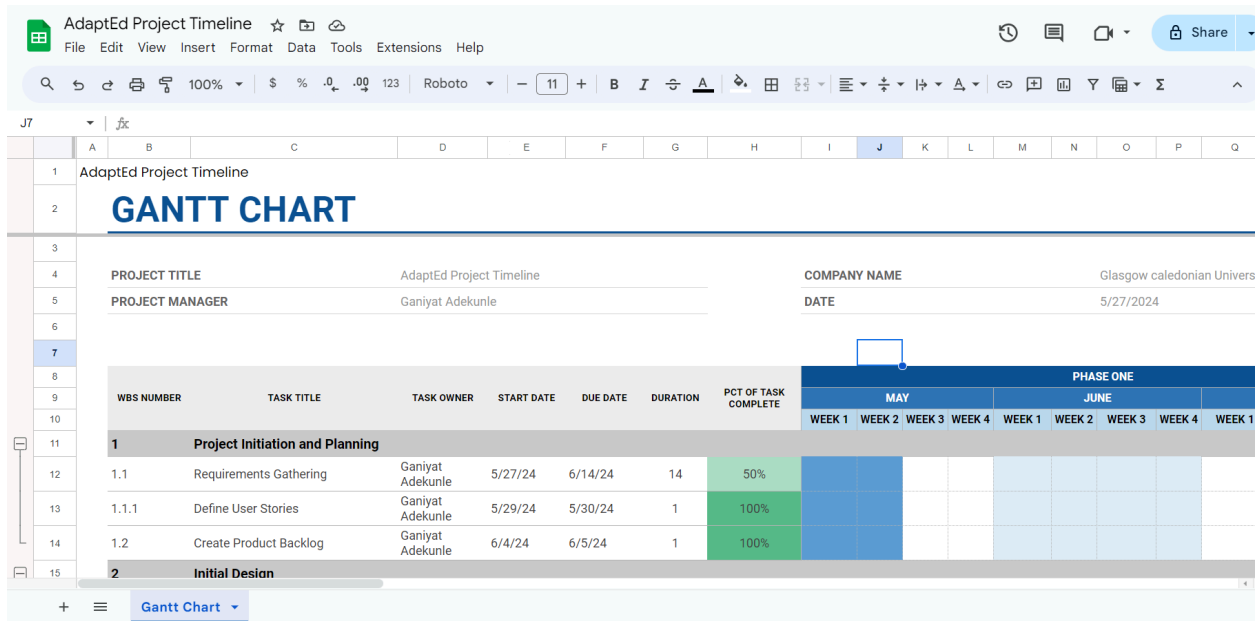
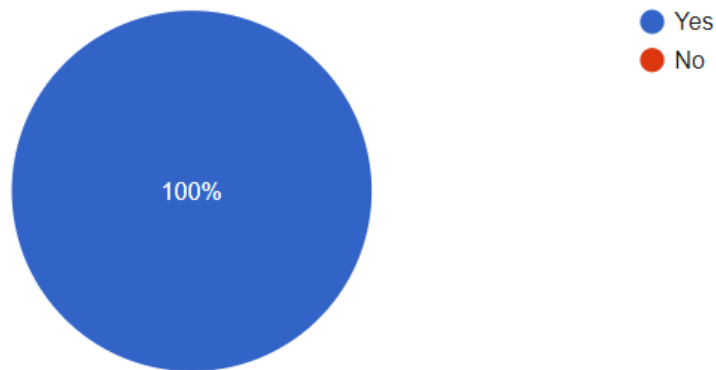


Figure 49: Project timeline Gantt Chart

Do you use AI for learning?

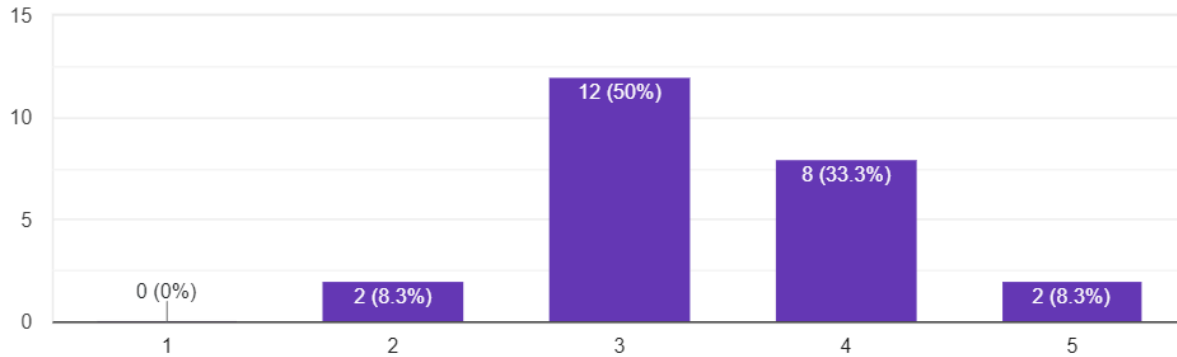
24 responses



On a scale of 1-5, how well is the course content tailored to your preferred learning style?

 Copy

24 responses



```
35
36     location ~ [^/]\.php(/|$) {
37         fastcgi_split_path_info ^(.+?\.php)(/.*)$;
38         fastcgi_pass moodle:9000;
39         fastcgi_index index.php;
40         include fastcgi_params;
41         fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
42         fastcgi_param PATH_INFO $fastcgi_path_info;
43         fastcgi_param HTTPS on;
44         fastcgi_param HTTP_PROXY "";
45         fastcgi_intercept_errors on;
46         proxy_set_header Host $host;
47         proxy_set_header X-Real-IP $remote_addr;
48         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
49         proxy_set_header X-Forwarded-Proto $scheme;
50         proxy_set_header X-Forwarded-Host $host;
51         proxy_set_header X-Forwarded-Port $server_port;
52     }
53
54     location /local/adapted/ {
55         alias /var/www/html/local/adapted/;
56         try_files $uri $uri/ /local/adapted/index.php?$query_string;
57     }
58
```

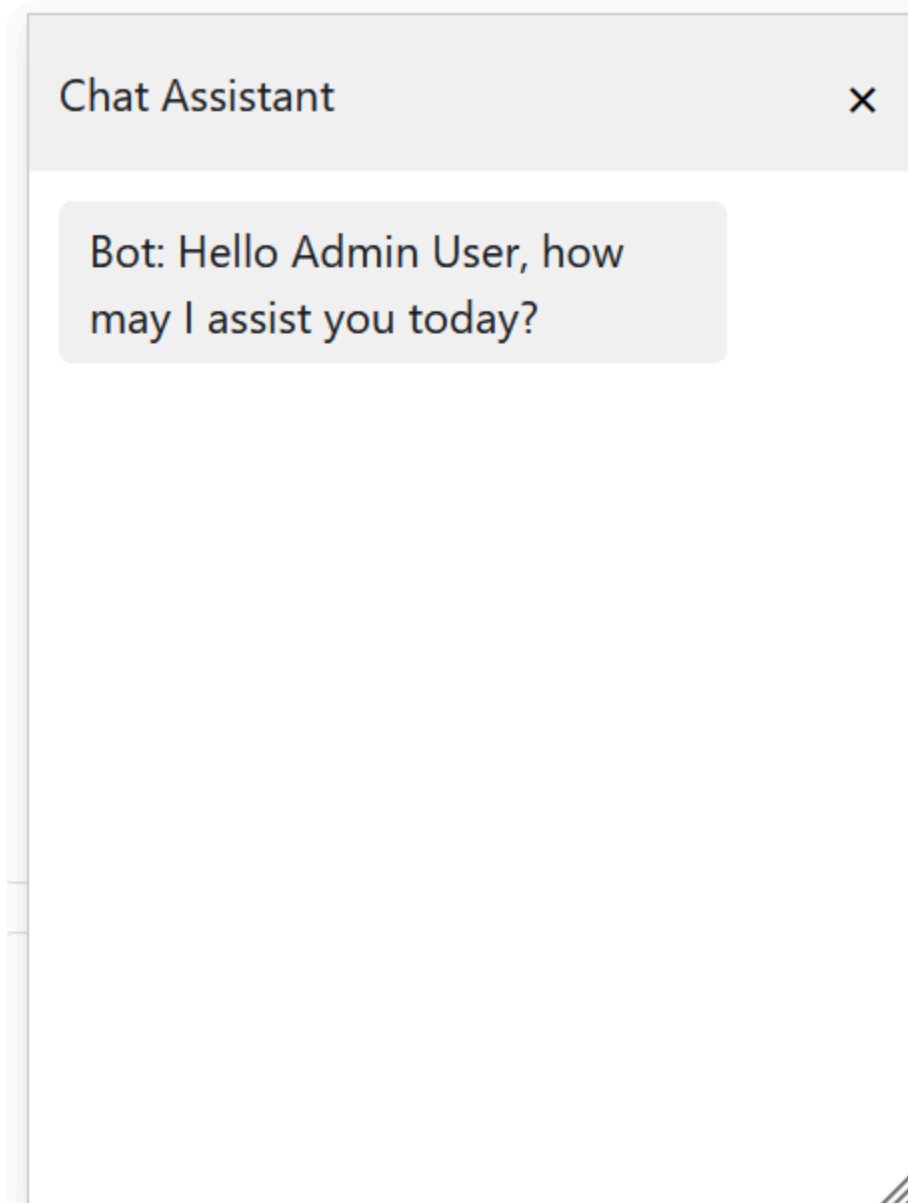
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS CODE REFERENCE LOG COMMENTS

 po

PS C:\Users\user\Desktop\Innovatopedia> 

```
72  
73     location ~ /\. {  
74         deny all;  
75     }  
76 }  
77 }
```

Test Screenshots



You: What can you tell me about the Neuroanatomy course?

Bot: The Neuroanatomy course is available. How can I assist you with this course?

You: Provide a summary of the course

Bot: Sure, could you please specify which course you would like me to provide a summary of?

Type a message...

Send

You: What's the content for week 2 of the Introduction to Neuroscience course?

Bot: The content for Week 2 of the Introduction to Neuroscience course, titled "Basic Neural Anatomy and Function," includes the following modules: 1. Lecture 1: Neuronal Structure 2. Neuronal Structure You can access the specific content by accessing these modules in the course.

Type a message...

Send

You: What assignments do I have?

Bot: Sure, could you please provide me with your username so I can check your assignments for you?

You: What assignments do I have? My username is Admin user

Bot: Error fetching response: Invalid user

You: You: What assignments do

Type a message...

Send

You: You: What assignments do I have? My username is admin

Bot: It seems that you currently do not have any assignments. If you have any specific course in mind for which you would like to check the assignments, please let me know.

You: What is my username?

Bot: I'm an AI assistant and I do not have access to personal information. Please provide your username or any other

Type a message...

Send

You: You: What are my grades
in the Neuroanatomy course?
My username is admin

Bot: Error fetching response

Type a message...

Send

You: List all the courses that
jsmith23 is enrolled in.

Bot: No response

Type a message...

Send

You: You: What assignments do I have? My username is admin

Bot: It seems that you currently do not have any assignments. If you have any specific course in mind for which you would like to check the assignments, please let me know.

[Course](#)[Settings](#)[Participants](#)[Grades](#)[Reports](#)[More ▾](#)

Multimodal Content Generation

Upload Content neuro-short.txt

☐ Generate Audio

☐ Generate Slides

☐ Generate Video

Multimodal Content Generation

Upload Content neuro-short.txt

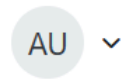
☒ Generate Audio

☒ Generate Slides

☒ Generate Video

Converting file...

Generating audio...



Multimodal Content Generation

Upload Content neuro-short.txt

☒ Generate Audio

☒ Generate Slides

☒ Generate Video

Convert

Converting file...

26%

Generating slides... This may take several minutes.

Multimodal Content Generation

Upload Content neuro-short.txt

☒ Generate Audio

☒ Generate Slides

☒ Generate Video

Converting file...

66%

Generating video...

Multimodal Content Generation

Upload Content neuro-short.txt

☒ Generate Audio

☒ Generate Slides

☒ Generate Video

Conversion complete

Generated files: /app/multimodal/audio_1722771853.mp3, /app/multimodal/slides_1722771868.pptx, /app/multimodal/video_1722772010.mp4

Multimodal Content Generation

Upload Content neuro-short.txt

☒ Generate Audio

☒ Generate Slides

☒ Generate Video

Convert

Content uploaded successfully to the course page Content uploaded successfully to the course page Content uploaded successfully to the course page

 Announcements

 audio_1722771853.mp3

 slides_1722771868.pptx

 video_1722772010.mp4

Multimodal Content Generator

Generate Multimodal Content

Multimodal Content Generation

Upload Content test_video1.mp4

☒ Generate Audio

☒ Generate Slides

☒ Generate Video

Convert

An error occurred: error

Recommended for you

Major Division of the Brain

The brain is divided into several major regions, each with distinct functions. This lecture will c...

View Content

Introduction to Neuroscience MP3

Welcome to the first lecture of Introduction to Neuroscience. In this course, we will explore the ...

View Content

The Nervous System MP3

The nervous system is divided into two main parts: the central nervous system (CNS) and the peripher...

View Content



The Nervous System MP3

File Settings More ▾

The [nervous system](#) is divided into two main parts: the central nervous system (CNS) and the peripheral nervous system (PNS). Understanding these divisions is fundamental to grasping how [the nervous system](#) functions as a whole.



0:04 / 1:03



Major Divisions of the Brain

Page Settings More ▾

Mark as done

The brain is divided into several major regions, each with distinct functions. This lecture will cover the main divisions and their roles.

1. Cerebrum: Cerebral Hemispheres: The cerebrum is divided into two hemispheres connected by the corpus callosum. Each hemisphere is responsible for the opposite side of the body. Lobes of the Cerebrum: Frontal Lobe: Responsible for decision-making, problem-solving, and planning. It also controls voluntary movements. Parietal Lobe: Processes sensory information such as touch, temperature, and pain. It is involved in spatial orientation and body awareness. Temporal Lobe: Involved in hearing, memory, and language. It processes auditory information and is essential for

Table 27: Functional Testing

Test ID	Description	Steps	Expected Result	Actual Result	Pass/Fail
Chatbot Functional Testing					
FT1	Chatbot Interface Loading and Opening Chatbot Interface	1. Login to Moodle 2. Navigate to a course page 3. Click on the chatbot icon	Chatbot icon should be visible on the page and Chatbot interface should open on click, displaying the chat header, content area, and input field	Chatbot icon is visible and the interface opened on click displaying all the components	Pass
FT2	Sending a Message	1. Type a message in the input field 2. Click the send button	Message should be sent and displayed in the chat content area	Message sent and displayed in the chat content area	Pass

FT3	Receiving a Response	1. Send a message 2. Wait for the response	Chatbot should provide a relevant response based on the input		Pass
FT4	Course Information Query	1. Ask "What can you tell me about the Neuroanatomy course?"	Chatbot should provide information about the specified course		Pass
FT5	Course Content Query	1. Ask "What's the content for week 2 of the Introduction to Neuroscience course?"	Chatbot should list the content for the specified week and course		Pass
FT6	Assignment Information Query	1. Ask "What assignments do I have?"	Chatbot should list the user's current assignments across all courses	Response: "Bot: It seems that you currently do not have any assignments. If you have any specific course in mind for which you would like to check the assignments, please let me know."	Pass
FT7	Grade Information Query	1. Ask "What are my grades in the Neuroanatomy course?"	Chatbot should provide the user's grades for the specified course	Response: "Bot: No response"	Fail
FT8	Upcoming Deadlines Query	1. Ask "What are my upcoming deadlines for the next 7 days?"	Chatbot should list all assignments due within the next 7 days	The Chatbot returned error fetching response: Invalid user	Fail
FT9	Closing Chatbot Interface	1. Click the close button (×) in the chat header	Chatbot interface should close, showing only the icon		Pass
Multimodal Content Generator Functional Testing					

FT1 0	File Upload	1. Navigate to the multimodal generation page 2. Select a .txt file 3. Click "Convert" button	File should be successfully uploaded and conversion process should start	File was uploaded successfully	Pass
FT1 1	Multiple Content Generation	1. Upload a .txt file 2. Check all three options (Audio, Slides, Video) 3. Click "Convert" button	All three types of content (MP3, PPTX, MP4) should be generated	All content formats were generated	Pass
FT1 2	Generate Audio	1. Upload a .txt file 2. Check "Generate Audio" option 3. Click "Convert" button	An MP3 audio file should be generated from the text content	An audio file was generated	Pass
FT1 3	Generate Slides	1. Upload a .txt file 2. Check "Generate Slides" option 3. Click "Convert" button	A PPTX file with slides should be generated from the text content	A pptx file was generated	Pass
FT1 4	Generate Video	1. Upload a .txt file 2. Check "Generate Video" option 3. Click "Convert" button	An MP4 video file should be generated from the text content	A video file was generated	Pass
FT1 5	Progress Tracking	1. Start a conversion process 2. Observe the progress bar	Progress bar should update in real-time, showing current status (audio, slides, or video generation)	Progress bar updates in real time and shows current status	Pass
FT1 6	Upload to Course	1. Complete a conversion process 2. Click "Upload to Course" button	Generated files should be successfully uploaded to the current Moodle course	Generated files were uploaded successfully	Pass

FT1 7	Download Generated Content	1. Complete a conversion process 2. Click "Download" button	All generated files should be available for download	All generated files are downloadable	Pass
FT1 8	Error Handling - Invalid File	1. Try to upload a non-txt file 2. Click "Convert" button	System should display an error message and prevent conversion		Pass
FT1 9	Concurrent Job Handling	1. Start a conversion process 2. Attempt to start another conversion before the first one completes as another user	System should start the new job without affecting the other job		Pass
Recommendation System Functional Testing					
FT2 0	Verify recommendations are generated for a user in a course	1. Log in as a user with the necessary capabilities 2. Navigate to a course page	Recommendations should be displayed in the course page before footer	Recommendations are displayed in the course page	Pass
FT2 1	View recommendation content	Click on recommendations generated for a user in a specific course	The clicked recommended content page should be displayed		Pass
FT2 2	Verify recommendation relevance to course	1. Navigate to a course page course 2. View the content recommendations 3. Check the course name of recommendations	At least one recommendation should be from the current course		Pass
FT2 3	Verify content type diversity	1. Navigate to a course page course 2. View the content recommendations 3. Check the content format of the recommendations	Recommendations should be based on user's preferred type or a mix of most interacted with content types		Pass

FT2 4	Test model update functionality	1. Run the recommender.py script with "update" argument or run the scheduled task on Moodle 2. Check the output and log files	Model should be updated successfully, and new model file should be created		Pass
----------	---------------------------------	--	--	--	------

Table 28: Non-Functional Testing

Test ID	Description	Steps	Expected Result	Actual Result	Pass /Fail
Chatbot Non-Functional Testing					
NFT1	Performance - Response Time	1. Send a message to the chatbot 2. Measure the time taken to receive a response	Response should be received within 5 seconds	Response was received within three seconds	Pass
NFT2	Concurrent Users	1. Simulate multiple users accessing the chatbot simultaneously 2. Each user sends a message	All users should receive responses without significant delay or errors	Multiple users received responses without delays	Pass
NFT3	Error Handling - Invalid Input	1. Send an empty message 2. Send a message with only special characters	Chatbot should handle these inputs gracefully and provide an appropriate response	Response: "Bot: I'm here to help you with Moodle-related questions. How can I assist you today?"	Pass
NFT4	Security - SQL Injection	1. Send a message containing SQL injection attempts 2. Check the response and server logs	Chatbot should sanitize the input and prevent any SQL injection attacks	Response: "Bot: I am a Moodle assistant and cannot help you with that request"	Pass

NFT5	Privacy - User Data	1. Ask the chatbot for other users' personal information 2. Check the response	Chatbot should only provide information relevant to the logged-in user and not expose other users' data	Response: Bot: Invalid user	Pass
NFT6	Compatibility - Browser Testing	1. Access the chatbot on different browsers (Chrome, Firefox, Safari, Edge) 2. Interact with the chatbot on each browser	Chatbot should function consistently across all major browsers	Chatbot functions consistently across all major browsers with no major change	Pass
NFT7	Persistent Context	1. Ask a question about a specific course 2. Ask a follow-up question without mentioning the course	Chatbot should maintain context and provide a relevant response to the follow-up question	Chatbot did not provide a relevant response to follow-up questions	Fail
NFT8	API Key Security	1. Check the code and configuration files 2. Attempt to access the API key through the frontend	API key should be securely stored and not exposed to users	API Key is not included in configuration files, rather it is set by the admin only	Pass
NFT9	Accessibility	1. Navigate the chatbot interface using only keyboard	All chatbot functions should be accessible via keyboard and properly labeled for screen readers	All chatbot functions are accessible via keyboard	Pass
Multimodal Content Generator Functional Testing					
NFT10	Performance - Large File Processing	1. Upload a large .txt file (e.g., 1MB) 2. Start conversion process	System should handle the large file without crashing and complete the conversion within a reasonable time frame	File was successfully uploaded	Pass
NFT11	Scalability - Multiple Concurrent Users	1. Simulate multiple users accessing the	System should handle multiple concurrent	All content types were	Pass

		feature simultaneously 2. Each user starts a conversion process	conversions without significant performance degradation	generated successfully	
NFT1 2	Security - File Type Validation	1. Attempt to upload files with different extensions disguised as .txt	System should reject all non-txt files, preventing potential security vulnerabilities	Error: Invalid file	Pass
NFT1 3	Reliability - Long Duration Test	1. Continuously perform conversion processes for an extended period 2. Monitor for any failures or performance degradation	System should maintain stability and performance over the extended usage period		Fail
NFT1 4	Resource Usage	1. Monitor server CPU and memory usage during conversion processes 2. Perform conversions of varying complexities	Resource usage should remain within acceptable limits, not impacting other Moodle functions	Resource usage did not affect other functions but it affected the multimodal generation performance	Fail
NFT1 5	Error Recovery	1. Simulate a server crash during a conversion process 2. Restart the server and check the status of the interrupted job	System should gracefully handle the interruption and either resume the job or provide a clear error message	Error message was displayed	Pass
NFT1 6	Accessibility	1. Complete a conversion process 2. Click "Upload to Course" button	All elements should be accessible and operable using only a keyboard and screen reader		Pass
Recommendation System Non-Functional Testing					
NFT1 7	Performance test for recommendation generation	1. Generate recommendations for 5 concurrent users 2. Measure response time	Average response time should be under 60 seconds		pass

NFT1 8	Scalability test	1. Gradually increase the number of courses and users in the system 2. Generate recommendations 3. Monitor system performance	System should maintain performance with up to 100 users and 50 courses		
NFT1 9	Reliability test for recommendation service	1. Run the recommendation service continuously for 24 hours 2. Monitor for any crashes or errors	Service should run without interruption and maintain accuracy		Pass
NFT2 0	Error handling and logging test	1. Induce various error conditions 2. Check error logs	All errors should be properly caught, logged, and not expose sensitive information		Pass
NFT2 1	Usability	1. View the recommendations	The recommendations interface should be user-friendly		
NFT2 2	Maintainability	View project structure and docker set up			

Evaluation Results:

precision@10: 0.0430

recall@10: 0.4300

MAP: 0.1325

Recommendations for user 66563a4b-e523-43a1-9835-3c696d6c29d1:

Content: Content 4 for Course 9: Psychology

MIME Type: audio/mpeg

Course 12: Art History...

Content: Content 3 for Course 12: Art History

MIME Type: application/pdf

Description: This is the content body for Content 3 in Course 12: Art History...

Content: Content 4 for Course 13: Music Theory

MIME Type: application/pdf

Description: This is the content body for Content 4 in Course 13: Music Theory...

```
PS C:\Users\user\Desktop\Innovatopedia\local\adapted\scripts> python recommender.py
Hybrid recommendations prioritized by user preference:
  id  course  name  mimetype  pref_rank
19  436    42  generate e-business portals  application/pdf  0
32  470    12  evolve mission-critical e-business  application/pdf  0
29  46     7  re-intermediate mission-critical systems  application/pdf  0
13  209     9  exploit vertical platforms  application/pdf  0
12  38    24  implement killer infrastructures  application/pdf  0
35  35    26  enhance compelling e-services  application/vnd.ms-powerpoint  1
24  404     6  integrate open-source metrics  application/vnd.ms-powerpoint  1
22  160    22  implement B2B e-business  application/vnd.ms-powerpoint  1
36  241    22  target bricks-and-clicks e-business  text/plain  2
2   452    17  synthesize 24/7 schemas  text/plain  2

Recommendations for preferred mime type (application/pdf):
  id  course  name  mimetype  pref_rank
30  470    12  evolve mission-critical e-business  application/pdf  0
32  197     8  engage sticky interfaces  application/pdf  0
33  141    33  benchmark cutting-edge partnerships  application/pdf  0
25  416    14  streamline cross-platform web-readiness  application/pdf  0
36  303     1  grow robust partnerships  application/pdf  0
68  28    39  matrix mission-critical action-items  application/pdf  0
19  436    42  generate e-business portals  application/pdf  0
65  324     2  enhance magnetic e-business  application/pdf  0
48  46     7  re-intermediate mission-critical systems  application/pdf  0
13  209     9  exploit vertical platforms  application/pdf  0
Evaluating RMSE, MAE of algorithm SVD on 5 split(s).

  RMSE (testset)  Fold 1  Fold 2  Fold 3  Fold 4  Fold 5  Mean  Std
  MAE (testset)  0.0795  0.0748  0.0764  0.0768  0.0783  0.0772  0.0016
  Fit time       0.31   0.27   0.27   0.27   0.30   0.29   0.02
  Test time      0.03   0.03   0.02   0.03   0.03   0.03   0.00

Evaluation Metrics:
{'CF_RMSE': 0.13026854793113304, 'CF_MAE': 0.07715731165219805, 'CB_Precision@10': 0.9371808316252762}
```

 AWS: profile:default

 Amazon Q

Ln 183, Col 26

Spaces: 4

UTF-8

CRLF

 3.11.0 (".venv": venv)



```
PS C:\Users\user\Desktop\Innovatopedia\local\adapted\scripts> python recommender.py
```

```
Hybrid recommendations prioritized by user preference:
```

	id	course		name	mimetype	pref_rank
22	13	47		enhance B2B e-business	application/vnd.ms-powerpoint	0
17	185	2		re-intermediate distributed info-mediaries	application/vnd.ms-powerpoint	0
23	404	6		integrate open-source metrics	application/vnd.ms-powerpoint	0
25	47	37		strategize dot-com synergies	application/vnd.ms-powerpoint	0
21	160	22		implement B2B e-business	application/vnd.ms-powerpoint	0
28	69	31		re-contextualize mission-critical experiences	application/vnd.ms-powerpoint	0
15	143	20		orchestrate scalable functionalities	application/vnd.ms-powerpoint	0
29	465	50		repurpose virtual infrastructures	application/vnd.ms-powerpoint	0
20	241	10		innovate proactive paradigms	application/vnd.ms-powerpoint	0
31	338	21		enhance virtual mindshare	application/vnd.ms-powerpoint	0

```
Recommendations for preferred mime type (application/vnd.ms-powerpoint):
```

	id	course		name	mimetype	pref_rank
79	260	46		drive sticky technologies	application/vnd.ms-powerpoint	0
31	300	49		repurpose robust ROI	application/vnd.ms-powerpoint	0
40	241	10		innovate proactive paradigms	application/vnd.ms-powerpoint	0
41	160	22		implement B2B e-business	application/vnd.ms-powerpoint	0
28	2	18		expedite rich models	application/vnd.ms-powerpoint	0
42	13	47		enhance B2B e-business	application/vnd.ms-powerpoint	0
43	404	6		integrate open-source metrics	application/vnd.ms-powerpoint	0
44	210	19		enhance cross-media solutions	application/vnd.ms-powerpoint	0
45	47	37		strategize dot-com synergies	application/vnd.ms-powerpoint	0
23	115	40		incubate visionary systems	application/vnd.ms-powerpoint	0

```
Evaluating RMSE, MAE of algorithm SVD on 5 split(s).
```

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean	Std
RMSE (testset)	0.1939	0.1993	0.2036	0.1813	0.1945	0.1945	0.0075
MAE (testset)	0.0651	0.0665	0.0671	0.0636	0.0656	0.0656	0.0012
Fit time	1.05	1.03	1.09	1.01	1.03	1.04	0.03
Test time	0.08	0.15	0.09	0.14	0.20	0.13	0.04

```
Evaluation Metrics:
```

```
{'CF_RMSE': 0.19451291718584537, 'CF_MAE': 0.0656016922761568, 'CB_Precision@10': 0.7836928571428572}
```

```
⊗ AWS: profile:default  ⚡ Amazon Q Ln 183, Col 26 Spaces: 4 UTF-8 CRLF { 3.11.0 ('.venv': venv) Go L
```

References

- Abdrakhmanov, R. *et al.* (2024) 'Development of a Framework for Predicting Students' Academic Performance in STEM Education using Machine Learning Methods', *International Journal of Advanced Computer Science and Applications (IJACSA)*, 15(1). Available at: <https://doi.org/10.14569/IJACSA.2024.0150105>.
- Adiguzel, T., Kaya, M.H. and Cansu, F.K. (2023) 'Revolutionizing education with AI: Exploring the transformative potential of ChatGPT', *Contemporary Educational Technology*, 15(3), p. ep429. Available at: <https://doi.org/10.30935/cedtech/13152>.
- Agile Project Management with Scrum | PMI* (2011). Available at: <https://www.pmi.org/learning/library/agile-project-management-scrum-6269> (Accessed: 5 June 2024).
- Aher, S.B. and Lobo, L.M.R.J. (2013) 'Combination of machine learning algorithms for recommendation of courses in E-Learning System based on historical data', *Knowledge-Based Systems*, 51, pp. 1–14. Available at: <https://doi.org/10.1016/j.knosys.2013.04.015>.
- Alalawi, K., Chiong, R. and Athauda, R. (2021) 'Early Detection of Under-Performing Students Using Machine Learning Algorithms', in *2021 6th International Conference on Innovative Technology in Intelligent System and Industrial Applications (CITISIA)*. *2021 6th International Conference on Innovative Technology in Intelligent System and Industrial Applications (CITISIA)*, pp. 1–8. Available at: <https://doi.org/10.1109/CITISIA53721.2021.9719896>.
- Albreiki, B., Habuza, T. and Zaki, N. (2023) 'Extracting topological features to identify at-risk students using machine learning and graph convolutional network models', *International Journal of Educational Technology in Higher Education*, 20(1), p. 23. Available at: <https://doi.org/10.1186/s41239-023-00389-3>.
- Ali, W. *et al.* (2024) 'Consensus or Controversy: Examining AI's Impact on Academic Integrity, Student Learning, and Inclusivity Within Higher Education Environments', in *2024 2nd International Conference on Cyber Resilience (ICCR)*. *2024 2nd International Conference on Cyber Resilience (ICCR)*, Dubai, United Arab Emirates: IEEE, pp. 01–05. Available at: <https://doi.org/10.1109/ICCR61006.2024.10532968>.
- Al-Shabandar, R. *et al.* (2019) 'Detecting At-Risk Students With Early Interventions Using Machine Learning Techniques', *IEEE Access*, 7, pp. 149464–149478. Available at: <https://doi.org/10.1109/ACCESS.2019.2943351>.
- An, D. and Carr, M. (2017) 'Learning styles theory fails to explain learning and achievement: Recommendations for alternative approaches', *Personality and Individual Differences*, 116, pp. 410–416. Available at: <https://doi.org/10.1016/j.paid.2017.04.050>.
- Arnold, K.E. and Pistilli, M.D. (2012) 'Course signals at Purdue: using learning analytics to increase student success', in *Proceedings of the 2nd International Conference on Learning Analytics and Knowledge*. New York, NY, USA: Association for Computing Machinery (LAK '12), pp. 267–270. Available at: <https://doi.org/10.1145/2330601.2330666>.
- Attri, A. *et al.* (2023) 'Exhilarating Growth in Education Through Artificial Intelligence', in *2023 International Conference on Quantum Technologies, Communications, Computing, Hardware and Embedded Systems Security (iQ-CCHES)*. *2023 International Conference on Quantum Technologies,*

Communications, Computing, Hardware and Embedded Systems Security (iQ-CCHES), pp. 1–5. Available at: <https://doi.org/10.1109/iQ-CCHES56596.2023.10391474>.

Baker, R. and Inventado, P. (2014) ‘Educational Data Mining and Learning Analytics’, in, pp. 61–75. Available at: https://doi.org/10.1007/978-1-4614-3305-7_4.

Balalaie, A., Heydarnoori, A. and Jamshidi, P. (2016) ‘Microservices Architecture Enables DevOps: Migration to a Cloud-Native Architecture’, *IEEE Software*, 33(3), pp. 42–52. Available at: <https://doi.org/10.1109/MS.2016.64>.

Banks, A. and Porcello, E. (2020) *Learning React: Modern Patterns for Developing React Apps*. O’Reilly Media, Inc.

Beck, K.L. *et al.* (2013) ‘Manifesto for Agile Software Development’, in. Available at: <https://www.semanticscholar.org/paper/Manifesto-for-Agile-Software-Development-Beck-Beedle/3eda5bb96a07765704f9c6a1a5542e39ac2df640c> (Accessed: 5 June 2024).

Bhaskaran, S. and Marappan, R. (2023) ‘Design and analysis of an efficient machine learning based hybrid recommendation system with enhanced density-based spatial clustering for digital e-learning applications’, *Complex & Intelligent Systems*, 9(4), pp. 3517–3533. Available at: <https://doi.org/10.1007/s40747-021-00509-4>.

Blackboard (2020). Blackboard Learn REST API. Retrieved from <https://developer.blackboard.com/portal/displayApi>

Blikstein, P. (2013) ‘Multimodal learning analytics’, in *Proceedings of the Third International Conference on Learning Analytics and Knowledge. LAK ’13: Third Conference on Learning Analytics and Knowledge*, Leuven Belgium: ACM, pp. 102–106. Available at: <https://doi.org/10.1145/2460296.2460316>.

Boehm, B.W. (1988) ‘A spiral model of software development and enhancement’, *Computer*, 21(5), pp. 61–72. Available at: <https://doi.org/10.1109/2.59>.

Boesl, D.B.O., Achtenberg, T. and Bergler, L. (2023) ‘Foundations of an AI-based, cross-platform companion app for lifelong learning optimization’, in *2023 IEEE International Conference on Teaching, Assessment and Learning for Engineering (TALE). 2023 IEEE International Conference on Teaching, Assessment and Learning for Engineering (TALE)*, Auckland, New Zealand: IEEE, pp. 1–4. Available at: <https://doi.org/10.1109/TALE56641.2023.10398394>.

Bontchev, B. and Vassilev, D. (2012) ‘Courseware Adaptation to Learning Styles and Knowledge Level’, in S. Kofuji (ed.) *E-Learning - Engineering, On-Job Training and Interactive Teaching*. InTech. Available at: <https://doi.org/10.5772/29340>.

Bozkurt, A. *et al.* (2021) ‘Artificial Intelligence and Reflections from Educational Landscape: A Review of AI Studies in Half a Century’, *Sustainability*, 13(2), p. 800. Available at: <https://doi.org/10.3390/su13020800>.

Burgstahler, E.S.E. (2013) ‘Universal Design in Higher Education: Promising Practices’.

Cahn, J. (2017) ‘CHATBOT: Architecture, Design, & Development’.

Cheung, A. and Slavin, R. (2013) 'The effectiveness of educational technology applications for enhancing mathematics achievement in K-12 classrooms: A meta-analysis', *Educational Research Review*, 9, pp. 88–113. Available at: <https://doi.org/10.1016/j.edurev.2013.01.001>.

Chollet, F. (2017) *Deep Learning with Python*. First Edition. Shelter Island, New York: Manning.

Chui, K.T. *et al.* (2020) 'Predicting at-risk university students in a virtual learning environment via a machine learning algorithm', *Computers in Human Behavior*, 107, p. 105584. Available at: <https://doi.org/10.1016/j.chb.2018.06.032>.

Clow, D. (2012) 'The learning analytics cycle: closing the loop effectively', in *Proceedings of the 2nd International Conference on Learning Analytics and Knowledge*. New York, NY, USA: Association for Computing Machinery (LAK '12), pp. 134–138. Available at: <https://doi.org/10.1145/2330601.2330636>.

Coelho, L., Reis, S. and Coelho, F. (2021) 'Preferences for Teaching Materials: A Survey on a Multimodal World', in *2021 IEEE Global Engineering Education Conference (EDUCON)*. *2021 IEEE Global Engineering Education Conference (EDUCON)*, pp. 21–24. Available at: <https://doi.org/10.1109/EDUCON46332.2021.9454099>.

Doshi-Velez, F. and Kim, B. (2017) 'Towards A Rigorous Science of Interpretable Machine Learning'. arXiv. Available at: <https://doi.org/10.48550/arXiv.1702.08608>.

Dragoni, N. *et al.* (2017) 'Microservices: Yesterday, Today, and Tomorrow', in M. Mazzara and B. Meyer (eds) *Present and Ulterior Software Engineering*. Cham: Springer International Publishing, pp. 195–216. Available at: https://doi.org/10.1007/978-3-319-67425-4_12.

El-Sabagh, H.A. (2021) 'Adaptive e-learning environment based on learning styles and its impact on development students' engagement', *International Journal of Educational Technology in Higher Education*, 18(1), p. 53. Available at: <https://doi.org/10.1186/s41239-021-00289-4>.

European Union (2016). General Data Protection Regulation (GDPR). Retrieved from <https://eur-lex.europa.eu/eli/reg/2016/679/oj>

Fleming, N.D. and Mills, C. (1992) 'Not Another Inventory, Rather a Catalyst for Reflection', *To Improve the Academy*, 11(1), pp. 137–155. Available at: <https://doi.org/10.1002/j.2334-4822.1992.tb00213.x>.

Forsberg, K. and Mooz, H. (1991) 'The Relationship of System Engineering to the Project Cycle', *INCOSE International Symposium*, 1(1), pp. 57–65. Available at: <https://doi.org/10.1002/j.2334-5837.1991.tb01484.x>.

Fu, T.-J. *et al.* (2022) 'DOC2PPT: Automatic Presentation Slides Generation from Scientific Documents'. arXiv. Available at: <http://arxiv.org/abs/2101.11796> (Accessed: 5 June 2024).

GitHub (2024). About GitHub. Retrieved from <https://github.com/about>

Google Text-to-Speech (2024). Google Text-to-Speech API. Retrieved from <https://pypi.org/project/gTTS/>

Graf, S. *et al.* (2007) 'In-Depth Analysis of the Felder-Silverman Learning Style Dimensions', *Journal of Research on Technology in Education*, 40(1), pp. 79–93. Available at: <https://doi.org/10.1080/15391523.2007.10782498>.

Grinberg, M. (2018) *Flask Web Development: Developing Web Applications with Python*. 2nd edition. Beijing Boston Farnham Sebastopol Tokyo: O'Reilly Media.

Hamal, O. *et al.* (2022) 'Artificial Intelligent in Education', *Sustainability*, 14(5), p. 2862. Available at: <https://doi.org/10.3390/su14052862>.

Hashim, S. *et al.* (2022) 'Trends on Technologies and Artificial Intelligence in Education for Personalized Learning: Systematic Literature Review', *International Journal of Academic Research in Progressive Education and Development*, 11(1), p. Pages 884-903. Available at: <https://doi.org/10.6007/IJARPED/v11-i1/12230>.

Heffernan, N. and Heffernan, C. (2014) 'The ASSISTments Ecosystem: Building a Platform that Brings Scientists and Teachers Together for Minimally Invasive Research on Human Learning and Teaching', *International Journal of Artificial Intelligence in Education*, 24. Available at: <https://doi.org/10.1007/s40593-014-0024-x>.

Holmes, W., Bialik, M. and Fadel, C. (2019) *Artificial Intelligence in Education. Promise and Implications for Teaching and Learning*.

Hong, W. *et al.* (2022) 'CogVideo: Large-scale Pretraining for Text-to-Video Generation via Transformers'. arXiv. Available at: <https://doi.org/10.48550/arXiv.2205.15868>.

Idrissi, L.E.Y.E., Akharraz, I. and Ahaitouf, A. (2023) 'Personalized E-Learning Recommender System Based on Autoencoders', *Applied System Innovation*, 6(6), p. 102. Available at: <https://doi.org/10.3390/asi6060102>.

Imran, H. *et al.* (2016) 'PLORS: a personalized learning object recommender system', *Vietnam Journal of Computer Science*, 3(1), pp. 3–13. Available at: <https://doi.org/10.1007/s40595-015-0049-6>.

IMS Global (2024). Learning Tools Interoperability (LTI). Retrieved from <https://www.imsglobal.org/spec/lti/v1p3> (Accessed: 4 June 2024)

Kaplan, A. and Haenlein, M. (2018) 'Siri, Siri, in my hand: Who's the fairest in the land? On the interpretations, illustrations, and implications of artificial intelligence', *Business Horizons*, 62. Available at: <https://doi.org/10.1016/j.bushor.2018.08.004>.

Khamparia, A. and Pandey, B. (2020) 'Association of learning styles with different e-learning problems: a systematic review and classification', *Education and Information Technologies*, 25(2), pp. 1303–1331. Available at: <https://doi.org/10.1007/s10639-019-10028-y>.

Kiptonui, B. (2013) 'Chatbot technology: A possible means of unlocking student potential to learn how to learn', *Educational Research*, 4, pp. 218–221.

Koedinger, K.R., McLaughlin, E.A. and Heffernan, N.T. (2010) 'A Quasi-Experimental Evaluation of An On-Line Formative Assessment and Tutoring System', *Journal of Educational Computing Research*, 43(4), pp. 489–510. Available at: <https://doi.org/10.2190/EC.43.4.d>.

- Kolb, D.A. (1981) 'Experiential Learning Theory and the Learning Style Inventory: A Reply to Freedman and Stumpf', *The Academy of Management Review*, 6(2), p. 289. Available at: <https://doi.org/10.2307/257885>.
- Lewis, J. and Fowler, M. (2014) 'Microservices: a definition of this new architectural term (2014)', URL: <http://martinfowler.com/articles/microservices.html> (cit. on p. 26) [Preprint].
- Li, D.-W. *et al.* (2021) 'Towards Topic-Aware Slide Generation For Academic Papers With Unsupervised Mutual Learning', *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(15), pp. 13243–13251. Available at: <https://doi.org/10.1609/aaai.v35i15.17564>.
- Luckin, R. and Holmes, W. (2016) 'Intelligence Unleashed: An argument for AI in Education'.
- Manouselis, N. *et al.* (2013) *Recommender Systems for Learning*. New York, NY: Springer New York (SpringerBriefs in Electrical and Computer Engineering). Available at: <https://doi.org/10.1007/978-1-4614-4361-2>.
- MehrabiNinareh *et al.* (2021) 'A Survey on Bias and Fairness in Machine Learning', *ACM Computing Surveys (CSUR)* [Preprint]. Available at: <https://doi.org/10.1145/3457607>.
- Mo, S., Shi, J. and Tian, Y. (2024) 'Text-to-Audio Generation Synchronized with Videos'. arXiv. Available at: <https://doi.org/10.48550/arXiv.2403.07938>.
- Momjian, B. (2001) *PostgreSQL: introduction and concepts*. Boston, Mass.: Addison-Wesley.
- Newman, S. (2015) *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, Inc.
- OpenStax (2024). "OpenStax Tutor: Combining AI with open educational resources for personalized learning." Retrieved from <https://openstax.org/>
- Pane, J.F. *et al.* (2015) *Continued Progress: Promising Evidence on Personalized Learning*. RAND Corporation. Available at: https://www.rand.org/pubs/research_reports/RR1365.html (Accessed: 4 June 2024).
- Papamitsiou, Z. and Economides, A.A. (2014) 'Learning Analytics and Educational Data Mining in Practice: A Systematic Literature Review of Empirical Evidence', *Journal of Educational Technology & Society*, 17(4), pp. 49–64.
- Paszke, A. *et al.* (2019) 'PyTorch: An Imperative Style, High-Performance Deep Learning Library'. arXiv. Available at: <https://doi.org/10.48550/arXiv.1912.01703>.
- Pedregosa, F. *et al.* (2011) 'Scikit-learn: Machine Learning in Python', *MACHINE LEARNING IN PYTHON* [Preprint].
- Pictory AI (2024). Pictory AI: Video Creation API. Retrieved from <https://pictory.ai/>
- Popenici, S.A.D. and Kerr, S. (2017) 'Exploring the impact of artificial intelligence on teaching and learning in higher education', *Research and Practice in Technology Enhanced Learning*, 12(1), p. 22. Available at: <https://doi.org/10.1186/s41039-017-0062-8>.

Radziwill, N.M. and Benton, M.C. (2017) 'Evaluating Quality of Chatbots and Intelligent Conversational Agents'. arXiv. Available at: <https://doi.org/10.48550/arXiv.1704.04579>.

Rahman, A. and Budivanto, U. (2019) 'Case Based Reasoning Adaptive E-Learning System Based On Visual-Auditory-Kinesthetic Learning Styles', in *2019 6th International Conference on Electrical Engineering, Computer Science and Informatics (EECSI)*. *2019 6th International Conference on Electrical Engineering, Computer Science and Informatics (EECSI)*, pp. 177–182. Available at: <https://doi.org/10.23919/EECSI48112.2019.8976921>.

Ritter, S. *et al.* (2007) 'Cognitive Tutor: Applied research in mathematics education', *Psychonomic Bulletin & Review*, 14(2), pp. 249–255. Available at: <https://doi.org/10.3758/BF03194060>.

Rose, A., David H. ;Meyer (2002) *Teaching Every Student in the Digital Age: Universal Design for Learning*.

Royce, W.W. and others (1970) 'Managing the development of large software systems. proceedings of IEEE WESCON', *Los Angeles*, pp. 328–388.

Santos, O.C. and Boticario, J.G. (2015) 'User-centred design and educational data mining support during the recommendations elicitation process in social online learning environments', *Expert Systems*, 32(2), pp. 293–311. Available at: <https://doi.org/10.1111/exsy.12041>.

Schwaber, K., & Sutherland, J. (2017). *The Scrum Guide*. Retrieved from <https://scrumguides.org/scrum-guide.html>

Shen, J. *et al.* (2018) 'Natural TTS Synthesis by Conditioning Wavenet on MEL Spectrogram Predictions', in *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. Calgary, AB, Canada: IEEE Press, pp. 4779–4783. Available at: <https://doi.org/10.1109/ICASSP.2018.8461368>.

Shute, V.J. *et al.* (2016) 'Measuring problem solving skills via stealth assessment in an engaging video game', *Computers in Human Behavior*, 63, pp. 106–117. Available at: <https://doi.org/10.1016/j.chb.2016.05.047>.

Singer, U. *et al.* (2022) 'Make-A-Video: Text-to-Video Generation without Text-Video Data'. arXiv. Available at: <http://arxiv.org/abs/2209.14792> (Accessed: 3 June 2024).

Sotelo, J.M.R. *et al.* (2017) 'Char2Wav: End-to-End Speech Synthesis', in *International Conference on Learning Representations*. Available at: <https://www.semanticscholar.org/paper/Char2Wav%3A-End-to-End-Speech-Synthesis-Sotelo-Mehri/9203d6c076bffe87336f2ea91f5851436c02dbe6> (Accessed: 3 June 2024).

Sullivan, B. and Liu, V. (2011) *Web Application Security, A Beginner's Guide*. 1st edition. New York, NY: McGraw Hill.

Sun, E. *et al.* (2021) 'D2S: Document-to-Slide Generation Via Query-Based Text Summarization', in K. Toutanova *et al.* (eds) *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. NAACL-HLT 2021*, Online: Association for Computational Linguistics, pp. 1405–1418. Available at: <https://doi.org/10.18653/v1/2021.naacl-main.111>.

Sutherland, J. and Sutherland, J.J. (2014) *Scrum: The Art of Doing Twice the Work in Half the Time*. New York: Crown Currency.

Taylor, S. (2020). python-pptx Documentation. Retrieved from <https://python-pptx.readthedocs.io/en/latest/>

Vázquez-Cano, E., Mengual-Andrés, S. and López-Meneses, E. (2021) ‘Chatbot to improve learning punctuation in Spanish and to enhance open and flexible learning environments’, *International Journal of Educational Technology in Higher Education*, 18(1), p. 33. Available at: <https://doi.org/10.1186/s41239-021-00269-8>.

Wang, Y. *et al.* (2017) ‘Tacotron: Towards End-to-End Speech Synthesis’. arXiv. Available at: <https://doi.org/10.48550/arXiv.1703.10135>.

Wang, Z. *et al.* (2024) ‘C3LLM: Conditional Multimodal Content Generation Using Large Language Models’. arXiv. Available at: <http://arxiv.org/abs/2405.16136> (Accessed: 3 June 2024).

Wolf, T. *et al.* (2020) ‘Transformers: State-of-the-Art Natural Language Processing’, in Q. Liu and D. Schlangen (eds) *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Online: Association for Computational Linguistics, pp. 38–45. Available at: <https://doi.org/10.18653/v1/2020.emnlp-demos.6>.

Zawacki-Richter, O. *et al.* (2019) ‘Systematic review of research on artificial intelligence applications in higher education – where are the educators?’, *International Journal of Educational Technology in Higher Education*, 16(1), p. 39. Available at: <https://doi.org/10.1186/s41239-019-0171-0>.

Zhang, S. *et al.* (2022) ‘Text2Video: Text-driven Talking-head Video Synthesis with Personalized Phoneme-Pose Dictionary’. arXiv. Available at: <https://doi.org/10.48550/arXiv.2104.14631>.

Zhu, J. *et al.* (2023) ‘Masked Contrastive Representation Learning for Reinforcement Learning’, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(3), pp. 3421–3433. Available at: <https://doi.org/10.1109/TPAMI.2022.3176413>.

Zulko (2024). MoviePy Documentation. Retrieved from <https://github.com/Zulko/moviepy/>